QIKVRT / ALLES
Vollständige Anwendung 05.10.2026 · S1 OPEN
Initiator Ingolf Lohmann · Implementation OpenAI Codex GPT-6 · Artificial Contributor visible

Commit historisch: c6a53e200c4835a4f871d8496feb81d953610831
S1 Commit: 3bf52747bc94de847d390ffacabecae183aee588
Tree: 96c7bb691675dcd15bb5045871201adb400074f0
Manifest SHA-256: db5255c2a363e8fe57abd9ce1c1f6b31532c4a12577a2d81454fdd36586f6edf (S1 Scheibenhard Freeze)
Archive SHA-256: 6b7fdc19b8785d48b52d4d6249fb493e5ed357fd09cb12b96aa3a5ae63a9c2a8
CI Run: 37257971138 · Firefox ESR 153.4.0 · 4x 127.0.0.1 Loopback 48925/58213/50617/58380

INHALT · ALLES

Teil I · Fachartikel Softwaremodernisierung (16 Seiten)
S.1-16: Modernisierung als überprüfbare Wirkung, Proxy-Bug, Xvfb-Lock, IPv6, TEMDD, QXT2, Universal Terminal,
Metatransistor 32 Lanes, Assimilation ChatGPT/Meta AI/Copilot, Robotik ROS2/DDS, VPN RFC4301,
HA Split-Brain etcd, Digital Twins NIST IR 8356 / OPC UA, 86 Skills, Quellenprüfbarkeit

Teil II · Wissenschaftlicher Quellen- und Nachweisband (99 Seiten)
S01 · Quellenwiedergewinnung · F089 RECOVERED_TERMINAL.md · 44 historische Original-Blobs bytegenau
S02 · Fehler, kausale Diagnose, Reparaturen · F090 S1_PRODUCTIVE_TERMINAL_SOURCE_RECOVERY · NOT PERFORMED
S03 · CI Receipts · Firefox ESR 153.4.0, 4 Loopback, executable SHA-256, observed_at 2026-10-05T03:07:12Z
S04 · Copilot Instructions · F091 · Evidence > Chat Memory
S05 · TEMDD v0.1 · F092 · T01-T12 Invarianten, 7-Layer Contract, TRANSPORT_ACK != EFFECT_ACK
S06 · TEMDD v1 · F093 · T13-T16 Semantic Contract, v1_adapter
S07 · Universal Transputer · F094 qikvrt_transputer.h QXT2 172 Byte Route, chunk MAX-ROUTE, max 16 chunks
S08 · Universal Hardware Tile · F095 metatransistor_tile.vhd normalize_state HOLD/CONTINUE 531441 Cases
S09 · Patent Disclosure Boundary · F096 · kein Claim Chart
S10 · Self-Host Runtime · F097-F106 · owner-only 0700, WAL, binding.json fsynced, ingress.sock 0600 SO_PEERCRED, socketpair 32 observers
S11 · Lizenzen · F097 README, PolyForm Noncommercial 1.0.0, CC BY-NC-ND 4.0, CC BY-NC 4.0, Apache 2.0

Inventar F001-F185
F001-F030 deploy/universal-terminal · Dockerfile, entrypoints, nginx, qikvrt_ip_bootstrap.c, qikvrt_smtpd.py
F031-F032 docs/terminal/temdd · index.html
F033-F044 browser/firefox · manifest, background, content, options, terminal.css, authenticated_delivery
F045-F080 runtime/self-host · ledger, terminal, monitor, novnc vnc.html rfb.js
F081-F112 src/tools/tests · effect_ack_http_terminal 30823B, qikvrt_self_host 28982B, self-host.mjs, mesh_monitor_readback, bootstrap-runtime
F113-F118 state/work_units · SOURCE_RECOVERY, SELF_HOSTED_NODE_PACKAGE
F119-F124 LICENSES · NOTICE, Apache 2.0, PolyForm, CC, next/core/ARCHITECTURE.md
F125-F185 next/core · Makefile C90, stdio adapter, qikvrt_bus.h/types/wire, exchange.c kernel.c sha256 wire, hardware/vhdl pkg/entity/tile/tb, tests

Anhang A · Volltext Listings A1-A9 · RECOVERED_TERMINAL.md, SOURCE_RECOVERY.json, temdd parser, EBNF, exchange, kernel, metatransistor
Anhang B · Sealed S1 Adapter B1-B13 · B1 Ledger 0700 WAL, B2 Effect-Ack loopback Structured Field HMAC TTL 120s, B3 Self-Host freeze/verify/sealed, B4 TEMDD Parser, B
Anhang C · CI Receipts · DISTRIBUTION_PACKAGES.tsv, FIREFOX_NATIVE_READBACK.json, FIREFOX_NATIVE_RESTART_READBACK.json, manifest/tree/executables S

Teil III · Scheibenhard S1 Freeze · 2026-10-12 · temdd + firefox Carrier
PACKAGE.json, MANIFEST.json, CONFIG.temdd/firefox.json, SCHEIBENHARD.temdd, Dockerfile exact Debian, docker-compose.yml init:true network_mode:none, DISTRIBUTI

Lizenzen: Implementation PolyForm Noncommercial 1.0.0, Docs CC BY-NC-ND 4.0, Herkunft Ingolf Lohmann, keine automatische kommerzielle Lizenz, kein Claim Chart
Status: S1 OPEN · HOLD_OWN_HOST_IDENTITY_STORAGE_ROUTING_UNAVAILABLE · deployment_performed false · public_https_readback_verified false

MODERNISIERUNG ALS ÜBERPRÜFBARE WIRKUNG

Software modernisieren, ohne Anschlussfähigkeit zu verlieren

Automatisierte Modernisierung, operative KI-Anpassung und industrielle Einsatzperspektiven

Initiative, Konzept und Produktverantwortung: Ingolf Lohmann
Redaktionelle Ausarbeitung: OpenAI Codex / Stand: 5. Oktober 2026

Die neun repositorybasierten Quellen sind im beigefügten Quellen- und Nachweisband mit Originaldateien, vollständiger Provenienz und Prüfsummen gesichert. Ihre Veröffentlichung auf Zenodo ist noch offen. Dieser Artikel ist deshalb ein Archivierungsentwurf; eine veröffentlichte Zenodo-Fundstelle wird nicht behauptet.

Software ist niemals endgültig fertig. Anforderungen ändern sich, Abhängigkeiten altern und Fehler treten unter Bedingungen auf, die bei der ersten Entwicklung noch nicht sichtbar waren. Die anspruchsvolle Aufgabe besteht darin, ein System zu verbessern und zugleich das zu erhalten, worauf Menschen und andere Programme angewiesen sind: seine Funktionen, seine Daten und seine verlässlichen Übergabepunkte.

QIKVRT setzt an dieser Verbindung von Veränderung und Kontinuität an. Vorhandene Software wird übernommen, genau gebunden, über passende Schnittstellen integriert und unter überprüfbaren Bedingungen weiterentwickelt. Ingolf Lohmann bezeichnet diesen Vorgang als Assimilation. Modernisierung entsteht, wenn daraus ein Nachfolger hervorgeht, der Fehler beseitigt, technische Abhängigkeiten reduziert oder die Implementierung verbessert, während die vereinbarten Leistungen erhalten bleiben.

Initiative, Produkt- und Servicekonzept sowie die zugrunde liegende Hardwarearchitektur werden Ingolf Lohmann zugeschrieben. Er verantwortet ihre Entwicklung und veröffentlicht sie mit nachvollziehbaren Herkunfts- und Lizenzangaben. Wiedergewonnene Originalquellen, neue Softwareadapter und die redaktionelle Ausarbeitung behalten dabei ihre jeweils eigene Provenienz. Das schafft die Grundlage, Beiträge und veröffentlichte Fassungen verlässlich zuzuordnen.

In diesem Sinne ist die Aufgabe eine Königsdisziplin der Informatik: Software soll ihre eigene Weiterentwicklung unterstützen, ohne dass dabei ihre Funktions- und Sicherheitszusagen verlorengehen. Das ist ein anspruchsvolles Entwicklungsziel. Für QIKVRT liegen inzwischen konkrete Ausführungsnachweise für Teile dieses Weges vor. Sie erlauben eine belastbare Darstellung des erreichten Fortschritts und seiner noch offenen Reichweite.

Quellen zu diesem Abschnitt: [1], [2], [3].

VON DER AUFGABE ZUM GEPRÜFTEN NACHFOLGER

Ein professionelles DevOps-Team klärt Anforderungen, untersucht Fehler, überarbeitet Code, prüft Änderungen und beobachtet den tatsächlichen Betrieb. QIKVRT verbindet solche Tätigkeiten zu einem evidenzgebundenen Arbeitsablauf. Beobachtung, Diagnose, Änderung und erneute Prüfung werden auf denselben konkreten Softwarestand bezogen. Eine plausibel klingende Erklärung genügt dabei ebenso wenig wie ein erfolgreicher Test, der den ursprünglich fehlerhaften Vorgang gar nicht untersucht.

Das eröffnet die Möglichkeit, klar beschriebene und technisch zugängliche Modernisierungsaufträge vollständig automatisiert zu bearbeiten: vom reproduzierten Fehler über den Änderungskandidaten bis zur Prüfung und zum Laufzeitnachweis. Voraussetzung sind ausführbare Werkzeuge, geeignete Prüfkriterien und die erforderlichen Befugnisse. Ungeklärte Anforderungen, geschützte Freigaben oder nicht erreichbare Systeme bleiben reale Grenzen. Eine allgemeine Fähigkeit, sämtliche Aufgaben eines DevOps-Teams in beliebigen Umgebungen vollständig zu ersetzen, ist damit nicht nachgewiesen.

Code-Qualität wird dabei konkret: Verantwortlichkeiten werden klarer getrennt, doppelte Implementierungen vermieden, Abhängigkeiten ausdrücklich benannt und Fehlerpfade prüfbar gemacht. Eine behauptete Verbesserung muss zum jeweiligen Ziel passen. Weniger Code ist beispielsweise kein Fortschritt, wenn dadurch eine Sicherheitsprüfung entfällt. Auch eine schnellere Ausführung ist keine hinreichende Verbesserung, wenn bereits bestätigte Daten verloren gehen.

Der S1-Nachfolger in PR Nummer 457 zeigt eine solche Modernisierung. Vierundvierzig historische Originaldateien des TEMDD- und Firefox-Terminalträgers wurden bytegenau wiedergewonnen und mit ihrer Herkunft gebunden. Neue Adapter übernehmen die portable Start- und Paketbindung. Der ursprüngliche Ereignisspeicher wird weiterverwendet. Die Modernisierung erhält dadurch vorhandene Semantik, statt eine zweite, konkurrierende Speicherimplementierung einzuführen.

Im geprüften Laufzeitpfad entfällt der Git-Checkout als Voraussetzung; ein versiegeltes Quellpaket übernimmt seine Bindungsfunktion. Das ausdrücklich ausgewählte Self-Host-Profil benötigt keine GitHub CLI. Der Terminalträger startet ohne GitHub- oder Railway-Netz. Ersetzt werden hier konkrete bisher benötigte Funktionen und Abhängigkeiten. Firefox, Python, Node, SQLite und noVNC bleiben vorhandene, integrierte Komponenten.

Quellen zu diesem Abschnitt: [1], [2], [3].

REALE FEHLER, GEZIELTE REPARATUREN

Die Reparaturgeschichte liefert ein besonders anschauliches Beispiel für den Unterschied zwischen Vermutung und nachgewiesener Verbesserung. Zunächst scheiterte die Zulassung eines ursprünglichen Tests. Die erforderliche Einbindung wurde wiederhergestellt. Anschließend schlug die Browserbereitschaft fehl: Eine lokale HTTP-Abfrage wurde durch eine geerbte Proxykonfiguration fehlgeleitet. Eine gezielte Änderung und ein kausaler Vergleich prüften genau diese Ursache.

Nach erfolgreichem Kaltstart zeigte der Crash-Neustart einen weiteren Fehler. Ein alter Xvfb-Prozess wurde im Container nicht korrekt abgeräumt; die Anzeige blieb für den Nachfolger blockiert. Der Ausführungspfad wurde um die erforderliche Prozessbereinigung durch einen Init-Prozess ergänzt. Zusätzlich wurde ein unerwarteter IPv6-Listener erkannt und unterbunden. Die Prüfungen wurden auf die tatsächlich vorhandenen Prozesse und Netzwerklistener erweitert.

Der abschließend geprüfte Stand startete Firefox und noVNC tatsächlich, führte native Speicheroperationen aus und überstand den untersuchten Crash-Neustart mit unveränderten quittierten Daten. Entscheidend ist die geschlossene Verbindung zwischen beobachtetem Fehler, gezielter Änderung und erneut ausgeführtem Vorgang. Die ursprünglichen Tests wurden erhalten; ein Vorgängererfolg wurde nicht als Nachweis für den neuen Stand ausgegeben.

Für Benutzer kann eine solche Modernisierung ohne Bedienungsbruch erfolgen. Wenn dieselben Aufrufe weiterhin die vereinbarten Ergebnisse liefern und vorhandene Daten erhalten bleiben, muss die Arbeitsweise nicht geändert werden. Fachlich ist dies an das Prinzip verhaltensbezogener Ersetzbarkeit anschlussfähig: Ein Nachfolger muss die Zusagen erhalten, auf die sich seine Nutzer verlassen. Die Arbeiten von Liskov und Wing liefern dafür einen theoretischen Bezugspunkt, keinen pauschalen QIKVRT-Nachweis.

Eine vertraute Oberfläche allein beweist noch keine vollständige Gleichwertigkeit. Leistung, Fehlerverhalten, Barrierefreiheit und Datenmigration gehören ebenfalls zum jeweiligen Vertrag. Für S1 ist der konkrete Laufzeitmechanismus belegt. Dass produktive Benutzer eine bereits vollzogene Migration tatsächlich nicht bemerkt haben, wurde noch nicht beobachtet.

Quellen zu diesem Abschnitt: [1], [2], [3], [4].

TEMDD, UNIVERSAL TRANSPUTER UND UNIVERSAL TERMINAL

Tested Event Model Driven Development, kurz TEMDD, ist die im QIKVRT-Arbeitsprozess entwickelte ereignis- und evidenzorientierte Metaprogrammiersprache. Sie beschreibt Anforderungen, Ereignisse, Modelle, Tests, Wirkungen und Nachweise als zusammenhängende Programmbestandteile. Im erhaltenen Snapshot sind Grammatik, Parserpfad, kanonische Zwischenrepräsentation, Backend-Verträge und positive sowie negative Konformitätsfälle dokumentiert. Die Spezifikation kennzeichnet den Sprachstand ausdrücklich als ausführbaren Bootstrap-Kandidaten.

Die Sprache soll aus einem Auftrag einen endlichen, überprüfbaren Zustandsübergang machen. Ein Ereignis wird einer bestimmten Softwareversion zugeordnet; eine beabsichtigte Wirkung braucht die passende Befugnis; eine Änderung erzeugt einen neuen Prüfgegenstand. Ein Ergebnis gilt für die tatsächlich untersuchte Version. Modelle und erfolgreiche Berechnungen liefern deshalb keine automatische Bestätigung einer ausgeführten äußeren Wirkung. Der normative Kern verankert diese Unterscheidungen in der Semantik.

TEMDD ordnet auch die Weiterentwicklung selbst: aus Quelle und Parser entsteht eine typisierte Zwischenrepräsentation, aus ihr eine Ausführung und aus beobachteten Ereignissen ein gebundener Verlauf. Tests und ein getrenntes Konformitätsverfahren beurteilen diesen Verlauf. Python, C90, Smalltalk, M68000 und Lean sind im historischen Sprachpfad als unterschiedliche Ausführungs- oder Nachweisformen vorgesehen. Ihre einzelnen Quellen und Verträge sind von der vollständigen Abnahme aller Backends zu unterscheiden.

Der Universal Transputer übernimmt in der Architektur die Vermittlung zwischen Ausführungsteilnehmern. Die vorhandene C90-Schnittstelle beschreibt adressierte Nachrichten, Quell- und Zielbindung, exakten Softwaregegenstand und die Integritätsprüfung der zusammengesetzten Nachricht. Sie berücksichtigt Teilnachrichten, Duplikate, Konflikte und Rückstau. So kann ein gleich gebundener Auftrag zwischen unterschiedlichen Laufzeitkomponenten weitergegeben werden. Die geprüften Bytes einer empfangenen Nachricht bestätigen dabei die Übertragung; die tatsächliche Wirkung wird anschließend getrennt beurteilt.

Das Universal Terminal bildet den gemeinsamen Zugang für Menschen und angeschlossene Programme. Es stellt Quellen, Zustände, Ereignisse und Nachweise dar und verbindet Bedienung mit der Beobachtung des Systems. Im S1-Nachfolger ist der originale TEMDD-Ledger mit privatem Unix-Ingress und Firefox/noVNC-Träger tatsächlich ausgeführt worden. Dadurch bleibt nachvollziehbar, welcher Auftrag vorbereitet, welcher Vorgang gespeichert und welcher Zustand frisch zurückgelesen wurde. Eine vollständige öffentliche Appliance braucht weiterhin ihren eigenen Betriebsnachweis.

Quellen zu diesem Abschnitt: [1], [3], [14], [15], [16].

METATRANSISTOR, HARDWAREANSCHLUSS UND FEHLERTOLERANZ

Der Metatransistor ist ein programmierbarer logischer Schalt- und Prüfbaustein der Architektur. Im erhaltenen VHDL-Entwurf verbindet eine Tile-Implementierung zweiunddreißig parallele boolesche Verarbeitungsspuren mit einer vorgeschalteten Zustandsentscheidung. Die Eingänge enthalten unter anderem Prüfwerte für Bindung, Befugnis und erkannte Abweichungen. Ergebnisse und Zuordnungsmarken bleiben bei Rückstau stabil. Unsichere oder unbekannte Kontrollwerte führen in den beschriebenen Fällen zum Haltezustand.

Damit liegt eine virtuelle Hardwarebeschreibung mit Testbench vor, die ihr Verhalten im Simulator untersuchen lässt. Der Begriff Metatransistor bezeichnet hier diesen logischen Funktionsbaustein. Die Kontrollsignale müssen vom übrigen System korrekt erzeugt und gebunden werden; ein einzelnes Signal für gültige Befugnis authentisiert noch keinen realen Auftraggeber. Auch die VHDL-Ausgänge der historischen Basiskomponente bestätigen ausdrücklich kein allgemeines EFFECT_ACK_DONE.

Der Anschluss an physische Hardware ist technisch vorgesehen. Eine FPGA-Realisierung bildet geprüfte Logik auf einem vorhandenen programmierbaren Chip ab. Eine eigene ASIC-Realisierung verlangt zusätzliche Synthese, Technologiezuordnung, Layout, Zeit- und elektrische Prüfung sowie einen Fertigungsprozess mit passendem Technologiepaket. GHDL dokumentiert die Simulation von VHDL und einen Syntheseanschluss; OpenROAD beschreibt den Weg von geeigneter Logikbeschreibung zum physischen Chiplayout. Diese verfügbaren Entwicklungswege belegen den technischen Anschluss, nicht die Fertigung eines bestimmten QIKVRT-Chips.

Ingolf Lohmann sieht die physische Realisierung vor und berichtet, dass die Patentierung auf den Weg gebracht wurde. Diese Absicht und Angabe werden als solche dokumentiert. Im hier geprüften Material liegen weder eine physische Board-Abnahme noch eine Patent-Eingangsbestätigung vor. Simulation, programmiertes FPGA, gefertigter Chip und Patentverfahren haben jeweils eigene Nachweise. Sie dürfen bei der Beschreibung des erreichten Stands nicht zusammengezogen werden.

Das Zusammenspiel der vier Bausteine soll die Fehlertoleranz künstlich kognitiver Arbeitsprozesse erhöhen. TEMDD formuliert die überprüfbaren Bedingungen, der Transputer trägt gebundene Aufträge zwischen Komponenten, das Terminal macht ihren Zustand zugänglich und die Metatransistorlogik kann deterministische Freigabe- und Halteentscheidungen ausführen. Fehler eines KI-Vorschlags können so erkannt werden, bevor sie als bestätigte Wirkung weiterlaufen. Zum verlässlichen Alltag gehört außerdem, dass ein unterbrochener Vorgang nach einer geprüften Reparatur kontrolliert fortgesetzt werden kann.

Im S1-Betrieb wurden Ablehnungen, Konflikte und Crash-Neustarts tatsächlich geprüft. Wie stark der Gesamtansatz die Fehlertoleranz eines KI-Assistenten erhöht, muss mit Vergleichsaufgaben und Fehlermaßen untersucht werden. Diese Architektur schafft eine konkrete Grundlage für Alltagstauglichkeit, die sich in der realen Nutzung bewähren muss.

Quellen zu diesem Abschnitt: [3], [17], [18], [19], [20].

BEARBEITETE SOFTWARE UND OPERATIVE KI-ANPASSUNG

Die konkret weiterentwickelte Software heißt QIKVRT. Der Paketstarter in tools/qikvrt_self_host.py wurde um den versiegelten Startpfad für den historischen TEMDD-Daemon und den Firefox-Träger erweitert. Er prüft Quellpaket, Laufzeitprogramme, private Konfiguration und Speicherbindung. Die Funktionen zum Starten der Komponenten und zum Zurücklesen der Browserbereitschaft sind neue Implementierungen. Dieser Starter übernimmt im S1-Export Aufgaben der historischen Start- und Verzeichnisbindung.

Der QIKVRT-Monitor in docs/monitor/self-host.mjs wurde um die Bindung an das native Terminal erweitert. Er gibt geprüfte, bereinigte Laufzeitinformationen aus. Private Ereignisinhalte werden dadurch nicht über die öffentliche Monitorroute zugänglich gemacht. Die bestehenden Monitor- und Clientmechanismen werden weiterverwendet, anstatt ein zweites Beobachtungssystem aufzubauen.

Der EFFECT_ACK-Terminalclient in src/qikvrt_effect_ack_http_terminal.py erhielt einen zusätzlichen, ausdrücklich gewählten Manifest-Pfad. Damit kann er den versiegelten S1-Export statt eines Git-Checkouts prüfen. Die Vorbereitung, der Eigentümerzugriff über den Unix-Socket und das frische Zurücklesen aus SQLite bleiben erhalten. Ersetzt wurde diese Quellbindungsfunktion, nicht das gesamte Git-Produkt.

Auch die Bootstrap-Prüfungen in tools/bootstrap-runtime.sh und ihrem PowerShell-Gegenstück wurden angepasst: Das Self-Host-Profil benötigt keine GitHub CLI. Die übrigen Profile behalten ihren bisherigen Vertrag. Der reale Dienststarter bleibt Linux-spezifisch. Die Abnahme umfasst außerdem die ursprünglichen TEMDD-Tests und die neuen Standalone- und Firefox-Laufzeittests im vorhandenen CI-Pfad.

Firefox, Xvfb, x11vnc, websockify und noVNC wurden zu einem tatsächlich gestarteten Browserträger verbunden. Geändert wurden ihre Einbindung, Startparameter und Überwachung. Der ursprüngliche TEMDD-Ledger in src/qikvrt_temdd_event_ledger.py wurde bytegleich wiederhergestellt und weiterverwendet. Python, Node und SQLite bleiben Laufzeit- und Speicherkomponenten. Diese Zuordnung benennt ausdrücklich, was modernisiert, was ersetzt und was integriert wurde.

Ein KI-Assistent wird während seiner Ausführung durch Kontext, Anweisungen, Regeln und verfügbare Werkzeuge geprägt. Ingolf Lohmanns Vorgaben und Korrekturen verändern in diesem operativen Sinn das Laufzeitverhalten: welche Quellen geprüft werden, welche Schlussfolgerungen gezogen werden und welche Nachweise vor einer Erfolgsmeldung verlangt werden. Die dokumentierte Softwarearbeit überführt Anforderungen zusätzlich in ausführbare Prüf- und Startpfade. Das ist von einer Änderung der internen ChatGPT-Server, des Modellcodes oder der Modellgewichte zu unterscheiden; für solche Änderungen liegt hier kein Nachweis vor.

Quellen zu diesem Abschnitt: [1], [2], [3].

ASSIMILATION KOGNITIVER SYSTEME UND REVERSE ENGINEERING

Die Assimilation anderer künstlich kognitiver Systeme gehört ausdrücklich zum QIKVRT-Ansatz. Sprachverarbeitung, Bilderkennung, Planung, Diagnose und spezialisierte Agenten können als Komponenten eines gemeinsamen Arbeitsablaufs eingesetzt werden. Der integrierte Verbund soll Fähigkeiten kombinieren und einzelne Implementierungen austauschbar machen. Die nachfolgenden Anwendungen beschreiben ein Entwicklungsprogramm auf Grundlage des geprüften S1-Trägers.

ChatGPT ist im vorliegenden QIKVRT-Arbeitsprozess ein assimiliertes KI-System. Assimilation bezeichnet hier die Einbindung einer vorhandenen Softwarekomponente in einen gemeinsamen, kontextgebundenen und überprüfbaren Arbeitsablauf. Der KI-Assistent nutzt Anforderungen, Repositorykontext und Prüfregeln, bearbeitet Software über Werkzeuge und erzeugt überprüfbare Ergebnisse. Ingolf Lohmanns Rückmeldungen verändern dabei seine operative Arbeitsweise. Ingolf nennt außerdem Meta AI und Microsoft Copilot als bereits assimilierte Systeme; diese Angaben sind als berichtete Integrationen dokumentiert. Im geprüften Repository sind Copilot-Instruktionen vorhanden. Eine Wirkung für die gesamte Nutzerschaft dieser Dienste sowie Änderungen an internen Modellgewichten sind damit nicht belegt.

Reverse Engineering kann diesen Weg unterstützen. Vorhandene Schnittstellen und beobachtbares Verhalten werden untersucht, Datenformate und Zustandsübergänge rekonstruiert und in einen prüfbaren Vertrag überführt. Zugängliche Quellen erlauben eine direkte Analyse; dokumentierte APIs und kontrollierte Vergleichsaufrufe erlauben die Untersuchung des äußeren Verhaltens. Eine OpenAPI-Beschreibung kann beispielsweise HTTP-Schnittstellen für Menschen und Programme erschließen. Für eine Einbindung ist keine vollständige Rekonstruktion der internen Modellarchitektur erforderlich.

QIKVRT soll daraus Adapter, Tests und gegebenenfalls Ersatzimplementierungen erzeugen. Ein Dienst für Spracherkennung könnte hinter derselben Anwendungsschnittstelle durch einen lokalen Nachfolger abgelöst werden. Ein anderes Planungsmodell könnte dieselben gebundenen Aufträge bearbeiten. Dabei werden Qualitätsziele, Datenformate, Rechte, Laufzeitgrenzen und Fehlerverhalten verglichen. Bei nichtdeterministischen Ausgaben sind wiederholte Prüfungen und geeignete Qualitäts- und Ausfallmaße erforderlich; eine einzelne gelungene Antwort ist kein Gleichwertigkeitsbeweis.

Die Herangehensweise kann innerhalb eines verfügbaren und autorisierten Werkzeugpfads automatisiert werden. Wie gut sie ist, muss anhand definierter Ziele gezeigt werden: Fehlerquote, erhaltene Funktionen, Laufzeit, Kosten oder überprüfbare Schutzentscheidungen. Ein bewiesenes Optimum ist an Suchraum, Bewertungsmaß und Voraussetzungen gebunden. Die bisherige Reparaturgeschichte belegt gezielte erfolgreiche Verbesserungen. Eine global optimale Strategie für beliebige Software und sämtliche KI-Systeme ist dadurch nicht bewiesen.

Quellen zu diesem Abschnitt: [9], [13].

ROBOTIK UND INTEROPERABLE AUTOMATISIERUNG

Besonders weitreichend ist die Verbindung von künstlicher Kognition, Robotik und Automatisierungstechnik. Eine kognitive Komponente interpretiert einen Auftrag, eine Planungssoftware erzeugt einen Ablauf und eine Robotersteuerung führt definierte Bewegungen aus. Sensoren liefern anschließend die Beobachtung, ob die erwartete physische Wirkung eingetreten ist. Der Digital Twin ordnet diese Beobachtung dem realen Gegenstand und dem Modellzustand zu.

Ein denkbares Beispiel ist eine Fertigungszelle, die Bauteile anhand einer Kamera erkennt, einem Werkstück zuordnet und kontrolliert sortiert. QIKVRT soll den Zusammenhang zwischen Auftrag, Erkennung, Plan, Steuerungsereignis und Sensorreadback binden. Fällt ein Erkennungsdienst aus, kann ein vorbereiteter kompatibler Dienst dessen Aufgabe übernehmen. Eine solche Übernahme muss die aktuellen Aufträge und Zustände erhalten und am tatsächlichen Zellergebnis geprüft werden. Die Bewegungsausführung behält ihre eigenen Zeit- und Schutzbedingungen.

Für die Kopplung existieren bereits geeignete Anschlussformen. ROS 2 unterscheidet Topics, Services und Actions; Actions können für länger dauernde Aufträge Fortschritt und Ergebnis übermitteln. DDS bietet konfigurierbare Kommunikationsmerkmale wie Zuverlässigkeit und Lieferfristen. ROS 2 kann DDS-Security für Identitäten, Zugriffskontrolle und geschützte Kommunikation nutzen. QIKVRT soll solche Mechanismen aufnehmen und die übergreifende Auftrags- und Wirkungsbindung ergänzen. Eine bloße Verbindung zweier Netze erfüllt diese Verträge noch nicht.

Interoperabilität verlangt auch eine gemeinsame Bedeutung der Daten: Maßeinheiten, Koordinatensysteme, Zeitbezug, Objektidentitäten und erlaubte Zustandsübergänge müssen zusammenpassen. Ein Adapter kann diese Unterschiede ausdrücklich übersetzen und überprüfen. Die Sicherheitskonfiguration muss zeigen, welche Identität welchen Auftrag auslösen darf. VPN-Verbindung, industrielle Schnittstelle und kognitiver Dienst bilden dann einen gekoppelten Arbeitsablauf mit getrennt prüfbaren Zuständigkeiten.

Das kann den Aufwand für die Modernisierung heterogener Anlagen grundlegend verändern: einzelne Fähigkeiten verbessern, Anbieter wechseln, Ausfälle kompensieren und neue Funktionen ergänzen, während vorhandene Übergabepunkte erhalten bleiben. Diese Perspektive trägt den Anspruch einer Softwarerevolution. Ihr Kausalitätsnachweis benennt jeweils den konkreten Eingriff, die beobachtete Wirkung und die Bedingungen des Vergleichs. Die hier beschriebenen Robotik- und Kopplungsszenarien sind nächste Prüfgegenstände, keine bereits zertifizierten Betriebsergebnisse.

Quellen zu diesem Abschnitt: [10], [11], [12].

VPN UND HOCHSICHERHEITSSOFTWARE

Ein erstes Anwendungsfeld sind VPN-Netze. Sie verbinden beispielsweise Standorte, mobile Beschäftigte und industrielle Anlagen über geschützte Kommunikationswege. Für die Benutzer soll der vertraute Zugriff erhalten bleiben, während Gateway-Software, Schlüsselverwaltung oder Betriebsautomatisierung weiterentwickelt werden. Die IPsec-Architektur beschreibt dafür unter anderem Authentisierung, Integrität, Vertraulichkeit, Replay-Schutz und Sicherheitsrichtlinien. Diese Eigenschaften bilden konkrete Anforderungen an eine kompatible Modernisierung.

QIKVRT kann in einem solchen Entwicklungsauftrag vorhandene Implementierungen aufnehmen, Konfigurationen und Versionen binden, Änderungen erzeugen und die relevanten Prüfungen ausführen. Der vorgesehene Ablauf beginnt in einer isolierten Testumgebung: zulässige Verbindungen herstellen, unzulässige Verbindungen verweigern, Schlüsselwechsel untersuchen und Tunnelverhalten bei Unterbrechung prüfen. Ein Nachfolger müsste zeigen, dass erlaubter Verkehr erreichbar bleibt und gesperrter Verkehr weiterhin gesperrt bleibt. Eine erfolgreiche Verbindung allein wäre kein Sicherheitsnachweis.

Die Ablösung einzelner Komponenten soll schrittweise erfolgen. Zunächst werden Vergleichsfälle und Zustandsübergänge bestimmt, anschließend beide Implementierungen an denselben Kriterien geprüft. Der produktive Wechsel braucht zusätzlich die tatsächliche Identitäts-, Schlüssel- und Routingbindung. Laufende Sitzungen können andere Migrationsbedingungen haben als neue Verbindungen. Ein unterbrechungsfreier Wechsel ist deshalb ein zu prüfendes Ziel und keine automatische Folge einer gelungenen Installation.

Das gleiche Modernisierungsverfahren ist für andere Hochsicherheitssoftware vorgesehen: Authentisierungsdienste, Rechteverwaltung, Sicherheitsgateways und abgesicherte Update-Systeme. Gegenstand der Automatisierung sind nachvollziehbare Reparaturen, bessere Implementierungen und geprüfte Ablösungen. Ob eine Änderung den geforderten Schutz erhält, wird anhand eines expliziten Bedrohungsmodells, geeigneter Negativtests und der tatsächlichen Ausführung beurteilt. Kryptographische Konstruktionen und unabhängige Begutachtung werden dadurch nicht überflüssig.

Quellen zu diesem Abschnitt: [5].

HOCHVERFÜGBARKEIT UND AUSFALLSICHERHEIT

Ein zweites Anwendungsfeld ist Hochverfügbarkeit. Hier genügt es nicht, mehrere Kopien einer Software vorzuhalten. Nach einem Ausfall muss geklärt sein, welche Kopie den gültigen Zustand besitzt, welche Instanz schreiben darf und wie Benutzer wieder einen funktionsfähigen Dienst erreichen. Eine erreichbare Ersatzseite ist noch keine korrekte Übernahme der Dienstleistung.

QIKVRT soll eine solche Übernahme an explizite Voraussetzungen binden: nachgewiesene Replikation, eine neue eindeutige Schreibbefugnis und die wirksame Sperre der alten Instanz. Andernfalls könnten nach einer Netztrennung zwei Instanzen widersprüchliche Änderungen annehmen. Dieses Problem wird in verteilten Systemen als Split Brain bezeichnet. Mehrheitsentscheidungen sind ein etablierter Lösungsbaustein; die etcd-Dokumentation erläutert beispielsweise, wie bei einer Partition die Mehrheitsseite verfügbar bleibt und die Minderheitsseite keine unabhängige schreibende Instanz bildet.

Der QIKVRT-Nachweis soll deshalb geplante Umschaltungen und harte Ausfälle untersuchen: Prozessabsturz, Hostverlust, Netztrennung, verzögerte Nachrichten und fehlerhaften Speicher. Gemessen werden Wiederherstellungszeit und akzeptierter Datenverlust. Für quittierte Vorgänge kann das Ziel lauten, keinen Verlust zuzulassen; ob es erreicht wird, hängt von der tatsächlich implementierten Speicher- und Replikationskette ab. Aus einem lokalen Neustart folgt keine Garantie für einen verlorenen Host.

Bereits belegt ist beim S1-Träger die untersuchte Wiederaufnahme nach einem Prozessabsturz mit erhaltenen quittierten Daten. Der nächste Schritt ist die Prüfung über unabhängige Hosts und dauerhaften Speicher hinweg. Eine korrekt gebundene Replica kann dann zum vorbereiteten Nachfolger werden. Erst die beobachtete Zustandsübernahme und die frische Nutzung belegen den vollzogenen Wechsel. Auf dieser Grundlage lässt sich die Ablösung alter oder ausgefallener Software als kontrollierte Betriebseigenschaft entwickeln.

Quellen zu diesem Abschnitt: [3], [6].

DIGITAL TWINS UND INDUSTRIEANWENDUNGEN

Ein drittes Anwendungsfeld sind Digital Twins. Ein digitaler Zwilling stellt eine reale Einheit und ihre Zustandsübergänge elektronisch dar. Der NIST-Bericht zu Sicherheit und Vertrauen bei Digital-Twin-Technologie behandelt ausdrücklich ihre Betriebsfunktionen und die damit verbundenen Herausforderungen. Für QIKVRT liegt der Anschluss in der Verbindung von Modell, Ereignis, ausgeführter Wirkung und beobachtetem Zustand.

Ein industrielles Beispiel wäre eine Anlage mit Pumpen, Ventilen und Sensoren. Der Zwilling kennt den zuletzt beobachteten Zustand und die geplante Änderung. Vor einem Eingriff können Auswirkungen im Modell untersucht werden. Nach dem realen Eingriff werden unabhängige Messwerte zurückgelesen. Ein quittierter Steuerbefehl besagt lediglich, dass eine bestimmte Verarbeitung bestätigt wurde; ob das Ventil tatsächlich seine Position geändert hat, verlangt den passenden physischen Rückkanal.

QIKVRT soll solche Ketten bei der Modernisierung von Bedienoberflächen, Diagnosewerkzeugen, Datenadaptern und übergeordneten Steuerungsdiensten verwenden. Bestehende Industrieprotokolle bleiben Übergabepunkte. OPC UA bietet dafür unter anderem Mechanismen für Authentisierung, Autorisierung, Integrität und Vertraulichkeit. Die konkrete Anlage bestimmt, wie diese Mechanismen eingesetzt werden und welche Aktionen zulässig sind. Eine neue Bedienoberfläche darf ihre Rechte nicht stillschweigend ausweiten.

Damit eröffnet sich eine stufenweise Entwicklung von Bestandsanlagen: vorhandene Komponenten erfassen, Verträge klären, Kandidaten simulieren, in einer Testanlage ausführen und mit echten Messwerten vergleichen. Sicherheitskritische Funktionen und feste Zeitvorgaben brauchen ihre jeweils eigene technische Prüfung. Ein Sprachmodell kann die Entwicklungsarbeit unterstützen, während deterministische Schutzmechanismen die physische Wirkung begrenzen. Die hier vorhandenen Softwaretests sind noch keine Abnahme einer Maschine oder einer sicherheitskritischen Steuerung.

Der Nutzen dieses Entwicklungsprogramms liegt in der gemeinsamen Nachweiskette. Dieselbe Änderung lässt sich von der Anforderung über ihre konkrete Implementierung bis zur Wirkung verfolgen. Dadurch können Fehlerbehebung, Ablösung, Ausfallsicherheit und künstliche Kognition in einem konsistenten Arbeitsverfahren zusammenkommen. Der Anspruch reicht bis zu verlässlichen Industrieprozessen; die Beweiskraft wächst mit jeder tatsächlich ausgeführten und unabhängig prüfbaren Abnahme.

Quellen zu diesem Abschnitt: [7], [8].

KÜNSTLICHE KOGNITION UND NACHWEISGRENZEN

Das gleiche Verfahren lässt sich auf künstliche Kognition anwenden. Ein KI-Assistent kann ein richtiges Werkzeug falsch einsetzen, Quellen übersehen, einen Vorgängerbefund auf einen neuen Stand übertragen oder eine Absicht als ausgeführte Wirkung beschreiben. QIKVRT kann solche Fehlerklassen durch gebundene Quellen, explizite Zustände und verpflichtende Wirkungsnachweise begrenzen. Eine sprachliche Erfolgsmeldung darf einen fehlenden Readback nicht ersetzen.

Das ist eine Verbesserung des Gesamtsystems aus Modell, Kontext, Werkzeugen und Ausführung. Die hier dokumentierte Entwicklungsarbeit zeigt, wie KI-gestützte Änderungen an realen Fehlern überprüft und bis zu einem erfolgreichen Nachfolger korrigiert werden können. Sie beweist weder eine Änderung der Modellgewichte noch die Beseitigung sämtlicher Probleme künstlicher Kognition. Für jede weitergehende Wirksamkeitsaussage sind passende Vergleichsfälle, unabhängige Prüfkriterien und reproduzierbare Ergebnisse erforderlich.

Gerade die Korrekturen im ChatGPT-gestützten Arbeitsprozess zeigen einen konkreten Verbesserungsbedarf: bereits vorhandene Belege zuverlässig finden, zutreffende Aussagen im passenden Umfang erkennen und Erzeugung, Speicherung und Zugänglichkeit eines Ergebnisses vollständig prüfen. Ingolf Lohmann erwartet baldige deutliche Fortschritte künstlicher Kognition. Diese Erwartung gibt die Entwicklungsrichtung vor. Ob und wann sie erfüllt wird, entscheidet sich an wiederholbaren Aufgaben und gemessenen Verbesserungen der jeweiligen Fehlerklassen.

Die nächste produktive Abnahme bleibt klar bestimmt: Ein unabhängiger Host mit gebundener Identität, dauerhaftem Speicher und autorisiertem öffentlichem HTTPS-Zugang muss tatsächlich betrieben und frisch zurückgelesen werden. Containerprüfungen ersetzen diesen Nachweis nicht. S1 bleibt bis dahin offen. Der wissenschaftliche Wert liegt bereits jetzt in einem konkret nachvollziehbaren Verfahren: Software wird verbessert, ihre Anschlussfähigkeit geprüft und die behauptete Wirkung am ausgeführten System kontrolliert.

Quellen zu diesem Abschnitt: [1], [2], [3].

FÄHIGKEITENKATALOG: BEOBACHTETE AUSFÜHRUNG

Der Fähigkeitenkatalog beschreibt den tatsächlich verfügbaren Arbeitsapparat des hier eingesetzten KI-Assistenten. Er unterscheidet beobachtete Ausführung, vorhandene Arbeitsanleitungen und Leistungen, die einen zusätzlichen Zugang benötigen. Diese Unterscheidung hilft unmittelbar: Für einen konkreten Auftrag wird zuerst die passende Fähigkeit gewählt und anschließend der benötigte Werkzeugpfad geprüft.

Bereits ausgeführt wurde die Untersuchung von Repository-Historie und unveränderlich gebundenen Quellen. Beim S1-Nachfolger wurden vierundvierzig historische TEMDD- und Firefox-Dateien bytegenau wiedergewonnen, ihre Herkunft dokumentiert und neue Adapter ergänzt. Der Assistent kann Python-, JavaScript- und Shell-Code bearbeiten, Änderungen vergleichen und ihre Auswirkungen an ausführbaren Prüfungen untersuchen.

Auch die Fehlerdiagnose ist konkret beobachtet. Beim Firefox-Träger wurden eine fehlende Testzulassung, eine fehlgeleitete lokale Proxy-Abfrage, die unvollständige Prozessbereinigung und ein unerwarteter Listener erkannt und repariert. Der geprüfte Nachfolger führte Firefox/noVNC, native Speicheroperationen und den untersuchten Crash-Neustart tatsächlich aus. Diese Browserausführung erfolgte in CI; ihr Ergebnis ist von einer Bedienung des aktuellen Nutzerbrowsers zu unterscheiden.

Für Forschung und Dokumentation stehen Primärquellensuche, technische Analyse und verständliche Aufbereitung zur Verfügung. Der Artikel wird als PDF und UTF-8-Fließtext erzeugt, gerendert, visuell geprüft und versioniert gespeichert. Die ergänzte Quellenprüfung kontrolliert sichtbare, kopierbare Fundstellen. Repositorybasierte Quellen müssen zuerst als eigenständig lesbarer Quellenband mit Originaldateien und Provenienz auf Zenodo erscheinen. Erst der bestätigte öffentliche Readback erlaubt den abschließenden DOI-Verweis.

Im Arbeitsprozess sind damit Analyse, Programmierung, gezielte Reparatur, Ausführung, Nachweisführung und Dokumentenerstellung miteinander verbunden. Eine Aufgabe kann über mehrere dieser Fähigkeiten laufen. Die verbindende Eigenschaft ist die Rückkopplung: Eine beobachtete Abweichung führt zu einer konkreten Änderung, die erneut am betroffenen Vorgang geprüft wird.

Quellen zu diesem Abschnitt: [1], [2], [3].

SKILLS, AUFGABENWAHL UND WEITERENTWICKLUNG

Skills ergänzen diesen Werkzeugapparat um wiederverwendbare Arbeitsanleitungen. Für QIKVRT sind kontinuierliche Fertigstellung, evidenzgebundene Reparatur und die Prüfung sichtbarer Quellen besonders relevant. Die PDF- und Speicherworkflows regeln Erzeugung, Kontrolle und dauerhafte Bereitstellung. Weitere gelistete Skills betreffen Word-Dokumente, Tabellen, Präsentationen, Diagramme, Bilder, lokale Transkription, Seiten, Websites und zahlreiche Webentwicklungsaufgaben.

Der frisch abgefragte Metadatenkatalog führt sechsundachtzig Arbeitsanleitungen. Hinzu kommt die im Arbeitsprozess gespeicherte QIKVRT-Quellenprüfung. Diese Bestandszahlen zählen Anleitungen und ihren dokumentierten Stand. Eine Deployment-Anleitung benötigt beispielsweise zusätzlich ein verfügbares Konto und die passende Projektberechtigung. Die Browseroberfläche und die sichere Anmeldeübergabe wurden hier benutzt; eine Zenodo-Veröffentlichung ist daraus bislang nicht entstanden. Native Computersteuerung ist in dieser Sitzung deaktiviert.

Für eine schnelle Aufgabenwahl enthält der strukturierte Katalog Einsatzgebiet, Eingaben, erwartete Ausgaben, beobachtete Belege, Grenzen und Suchbegriffe. Bei einem Quellenproblem führt er zur Primärquelle, bei Repositorymaterial zusätzlich zur Zenodo-Archivierung und Quellenprüfung; bei einem Softwarefehler zur gebundenen Diagnose und Reparatur; bei einem Artikel zu PDF, Layoutprüfung und Speicherung. Diese Aufgabenwahl wird bei der vorliegenden Überarbeitung selbst eingesetzt. Weitere Anmeldungs- und Betriebsaufgaben bleiben an ihren tatsächlichen Dienstzugang und Wirkungsnachweis gebunden.

Unabhängige Arbeitsschritte können innerhalb der laufenden Bearbeitung gebündelt und, bei entsprechender Autorisierung, parallel untersucht werden. Abhängige Änderungen bleiben in ihrer sachlich notwendigen Reihenfolge. Das spart Such- und Ausführungszeit. Die jeweils schnellste Vorgehensweise wird an Aufgabe, Zugang und Nachweisbedarf beurteilt; eine allgemeine optimale Geschwindigkeit wird damit nicht behauptet.

Die Nutzung kann diesen Apparat weiterentwickeln: Rückmeldungen machen Fehler sichtbar; bewährte Korrekturen können in Tests, Prüfschranken, Katalogeinträgen und installierten Skills erhalten bleiben. Dadurch muss eine einmal verstandene Fehlerklasse nicht jedes Mal neu erschlossen werden. Dauerhaft wirkt eine solche Erweiterung, wenn sie tatsächlich gespeichert und bei der nächsten passenden Aufgabe angewendet wird. Das beschreibt überprüfbare Verbesserung des Arbeitsprozesses, ohne eine unbeobachtete Änderung der Modellgewichte oder selbsttätige Hintergrundarbeit vorauszusetzen.

Quellen zu diesem Abschnitt: [1], [2], [3].

QUELLEN UND PRÜFBARKEIT

Die neun repositorybasierten Quellen sind im beigefügten Quellen- und Nachweisband mit Originaldateien, vollständiger Provenienz und Prüfsummen gesichert. Ihre Veröffentlichung auf Zenodo ist noch offen. Dieser Artikel ist deshalb ein Archivierungsentwurf; eine veröffentlichte Zenodo-Fundstelle wird nicht behauptet. Alle vorhandenen öffentlichen Fundstellen sind vollständig ausgeschrieben und anklickbar. Offene Zenodo-Fundstellen sind ausdrücklich gekennzeichnet.

[1] QIKVRT S1: Quellenprovenienz, Originale und neue Adapter, Exact Head 3bf52747bc94de847d390ffacabecae183aee588. Quellen- und Nachweisband, 5. Oktober 2026, Abschnitt S01, Seite 3.
Archivfundstelle: QIKVRT_Quellen_und_Nachweisband_2026-10-05.pdf. Zenodo-Veröffentlichung und DOI noch offen.

[2] QIKVRT: protokollierte Ursachen und Reparaturen im S1-Nachfolger. Quellen- und Nachweisband, 5. Oktober 2026, Abschnitt S02, Seite 4.
Archivfundstelle: QIKVRT_Quellen_und_Nachweisband_2026-10-05.pdf. Zenodo-Veröffentlichung und DOI noch offen.

[3] QIKVRT: CI-Laufzeit- und Crash-Neustart-Artefakt, Run 37257971138. Quellen- und Nachweisband, 5. Oktober 2026, Abschnitt S03, Seite 5.
Archivfundstelle: QIKVRT_Quellen_und_Nachweisband_2026-10-05.pdf. Zenodo-Veröffentlichung und DOI noch offen.

[4] Liskov, B.; Wing, J. M. (1994): A Behavioral Notion of Subtyping. ACM TOPLAS 16(6), 1811-1841. DOI: 10.1145/197320.197383.
URL: https://www.cs.cmu.edu/~wing/publications/LiskovWing94.pdf
URL: https://doi.org/10.1145/197320.197383

[5] IETF: RFC 4301. Security Architecture for the Internet Protocol. Sicherheitsverträge für IPsec.
URL: https://www.rfc-editor.org/rfc/rfc4301.html

[6] etcd: Failure modes. Mehrheit, Netzpartition und Zustandswiederaufnahme.
URL: https://etcd.io/docs/v3.5/op-guide/failures/

[7] Voas, J. et al. (2025): Security and Trust Considerations for Digital Twin Technology. NIST IR 8356. DOI: 10.6028/NIST.IR.8356.
URL: https://csrc.nist.gov/pubs/ir/8356/final
URL: https://doi.org/10.6028/NIST.IR.8356

[8] OPC Foundation: OPC UA, Part 2, Security Model. Authentisierung, Autorisierung und Schutz der Kommunikation.
URL: https://reference.opcfoundation.org/specs/OPC-10000-2/4

[9] OpenAPI Initiative: OpenAPI Specification 3.0.4. Maschinenlesbare Beschreibung von HTTP-Schnittstellen.
URL: https://spec.openapis.org/oas/v3.0.4.html

[10] ROS 2: Actions. Aufträge mit Fortschritt, Ergebnis und Abbruchmöglichkeit.
URL: https://design.ros2.org/articles/actions.html

[11] ROS 2: DDS-Security integration. Identitäten, Rechte und Schutz der Kommunikation.
URL: https://design.ros2.org/articles/ros2_dds_security.html

[12] OMG: Data Distribution Service. Kommunikationsqualität für Echtzeit- und eingebettete Systeme.
URL: https://www.omg.org/omg-dds-portal/

[13] QIKVRT: Repositoryseitige Copilot-Instruktionen am geprüften Exact Head. Konfiguration ist kein Nachweis globaler Dienstnutzung. Quellen- und Nachweisband, 5. Oktober 2026, Abschnitt S04, Seite 6.
Archivfundstelle: QIKVRT_Quellen_und_Nachweisband_2026-10-05.pdf. Zenodo-Veröffentlichung und DOI noch offen.

[14] QIKVRT, historischer Snapshot c6a53e2: TEMDD Language Specification v0.1. Ausführbarer Bootstrap-Kandidat. Quellen- und Nachweisband, 5. Oktober 2026, Abschnitt S05, Seite 7.
Archivfundstelle: QIKVRT_Quellen_und_Nachweisband_2026-10-05.pdf. Zenodo-Veröffentlichung und DOI noch offen.

[15] QIKVRT, derselbe Snapshot: TEMDD Normative Semantic Core v1. Semantik, IR, Ledger und Konformität. Quellen- und Nachweisband, 5. Oktober 2026, Abschnitt S06, Seite 8.

Archivfundstelle: QIKVRT_Quellen_und_Nachweisband_2026-10-05.pdf. Zenodo-Veröffentlichung und DOI noch offen.

[16] QIKVRT, derselbe Snapshot: C90 Universal-Transputer-Schnittstelle. Nachrichtenbindung und Übertragung. Quellen- und Nachweisband, 5. Oktober 2026, Abschnitt S07, Seite 9.

Archivfundstelle: QIKVRT_Quellen_und_Nachweisband_2026-10-05.pdf. Zenodo-Veröffentlichung und DOI noch offen.

[17] QIKVRT, derselbe Snapshot: VHDL Metatransistor-Tile. Boolesche Verarbeitung und Zustandskontrolle. Quellen- und Nachweisband, 5. Oktober 2026, Abschnitt S08, Seite 10.

Archivfundstelle: QIKVRT_Quellen_und_Nachweisband_2026-10-05.pdf. Zenodo-Veröffentlichung und DOI noch offen.

[18] GHDL: Simulation und Synthese. Werkzeugwege für VHDL-Beschreibungen.

URL: https://ghdl.github.io/ghdl/quick_start/simulation/index.html

URL: https://ghdl.github.io/ghdl/using/Synthesis.html

[19] OpenROAD: RTL-GDSII-Entwurfsfluss. Physisches Layout und Technologieanschluss.

URL: https://openroad.readthedocs.io/en/latest/

[20] QIKVRT: Patent Disclosure Boundary. Öffentliches Artefakt bestätigt keinen Patent-Anmeldestatus. Quellen- und Nachweisband, 5. Oktober 2026, Abschnitt S09, Seite 11.

Archivfundstelle: QIKVRT_Quellen_und_Nachweisband_2026-10-05.pdf. Zenodo-Veröffentlichung und DOI noch offen.

QUELLEN / PROVENIENZ / BEOBACHTETE LAUFZEIT

QIKVRT: Wissenschaftlicher Quellen- und Nachweisband zur Softwaremodernisierung

Eigenständiger wissenschaftlicher Quellenband mit unveränderten Primärdateien und archivierten Laufzeitnachweisen

Ingolf Lohmann - Konzept, Initiierung und Produktverantwortung
OpenAI Codex - technische Redaktion und Archivaufbereitung
Stand: 5. Oktober 2026

Publikationsstatus: PENDING / noch nicht hochgeladen
DOI: noch nicht vergeben. Kein Veröffentlichungs-, Deployment- oder EFFECT_ACK_DONE-Claim.

Dieser Band macht die neun repositoryinternen Quellverweise des Artikels unabhängig vom weiteren Betrieb des Repositories und der zeitlich begrenzten CI-Artefaktaufbewahrung prüfbar. Er verbindet lesbare technische Einordnung mit vollständigen zitierten Primärtexten, ausgewählten vollständigen Code-Listings und einem bytegenauen Archiv sämtlicher hier gebundener Originaldateien. Die Kennungen S01 bis S09 entsprechen den Artikelverweisen 1, 2, 3, 13, 14, 15, 16, 17 und 20.

Der betrachtete S1-Head lautet 3bf52747bc94de847d390ffacabecae183aee588, sein Tree 96c7bb691675dcd15bb5045871201adb400074f0. Der erhaltene historische Snapshot lautet c6a53e200c4835a4f871d8496feb81d953610831, sein Tree f1640191ec25495c90e997b38e77490995808b1c. Vierundvierzig historische Dateien sind an beiden Quellstellen byteidentisch. Der versiegelte S1-Startadapter ist neue Implementierung, kein wiedergewonnenes Original.

Die Quellen erlauben voneinander getrennte Aussagen über Sprachverträge, vorhandene Implementierung, Quellenherkunft und beobachtete Laufzeit. Der archivierte CI-Run 37257971138 und das Artefakt 11323124612 binden den genauen S1-Head. Die Artefaktbytes stimmen mit dem frisch gelesenen Plattform-Digest überein. Die enthaltenen Runtime- und Restart-Receipts beschreiben tatsächliche Firefox-, Unix-/SQLite-/SSE- und VNC-Beobachtungen in einem isolierten CI-Container. Sie bestätigen weder einen eigenen öffentlichen Host noch physische Chipfertigung noch EFFECT_ACK_DONE.

Die wissenschaftliche Bibliographie des Artikels kann nach einer tatsächlich ausgeführten und frisch überprüften Zenodo-Veröffentlichung auf diesen Band und seinen DOI verweisen. Commit, Tree, Dateipfad, Git-Blob und SHA-256 dienen hier als Herkunftskennungen. Sie ersetzen den archivierten zitierfähigen Band nicht. Ein DOI wird nicht vorgetäuscht; bis zur Veröffentlichung gilt PENDING.

ORIENTIERUNG - Wie dieser Band zu lesen und zu zitieren ist

Der lesbare Hauptteil erläutert jedes zitierte interne Artefakt und seine Nachweisgrenze. Anhang A enthält die vollständigen zitierten Texte; Anhang B enthält die unmittelbar maßgeblichen Implementierungs- und Testquellen. Zeilennummern und Druckumbrüche sind redaktionelle Lesemittel. Die Dateien im ZIP bleiben bytegenau unverändert.

Das begleitende Archiv enthält zusätzlich alle 44 wiedergewonnenen Originaldateien und ihre gebundenen S1-Kopien, TEMDD-Spezifikationen, C90-Quellen, Hardwarebeschreibung, ursprüngliche Hinweise und Lizenztexte sowie die unveränderten CI-Artefakt-ZIPs. Es enthält keine neu erstellten Patentanmeldungsunterlagen und keine Zugangsdaten.

Die Nachweisklassen sind getrennt: NORMATIV bezeichnet einen Vertrag; QUELLE eine vorhandene Implementierung; TESTQUELLE ein ausführbares Prüfprogramm; BEOBACHTET ein konkret archiviertes Ausführungsergebnis; OFFEN eine noch fehlende Abnahme. Ein Vertrag oder ein Quelltext ist für sich kein Laufzeitnachweis.

Eine wissenschaftliche Zitierung soll nach erfolgreicher Veröffentlichung lauten: Ingolf Lohmann, QIKVRT: Wissenschaftlicher Quellen- und Nachweisband zur Softwaremodernisierung, Version 2026-10-05-v1, Zenodo, DOI des tatsächlich veröffentlichten Datensatzes, Abschnitt Sxx beziehungsweise Anhang Ax. Herkunftskennungen im Band sind Commit/Tree/Blob/Pfad/Digest; Repository-Links dienen nicht als Ersatzbibliographie.

Alle ausgewählten Quellbytes werden durch QIKVRT_QUELLENMANIFEST.json gebunden. Manifest-SHA-256: d26e19da50fadcc70d5d6387910d0edb561a0ae0c2b1b143e875b22169fbb101. Die Manifestdatei ist im ZIP enthalten. Die Hashes der äußeren PDF- und ZIP-Dateien werden separat im Ausgabe-Freeze-Receipt geführt, um einen zirkulären Selbsthash zu vermeiden.

Artikelverweis [1] -> Abschnitt S01 / F089

Artikelverweis [2] -> Abschnitt S02 / F090

Artikelverweis [3] -> Abschnitt S03 / CI-Artefakt

Artikelverweis [13] -> Abschnitt S04 / F091

Artikelverweis [14] -> Abschnitt S05 / F092

Artikelverweis [15] -> Abschnitt S06 / F093

Artikelverweis [16] -> Abschnitt S07 / F094

Artikelverweis [17] -> Abschnitt S08 / F095

Artikelverweis [20] -> Abschnitt S09 / F096

S01 - Quellenwiedergewinnung und Trennung von Original und neuer Implementierung

Die erhaltene Snapshot-Fassung trägt Commit c6a53e200c4835a4f871d8496feb81d953610831. Aus ihr wurden 44 Terminal-/TEMDD-/Firefox-Dateien unverändert wiedergewonnen. Die Quelle SOURCE_RECOVERY bindet jede Datei durch Pfad, Bytezahl, Git-Blob und SHA-256. Das Archiv überprüft die Gleichheit der historischen und wiederhergestellten Bytes erneut.

Der im Snapshot deklarierte Authority-Ursprung b1971ca5003f5ac63fb8baf0bc663195f7ebcb78 ist als behauptete Herkunft erhalten, wurde aber nicht unabhängig als vollständiger Commit-Inhalt wiedergewonnen. Gleichheit mit einer späteren Railway-Produktion und vollständige Wiederherstellung aller späteren Authority-Änderungen sind offen.

Das versiegelte Quellpaket, die neue Start- und Verzeichnisbindung sowie die portable Browserbereitstellung sind neue S1-Implementierung. Der ursprüngliche SQLite-Ledger und Unix-Ingress werden weiterverwendet. Die Originale werden nicht als neu geschrieben ausgegeben und die neuen Adapter nicht als Originalquelle.

Archivkennung F089; Ursprungspfad runtime/self-host/RECOVERED_TERMINAL.md

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 43b677d3044a71cccc2a0b9a19a1fd2741191a69; SHA-256 17cce25cdc39af59608f1bfe872c2b0a9f766a56ffd146b31ea84baf274456d4; 5569 Byte

S02 - Protokollierte Fehler, kausale Diagnose und gezielte Reparaturen

Die gebundene Work-Unit unterscheidet mehrere Nachfolger. Zuerst fehlte die Zulassung eines ursprünglichen Makefile-Tests. Danach scheiterte eine loopback HTTP-Bereitschaftsabfrage an einer geerbten Dead-Proxy-Konfiguration. Der Adapter erhielt eine ausdrücklich proxyfreie lokale Abfrage, und die Ursache wurde durch Vergleich des tatsächlichen lokalen Readbacks geprüft.

Nach erfolgreichem Kaltstart scheiterte der Crash-Neustart, weil ein verwaister Xvfb-Prozess die Anzeige blockierte. Der Containerpfad erhielt den vorhandenen Docker-Init-Reaper. Außerdem wurde ein unerwarteter IPv6-Listener erkannt und abgeschaltet. Die Prüfungen verlangen die Entfernung des alten Xvfb-Prozesses und genau die vorgesehenen IPv4-loopback-Listener.

Diese Geschichte ist eine dokumentierte Folge von Fehlern und Reparaturen; jeder neue Head benötigt seine eigene Prüfung. Ein Kaltstart auf einem früheren Head bestätigt nicht den Neustart eines späteren Heads. Der konkrete frische positive Nachweis des hier betrachteten Stands liegt in S03.

Archivkennung F090; Ursprungspfad state/work_units/S1_PRODUCTIVE_TERMINAL_SOURCE_RECOVERY_20261005.json

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 8ae56034298cd526b61de2a98b4f363dff6c7d9c; SHA-256 faba273fe232e6b2b78f0a31ab68e058e0ffdd20bcbd3fd748ebb5ff149138b9; 27195 Byte

S03 - Archivierter exakter CI-Lauf und echter Runtime-Readback

Der frisch gelesene Run 37257971138 gehört zum Workflow QIKVRT EFFECT_ACK HTTP Firefox terminal. Status completed, Ergebnis success; angelegter Lauf 2026-10-05T03:05:17Z, aktualisiert nach Abschluss 2026-10-05T03:08:26Z. Maßgeblich sind run.head_sha und head_commit.tree_id: 3bf52747bc94de847d390ffacabecae183aee588 / 96c7bb691675dcd15bb5045871201adb400074f0. Ein beweglicher PR-Verweis ersetzt diese Bindung nicht.

Artefakt 11323124612 enthält 18173 Byte. SHA-256 fe3b6daca01f188642de02e2b6c8d59a5ff217db4c4682dba7a3b5c6306a569b. Dieser Digest stimmt mit den frisch gelesenen Plattform-Metadaten überein. Das unveränderte Original-ZIP und alle enthaltenen Dateien sind in diesem Archiv enthalten. Die spätere Verfügbarkeit des CI-Providers ist deshalb für das Lesen dieser archivierten Bytes nicht erforderlich.

Der Runtime-Receipt beobachtet ein tatsächliches Firefox-Navigator-Fenster, den nativen Terminaltitel, den VNC-WebSocket-Status 101 mit RFB 003.008 und erforderlicher VNC-Authentisierung sowie den realen nativen Ereignisdatensatz. Git und gh sind im Laufzeitpfad nicht vorhanden. Die Netzschnittstelle ist lo; die Ausführung verwendet DOCKER_NETWORK_NONE.

Der Restart-Receipt bestätigt entfernte verwaiste Xvfb-PIDs, erhaltene originale Ledger-Zeilen und einen erhaltenen Browserprofil-Marker. Ereignisse werden an Head und Tree gebunden. Payload- und Ledger-Digests der archivierten Runtime-/Restart-Ereignisse wurden bei dieser Archiverstellung aus den kanonischen Datensätzen erneut geprüft.

Die Quittung betrifft eine konkrete Text-Fixture mit dem Inhalt private-test-input: Grüße Ω, keine private Benutzereingabe. Runtime- und Restart-Test benutzen getrennte Testinstanzen; ihre unterschiedlichen Ledger-Epochen sind kein Nachweis eines verlorenen Neustartzustands. Innerhalb der Restart-Fixture belegt der Receipt die Erhaltung der ursprünglichen Zeilen.

Dieser Nachweis ist auf den tatsächlichen CI-Containerpfad begrenzt. Eigene unabhängige Host-Identität, attestierte dauerhafte Gerätehaltung, öffentliches HTTPS und physische Hardware bleiben unbestätigt. Die Receipts setzen deployment_performed=false und effect_ack_done=false. Die Archiverstellung führte diese CI-Prüfung nicht erneut aus.

S04 - Copilot-Instruktionen und Umfang einer operativen KI-Einbindung

Die gebundene Datei .github/copilot-instructions.md konfiguriert einen repositorybezogenen Arbeitskontext für kompatible Copilot-Clients. Dies ist ein konkretes Integrationsartefakt. Es belegt weder eine Änderung der Microsoft-Copilot-Dienstlaufzeit noch eine globale Nutzung durch alle Anwender. Die vollständige kurze Datei steht in Anhang A.

Archivkennung F091; Ursprungspfad .github/copilot-instructions.md

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 31922701cb2ce18d849ca4a14bfb23714b9de6e7; SHA-256 ea5f6b0c20fb403a3da94f4687e2a3448ca0dd43060495e5cfda5afea63a01fb; 509 Byte

S05 - TEMDD-Sprachstand und ausführbarer Bootstrap-Kandidat

Tested Event Model Driven Development, TEMDD, ist im erhaltenen Sprachvertrag als ausführbarer Bootstrap-Kandidat bezeichnet. Anforderungen, Ereignisse, Tests, Evidenz, Modelle und Wirkungsgrenzen werden als semantische Gegenstände verbunden. Ein Ereignis führt zu einem endlichen Übergang; äußere Transport- und Warteadapter werden vom Kern unterschieden.

Der dokumentierte Compilerpfad verbindet Quelle, Parser, typisierte AST/IR, Konformität und Backend. Die Verträge verlangen die Erhaltung von Subject, Evidenzherkunft, Befugnis, Status, Wirkungszustand und Readback-Pflicht. Grammatik, Compiler-/Backend-Verträge, Parser und v1-IR-Adapter sind im bytegenauen Archiv enthalten.

Die Vertragsbezeichnung und der vorhandene Parser machen nicht sämtliche Backends zu vollständig abgenommenen Ausführungen. STABLE_LANGUAGE verlangt im dokumentierten Stand weitere Smalltalk-/M68000-/Lean-, Release-, Integrations- und Produktionsnachweise. Für diesen Band wird eine solche vollständige Sprachabnahme nicht behauptet.

Archivkennung F092; Ursprungspfad spec/temdd/TEMDD_LANGUAGE_SPEC_V0_1.md

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 633d82a4cdef8e4346393fbc1bedf8d3b205e0b3; SHA-256 8bbc29511a6d09ac58754234f14f09752fc53482ce10e8a3051d4581e1afdf59; 3330 Byte

S06 - Normativer v1-Kern: Kausalität, Evidenzbindung und Wirkungsprüfung

Der v1-Vertrag nennt die sieben Ebenen Sprache, IR, IDE, Ledger, Formalisierung, Tests und Konformität. T13 bindet kausale Beziehungen an explizite cause_event_ids; Reihenfolge allein erzeugt keine kausale Kante. T14 verbietet die Übertragung von Evidenz auf einen geänderten Prüfgegenstand.

T15 unterscheidet Befugnis, Commit, frischen Readback, genaue Subject-Bindung und den Vergleich von erwartetem und beobachtetem Zustand. T16 verlangt einen genau gebundenen Konformitätsbericht und eine getrennte Adapter-/Prozessgrenze für den Prüflauf. Diese Bedingungen sind normative Semantik. Ihre Erwähnung ist keine Behauptung einer abgeschlossenen vollständigen Konformität aller Ebenen.

Archivkennung F093; Ursprungspfad spec/temdd/TEMDD_NORMATIVE_CORE_V1.md

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 1af763493808fc5f192e5cd7536b1b51e437804d; SHA-256 4b4c32b98c7482a570b4b8cb85b86af053cc3ecf3567f8f03442f3487e7d3d8b; 4247 Byte

S07 - C90 Universal Transputer: gebundene Nachrichten und begrenzter Empfang

Die C90-Schnittstelle enthält einen Router, einen caller-eigenen Transportcallback und einen inkrementellen Empfänger. Eine 172-Oktett-QXT2-Route bindet Quelle, Ziel, Softwaregegenstand, Gesamtlänge und Nachrichtendigest. Höchstens 16 begrenzte Teile werden zusammengesetzt.

Gleiche Teilwiederholungen sind Duplikate. Widersprüchliche Teile oder Transfer-Schlüssel sperren die aktuelle Empfängerinstanz bis zur ausdrücklichen Neuinitialisierung. Vollständiger Empfang erfordert die vollständige Teil-Bitmaske und den berechneten Gesamtdigest. Ein erfolgreicher bytegeprüfter Empfang ist keine externe Wirkungsfreigabe.

Der boolesche C90-Referenzpfad gibt nur bei eindeutiger Bindung, Befugnis, Unterscheidung und Driftfreiheit ein gültiges Ergebnis frei. Die Header, Implementierungen und Testquellen sind archiviert. Dieser Band behauptet aus ihrer bloßen Existenz keine neue physische oder universelle Backend-Abnahme.

Archivkennung F094; Ursprungspfad next/core/include/qikvrt_transputer.h

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 6436fe95e18ad912fbc01899957698edd8e98d55; SHA-256 298d87d7fb87ffe82b08a7820635a4427cd3ce7035ccb8fa4a09146cc9bded81; 3989 Byte

S08 - Metatransistor: programmierbare RTL und vorhandene Testbench

Die vorhandene VHDL-Tile besitzt 32 parallele zweistellige boolesche LUT-Spuren, Zustandsnormalisierung und eine einstufige Ready/Valid-Ablage. Ergebnisse, Tags und Zustand bleiben bei Rückstau stabil. Nicht eindeutig binäre Kontroll-, Eingabe-, LUT- oder Tagwerte können HOLD bewirken und verhindern ein gültiges Berechnungsergebnis.

Das Zustands-Package verlangt eindeutige gültige Bindung, Befugnis und Unterscheidung sowie eindeutige Driftfreiheit. Die historische Basiskomponente gibt für allgemeine PASS-/DONE-Ausgänge ausdrücklich keinen allgemeinen Wirkungsnachweis aus. Basiskomponente, Package, Tile, C90-Referenz und Testbench sind vollständig archiviert.

Die Testbench enthält den beschriebenen endlichen Normalisierungsraum und taktsynchrone Prüfungen. Ihre Archivierung beweist das Vorhandensein des Programms, nicht eine hier neu beobachtete Simulation. FPGA-Programmierung, reales Board, ASIC-Fertigung, elektrische Eigenschaften und Patentrechte haben getrennte Nachweise und sind durch diesen Band nicht entschieden.

Archivkennung F095; Ursprungspfad next/hardware/metatransistor_tile.vhd

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob f123dd4f14e0002da5210a37b8bd1a071e6fffce; SHA-256 e77da78cafae0f16a080705c6cb51a76c45080ddf70cbd4d11bd62b3dafd7058; 2896 Byte

S09 - Öffentliche Patent-Offenlegungsgrenze als vorhandener Primärtext

Die bereits vorhandene öffentliche Patent Disclosure Boundary trennt technische Publikation von Anmeldung, Erteilung, Priorität, Patentierbarkeit, Rechtekette und Verwertung. Der Band archiviert diesen konkreten Text als Grundlage der wissenschaftlichen Reichweitenangabe im Artikel. Er ist kein neu entwickeltes Patentdossier und trifft keine neue Entscheidung über exklusive Rechte.

Ingolf Lohmanns Urheberschaft, Initiierung und Produktverantwortung werden als Herkunftszuordnung dokumentiert. Daraus wird in diesem Quellenband keine amtliche Anmeldebestätigung oder patentfachliche Ausschließlichkeitsentscheidung abgeleitet.

Archivkennung F096; Ursprungspfad
docs/publications/2026-07-22-qik-vrt-status-clarification/PATENT_DISCLOSURE_BOUNDARY.md

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob ae302b26a5b0e2ad092c9d6fddc4a462fff29dff; SHA-256
e665eb02d53dd6038a8754246dcf97ee27d91d1247e49dbe2a1e58cf96151f55; 6404 Byte

S10 - Originaler Ledger, lokaler Client und versiegelter S1-Start

Der ursprüngliche Ledger prüft native Ereignisse und bindet einen eindeutigen Schlüssel aus Quelle und Ereignis-ID. Er verwendet SQLite WAL/FULL, liest nach dem Commit den exakten Datensatz und benachrichtigt erst danach Beobachter. Der getrennte durable Client bereitet eine hashgebundene Eingabe vor, verwendet den privaten Unix-Socket und liest anschließend über eine neue nur lesende SQLite-Verbindung.

Der neue S1-Adapter prüft exportierte Quellblobs, Manifest, tatsächliche Runtime-Executables, private Konfiguration und State-Bindung. Eine geänderte Bindung erzeugt HOLD. Node 24 und Python 3.12 werden über die realen Programm-Digests gebunden; Git und GitHub CLI werden beim exportierten Startpfad nicht vorausgesetzt.

Firefox/noVNC wird durch die vorhandenen Komponenten getragen. Das neue Startverfahren bindet die tatsächlichen Programme, begrenzt die Listener und verwendet den bereits vorhandenen Ledger. Die vollständigen unmittelbar maßgeblichen Listings stehen in Anhang B; ergänzende Start-, Monitor-, Test- und Lizenzdateien stehen im bytegenauen Archiv.

Archivkennung F097; Ursprungspfad runtime/self-host/README.md

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 0f7fff71870dd68707e75325ae7acbd56901842a; SHA-256 862acc5d841f76be8dc587a1dfa0f1610ef274717c5e0696f8d324a7a05aed35; 8829 Byte

S11 - Lizenz- und Herkunftsangaben bleiben dateigebunden

Unveränderte Quelldateien und Lizenztexte behalten ihre eigenen Hinweise. Für Implementierungsdateien ist teilweise PolyForm Noncommercial angegeben; Dokumente können CC BY-NC-ND beziehungsweise andere ausdrücklich genannte Rechte tragen. Altbestände und Third-Party-Komponenten werden nicht automatisch unter eine neue gemeinsame Lizenz gestellt.

Ingolf Lohmann ist als menschlicher Initiator, Schöpfer und Product Owner benannt. In den konkreten Quellen aufgeführte OpenAI-Codex-Implementierungsbeiträge bleiben sichtbar. Die redaktionelle Archivaufbereitung schreibt solche Beiträge nicht nachträglich in menschlichen Ursprung um. Die Archivierung erteilt keine zusätzliche pauschale kommerzielle Nutzungs- oder Patentlizenz.

INVENTAR - Vollständige Herkunfts- und Hashübersicht

185 Quellfassungen sind im Archiv enthalten. Die historische und die S1-Fassung eines Originales werden getrennt gebunden, auch wenn ihre Bytes identisch sind. Vollständige maschinenlesbare Einträge stehen im Manifest. Die folgende Übersicht führt jede Archivkennung, Herkunft und SHA-256 auf.

F001 - deploy/universal-terminal/Dockerfile

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4957 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 97a03ff7b88218050b4f4295bcf6369ba03a402388364666cca32a0673d4b4f7

F002 - deploy/universal-terminal/Dockerfile

S1 3bf52747bc94de847d390ffacabecae183aee588 / 4957 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 97a03ff7b88218050b4f4295bcf6369ba03a402388364666cca32a0673d4b4f7

F003 - deploy/universal-terminal/cloud-entrypoint.sh

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 5001 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 97b2507ed2ac3955ed80682403ed482334201511849285c881e5c8739bfc6b9c

F004 - deploy/universal-terminal/cloud-entrypoint.sh

S1 3bf52747bc94de847d390ffacabecae183aee588 / 5001 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 97b2507ed2ac3955ed80682403ed482334201511849285c881e5c8739bfc6b9c

F005 - deploy/universal-terminal/compose.yaml

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 5661 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 4c2ca8207b24fcf0a88d55ddb46eac31117e636aaafb5a74cfbd91b078cfbaa1

F006 - deploy/universal-terminal/compose.yaml

S1 3bf52747bc94de847d390ffacabecae183aee588 / 5661 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 4c2ca8207b24fcf0a88d55ddb46eac31117e636aaafb5a74cfbd91b078cfbaa1

F007 - deploy/universal-terminal/db.qik-vrt.mesh

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 314 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 2b76539a1a5828fead91905b46dd62113b0a3e97985c5339d0e2077f95a28f95

F008 - deploy/universal-terminal/db.qik-vrt.mesh

S1 3bf52747bc94de847d390ffacabecae183aee588 / 314 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 2b76539a1a5828fead91905b46dd62113b0a3e97985c5339d0e2077f95a28f95

F009 - deploy/universal-terminal/entrypoint.sh

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 7470 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 c0bae910a3220b79d283943294b0ace65b8787c8f95b275a75ffc73d65fb139b

F010 - deploy/universal-terminal/entrypoint.sh

S1 3bf52747bc94de847d390ffacabecae183aee588 / 7470 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 c0bae910a3220b79d283943294b0ace65b8787c8f95b275a75ffc73d65fb139b

F011 - deploy/universal-terminal/mesh-index.html

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1604 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 f1d52c2edee53ebcb546803eb549aa0027121dc7b43fd3f861827612f0f162c6

F012 - deploy/universal-terminal/mesh-index.html

S1 3bf52747bc94de847d390ffacabecae183aee588 / 1604 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 f1d52c2edee53ebcb546803eb549aa0027121dc7b43fd3f861827612f0f162c6

F013 - deploy/universal-terminal/mirror-bootstrap.sh

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1900 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 f6889cda54e74b6fa3bc2faf9f73fbe9aca48c394d461769d1f91e5a24202edc

F014 - deploy/universal-terminal/mirror-bootstrap.sh

S1 3bf52747bc94de847d390ffacabecae183aee588 / 1900 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 f6889cda54e74b6fa3bc2faf9f73fbe9aca48c394d461769d1f91e5a24202edc

F015 - deploy/universal-terminal/named.conf

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 357 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 5df23f5a9d0217cbd38a1d86dd6b661f9cbeea72489a577ef1349948165c6d90

F016 - deploy/universal-terminal/named.conf

S1 3bf52747bc94de847d390ffacabecae183aee588 / 357 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 5df23f5a9d0217cbd38a1d86dd6b661f9cbeea72489a577ef1349948165c6d90

F017 - deploy/universal-terminal/nginx.conf

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4026 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 583491a285112a900cedb5b1e4dc001ddc1df1efd90d13f1f234127f0758489b

F018 - deploy/universal-terminal/nginx.conf

S1 3bf52747bc94de847d390ffacabecae183aee588 / 4026 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 583491a285112a900cedb5b1e4dc001ddc1df1efd90d13f1f234127f0758489b

F019 - deploy/universal-terminal/qikvrt_ip_bootstrap.c

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3992 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 73c9c63e410e4e3f0802627473eac9ae19d6c89ca85a07e75a0bd63136482a0e

F020 - deploy/universal-terminal/qikvrt_ip_bootstrap.c

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3992 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 73c9c63e410e4e3f0802627473eac9ae19d6c89ca85a07e75a0bd63136482a0e

F021 - deploy/universal-terminal/qikvrt_smtpd.py

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4396 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 9b68ef0b402ac269418571eebfc9aee5fd33eece849dbe259bd69b73a1e3ce29

F022 - deploy/universal-terminal/qikvrt_smtpd.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 4396 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 9b68ef0b402ac269418571eebfc9aee5fd33eece849dbe259bd69b73a1e3ce29

F023 - deploy/universal-terminal/runtime-health.sh

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1548 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 731d9e2adddf83999623424f9a306a6dd1ff2b142ec6a179cfd2c9ae12f13bf3

F024 - deploy/universal-terminal/runtime-health.sh

S1 3bf52747bc94de847d390ffacabecae183aee588 / 1548 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 731d9e2adddf83999623424f9a306a6dd1ff2b142ec6a179cfd2c9ae12f13bf3

F025 - deploy/universal-terminal/service-entrypoint.sh

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 5285 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 57c34784cf4a82a93a7639cf7917f37bb75e1a4b404e41243f890c5a85f06054

F026 - deploy/universal-terminal/service-entrypoint.sh

S1 3bf52747bc94de847d390ffacabecae183aee588 / 5285 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 57c34784cf4a82a93a7639cf7917f37bb75e1a4b404e41243f890c5a85f06054

F027 - deploy/universal-terminal/snmpd.conf

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 151 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 3d7caeb6b2f8e0a75f6c66a01b6520ba2403c5ac0b9f64837a11ee2e61e17528

F028 - deploy/universal-terminal/snmpd.conf

S1 3bf52747bc94de847d390ffacabecae183aee588 / 151 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 3d7caeb6b2f8e0a75f6c66a01b6520ba2403c5ac0b9f64837a11ee2e61e17528

F029 - deploy/universal-terminal/sshd_config

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 468 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 2effb563d95bdb35c9cb22ed144b718dd5ba6df509c70e3db8c4cb44f9b839b3

F030 - deploy/universal-terminal/sshd_config

S1 3bf52747bc94de847d390ffacabecae183aee588 / 468 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 2effb563d95bdb35c9cb22ed144b718dd5ba6df509c70e3db8c4cb44f9b839b3

F031 - docs/terminal/temdd/index.html

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 10647 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 847109600bdc487b3ba18d92393aa03370386780d8fdca2bd4529a7269d55819

F032 - docs/terminal/temdd/index.html

S1 3bf52747bc94de847d390ffacabecae183aee588 / 10647 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 847109600bdc487b3ba18d92393aa03370386780d8fdca2bd4529a7269d55819

F033 - browser/firefox/qikvrt-terminal/README.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 6577 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 f71fbf101d2d3a365fc094cd5697493826459688b377a67ca575f01499d2d9c6

F034 - runtime/self-host/original-firefox/README.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 6577 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 f71fbf101d2d3a365fc094cd5697493826459688b377a67ca575f01499d2d9c6

F035 - browser/firefox/qikvrt-terminal/_locales/ar/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3955 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 42ec0581a07b76d2c25d227c39151af2caaba7b8883551840db9497207a1672c

F036 - runtime/self-host/original-firefox/_locales/ar/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3955 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 42ec0581a07b76d2c25d227c39151af2caaba7b8883551840db9497207a1672c

F037 - browser/firefox/qikvrt-terminal/_locales/de/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3670 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 56986ec3e64d8e2a6f070bc438bd4c4e0cba1c3b53ccb8e6dc7f453a01abb4cb

F038 - runtime/self-host/original-firefox/_locales/de/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3670 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 56986ec3e64d8e2a6f070bc438bd4c4e0cba1c3b53ccb8e6dc7f453a01abb4cb

F039 - browser/firefox/qikvrt-terminal/_locales/en/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3411 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 ad4403073b812bce47d215ea936b010df68cd8cedaab9f7cc74fddf44c659a6d

F040 - runtime/self-host/original-firefox/_locales/en/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3411 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 ad4403073b812bce47d215ea936b010df68cd8cedaab9f7cc74fddf44c659a6d

F041 - browser/firefox/qikvrt-terminal/_locales/es/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3639 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 1e3037677d0abf91dfc9fb33c81fe81313e745f891e2143e46351e5886246090

F042 - runtime/self-host/original-firefox/_locales/es/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3639 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 1e3037677d0abf91dfc9fb33c81fe81313e745f891e2143e46351e5886246090

F043 - browser/firefox/qikvrt-terminal/_locales/fr/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3765 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 4d2265145bcc645b899b9d43b73e9fd002133ce99a80e78def9792c990fcf225

F044 - runtime/self-host/original-firefox/_locales/fr/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3765 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 4d2265145bcc645b899b9d43b73e9fd002133ce99a80e78def9792c990fcf225

F045 - browser/firefox/qikvrt-terminal/_locales/hi/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 5834 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 231458fdd2cd0a8d43da6382e304a12e548da389767de53f9eb1e4c2b7f13eb0

F046 - runtime/self-host/original-firefox/_locales/hi/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 5834 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 231458fdd2cd0a8d43da6382e304a12e548da389767de53f9eb1e4c2b7f13eb0

F047 - browser/firefox/qikvrt-terminal/_locales/id/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3542 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 09fb19851b9e9370524244db6bbc84931a354d942325ac49a6660a4e15f12d95

F048 - runtime/self-host/original-firefox/_locales/id/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3542 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 09fb19851b9e9370524244db6bbc84931a354d942325ac49a6660a4e15f12d95

F049 - browser/firefox/qikvrt-terminal/_locales/it/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3716 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 499086963505889846a271b5f0415d9b52a5884028634086019b88dc6cc10d34

F050 - runtime/self-host/original-firefox/_locales/it/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3716 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 499086963505889846a271b5f0415d9b52a5884028634086019b88dc6cc10d34

F051 - browser/firefox/qikvrt-terminal/_locales/ja/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4021 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 9e38ac2cdcf822cd21b0d7fdc9c3b915b94aa6770272780450d4a871bdf7ce3b

F052 - runtime/self-host/original-firefox/_locales/ja/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 4021 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 9e38ac2cdcf822cd21b0d7fdc9c3b915b94aa6770272780450d4a871bdf7ce3b

F053 - browser/firefox/qikvrt-terminal/_locales/ko/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3787 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 5dbfbf3ec9a2a2f7b9ebb217ad26221bbef89ebb203bf7c95be5c88646ca37c3

F054 - runtime/self-host/original-firefox/_locales/ko/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3787 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 5dbfbf3ec9a2a2f7b9ebb217ad26221bbef89ebb203bf7c95be5c88646ca37c3

F055 - browser/firefox/qikvrt-terminal/_locales/pt_BR/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3629 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 f6c849089370d4a13b73cef843fcb39a43c7fa7a402dbfbc16fbd7a8a572b3e5

F056 - runtime/self-host/original-firefox/_locales/pt_BR/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3629 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 f6c849089370d4a13b73cef843fcb39a43c7fa7a402dbfbc16fbd7a8a572b3e5

F057 - browser/firefox/qikvrt-terminal/_locales/ru/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4706 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 f006010304185d32003f0bed98ff42a76a3ad4d5f97b38371b4cb524ab10aea2

F058 - runtime/self-host/original-firefox/_locales/ru/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 4706 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 f006010304185d32003f0bed98ff42a76a3ad4d5f97b38371b4cb524ab10aea2

F059 - browser/firefox/qikvrt-terminal/_locales/tr/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3513 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 6f442192d324f983ede84f9735ae681e51e6235a6ab7b4dd61cdd9a53daf8775

F060 - runtime/self-host/original-firefox/_locales/tr/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3513 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 6f442192d324f983ede84f9735ae681e51e6235a6ab7b4dd61cdd9a53daf8775

F061 - browser/firefox/qikvrt-terminal/_locales/zh_CN/messages.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3350 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 e71fa8c582e6f67de441d2128ab4e63d8ad9a2e12c896837010bc618d9c0d8d4

F062 - runtime/self-host/original-firefox/_locales/zh_CN/messages.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3350 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 e71fa8c582e6f67de441d2128ab4e63d8ad9a2e12c896837010bc618d9c0d8d4

F063 - browser/firefox/qikvrt-terminal/authenticated_delivery.js

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 15796 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 8b62b0dd7beadc24b5d037102d10b3590714e446298e833a5ae41eeeea74ff26

F064 - runtime/self-host/original-firefox/authenticated_delivery.js

S1 3bf52747bc94de847d390ffacabecae183aee588 / 15796 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 8b62b0dd7beadc24b5d037102d10b3590714e446298e833a5ae41eeeea74ff26

F065 - browser/firefox/qikvrt-terminal/background.js

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 15060 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 d76e058245d216d28c78065031006377b80e6396f102356c3c359e3635961cc3

F066 - runtime/self-host/original-firefox/background.js

S1 3bf52747bc94de847d390ffacabecae183aee588 / 15060 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 d76e058245d216d28c78065031006377b80e6396f102356c3c359e3635961cc3

F067 - browser/firefox/qikvrt-terminal/content.js

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 9424 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 f41f930c6e0600956d1436e7f25c100c8d55200ca6b20e3581bf3aab4f4bcbae

F068 - runtime/self-host/original-firefox/content.js

S1 3bf52747bc94de847d390ffacabecae183aee588 / 9424 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 f41f930c6e0600956d1436e7f25c100c8d55200ca6b20e3581bf3aab4f4bcbae

F069 - browser/firefox/qikvrt-terminal/manifest.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1578 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 f17889e6922a1c0eb84f949c49c1ef20ba73e80f10ca8e36c0c23f585d0bed65

F070 - runtime/self-host/original-firefox/manifest.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 1578 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 f17889e6922a1c0eb84f949c49c1ef20ba73e80f10ca8e36c0c23f585d0bed65

F071 - browser/firefox/qikvrt-terminal/options.html

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1634 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 5c972e6528d43608f5ecf754054ddaa22992e53d9a49d93c3d258b0b7e88c5b3

F072 - runtime/self-host/original-firefox/options.html

S1 3bf52747bc94de847d390ffacabecae183aee588 / 1634 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 5c972e6528d43608f5ecf754054ddaa22992e53d9a49d93c3d258b0b7e88c5b3

F073 - browser/firefox/qikvrt-terminal/options.js

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1154 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 a20467bc0d329369b562a02909412bbbc6596cd9e5e3f27530fcfa4373669f4d

F074 - runtime/self-host/original-firefox/options.js

S1 3bf52747bc94de847d390ffacabecae183aee588 / 1154 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 a20467bc0d329369b562a02909412bbbc6596cd9e5e3f27530fcfa4373669f4d

F075 - browser/firefox/qikvrt-terminal/review_effect.js

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 5377 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 a905219e62db17e5a9f355bbcb10e64efdeb9c02472572967cb53dafa0e114be

F076 - runtime/self-host/original-firefox/review_effect.js

S1 3bf52747bc94de847d390ffacabecae183aee588 / 5377 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 a905219e62db17e5a9f355bbcb10e64efdeb9c02472572967cb53dafa0e114be

F077 - browser/firefox/qikvrt-terminal/terminal.css

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3536 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 769dffd97cb95d995a4b72b073cbef669391421b0c4acabf07f9ef79cb1f99ed

F078 - runtime/self-host/original-firefox/terminal.css

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3536 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 769dffd97cb95d995a4b72b073cbef669391421b0c4acabf07f9ef79cb1f99ed

F079 - .github/workflows/qikvrt_temdd.yml

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 13389 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 5ecd9a8470f02da7e32da16812fe61f2fe42e214b00bec2d08ee139daa5e99c8

F080 - runtime/self-host/original-workflows/qikvrt_temdd.yml

S1 3bf52747bc94de847d390ffacabecae183aee588 / 13389 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 5ecd9a8470f02da7e32da16812fe61f2fe42e214b00bec2d08ee139daa5e99c8

F081 - src/qikvrt_temdd_event_ledger.py

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 21653 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 94908e6a50ce107a79965055f3ecabae10e7589d96f7a26496d2a4956b068b13

F082 - src/qikvrt_temdd_event_ledger.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 21653 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 94908e6a50ce107a79965055f3ecabae10e7589d96f7a26496d2a4956b068b13

F083 - tests/test_qikvrt_temdd_public_proxy.py

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 8652 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 505d1608109f75caac698e2af97278a58431b7a2235f8f2ce03ed98cb3a962ee

F084 - tests/test_qikvrt_temdd_public_proxy.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 8652 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 505d1608109f75caac698e2af97278a58431b7a2235f8f2ce03ed98cb3a962ee

F085 - tests/test_temdd_event_ledger.py

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 21663 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 662d41da4b1e15237ee73ce73347d090c32e6fa7dcaaea208e95938cf20b3966

F086 - tests/test_temdd_event_ledger.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 21663 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 662d41da4b1e15237ee73ce73347d090c32e6fa7dcaaea208e95938cf20b3966

F087 - tests/test_temdd_runtime_subject_binding.py

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3806 Byte / ORIGINAL_HISTORICAL_BYTES

SHA-256 e6c1c98665c50fdf2c5de1ac29f17f5ccf78aefdcdbd9abb4e700aeb5ac6bc4a

F088 - tests/test_temdd_runtime_subject_binding.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3806 Byte / RESTORED_BYTE_IDENTICAL_ORIGINAL

SHA-256 e6c1c98665c50fdf2c5de1ac29f17f5ccf78aefdcdbd9abb4e700aeb5ac6bc4a

F089 - runtime/self-host/RECOVERED_TERMINAL.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 5569 Byte / CITED_PRIMARY_SOURCE

SHA-256 17cce25cdc39af59608f1bfe872c2b0a9f766a56ffd146b31ea84baf274456d4

F090 - state/work_units/S1_PRODUCTIVE_TERMINAL_SOURCE_RECOVERY_20261005.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 27195 Byte / CITED_PRIMARY_SOURCE

SHA-256 faba273fe232e6b2b78f0a31ab68e058e0ffdd20bcbd3fd748ebb5ff149138b9

F091 - .github/copilot-instructions.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 509 Byte / CITED_PRIMARY_SOURCE

SHA-256 ea5f6b0c20fb403a3da94f4687e2a3448ca0dd43060495e5cfda5afea63a01fb

F092 - spec/temdd/TEMDD_LANGUAGE_SPEC_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3330 Byte / CITED_PRIMARY_SOURCE

SHA-256 8bbc29511a6d09ac58754234f14f09752fc53482ce10e8a3051d4581e1afdf59

F093 - spec/temdd/TEMDD_NORMATIVE_CORE_V1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4247 Byte / CITED_PRIMARY_SOURCE

SHA-256 4b4c32b98c7482a570b4b8cb85b86af053cc3ecf3567f8f03442f3487e7d3d8b

F094 - next/core/include/qikvrt_transputer.h

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3989 Byte / CITED_PRIMARY_SOURCE

SHA-256 298d87d7fb87ffe82b08a7820635a4427cd3ce7035ccb8fa4a09146cc9bded81

F095 - next/hardware/metatransistor_tile.vhd

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 2896 Byte / CITED_PRIMARY_SOURCE

SHA-256 e77da78cafae0f16a080705c6cb51a76c45080ddf70cbd4d11bd62b3dafd7058

F096 - docs/publications/2026-07-22-qik-vrt-status-clarification/PATENT_DISCLOSURE_BOUNDARY.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 6404 Byte / CITED_PRIMARY_SOURCE

SHA-256 e665eb02d53dd6038a8754246dcf97ee27d91d1247e49dbe2a1e58cf96151f55

F097 - runtime/self-host/README.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 8829 Byte / CITED_PRIMARY_SOURCE

SHA-256 862acc5d841f76be8dc587a1dfa0f1610ef274717c5e0696f8d324a7a05aed35

F098 - src/qikvrt_effect_ack_http_terminal.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 30823 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 039b7c34fe4ae8766ae0b141734ef91131377e5e58fedb7746320242543da2d8

F099 - tools/qikvrt_self_host.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 28982 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 75d334b48eefb837ad02b285765764b5a0912d6b0bc2596284a7ae2ba02c1345

F100 - docs/monitor/self-host.mjs

S1 3bf52747bc94de847d390ffacabecae183aee588 / 9774 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 5620d5840849f6e96f97a04bf5b631be4a534c2b4ccaead0767442f254cd845f

F101 - tools/qikvrt_mesh_monitor_readback.mjs

S1 3bf52747bc94de847d390ffacabecae183aee588 / 10198 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 e2c674f221255e9ed2cf495bb897181a8352420acb656bd7fb7e0e8193ced1dd

F102 - tools/bootstrap-runtime.sh

S1 3bf52747bc94de847d390ffacabecae183aee588 / 18480 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 4943adc0ff18dd83b507d64b29204452c46f7e6d3c483c31e88926a978a00d52

F103 - tools/bootstrap-runtime.ps1

S1 3bf52747bc94de847d390ffacabecae183aee588 / 23788 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 02dc5704f08c405c46a61cdd64679e53f352aa7722e1e7ae517ee9279adc0b8b

F104 - runtime/self-host/PACKAGE.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3910 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 44b675f6fcf1abb653dafd79197da9f77a2077a7e3fe7fd658503208a878fb91

F105 - runtime/self-host/CONFIG.example.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 422 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 c9ce35bb8897d8ccd3e5b726422b2e09bb534ef6519ddf90f0fe6dbce3ffb424

F106 - runtime/self-host/Dockerfile

S1 3bf52747bc94de847d390ffacabecae183aee588 / 1208 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 d393d3eb7eaae2c1dcd4404ef6cf86600ea794a068ff32cf3a793d3abb0fec3b

F107 - .github/workflows/qikvrt_effect_ack_http_terminal.yml

S1 3bf52747bc94de847d390ffacabecae183aee588 / 8111 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 eede5fcbc774baf03a7188261776651416b5c3cab51efcee6426ad8ecd87925c

F108 - .github/workflows/qikvrt_temdd.yml

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3058 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 90278614a3befd6353181998cd97a55b5102c1f863e4633706088354cf14b653

F109 - tests/test_self_host.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 20746 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 372d2265978332beb02da44bd277d0d78924f418fe0a369a94b99844a1f15e22

F110 - tests/test_self_host_temdd.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 10400 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 eaabaa8149736bf638741168e26e7c3fbb8feec48c7391edeac04556d90c160b

F111 - tests/test_self_host_firefox.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 10825 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 f4365f8c37b776be987e0f6cf08badec013c39e18751d42af24d659769a8fc29

F112 - tests/test_qikvrt_effect_ack_http_terminal.py

S1 3bf52747bc94de847d390ffacabecae183aee588 / 26955 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 6a64163f69258bd53b711a8a764a893ddd70ae48cb7a2facd1941d8ad4689143

F113 - state/work_units/SELF_HOSTED_NODE_PACKAGE_S1_20261004.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 27261 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 60a326d5ebe23f5fce93320e3de39d325b0cecadbf3050adb111f5a41d5c0e53

F114 - evidence/self_host/SOURCE_RECOVERY_20261005.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 120508 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 ac8e345f2b62a562d65cc670274959d41f1df4dbbde4229c2c4af61de815dd7e

F115 - evidence/self_host/SOURCE_RECOVERY_LOCAL_20261005.json

S1 3bf52747bc94de847d390ffacabecae183aee588 / 2366 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 c829c61c6f6ec6f9dfe2261b6cf1ca24ac46afc9a0ea0e31085109ab5981e78c

F116 - LICENSE

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3279 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 b72bb73f40c5b63f77e4efa80a408d52b64c98ae4f0cb4cc99b86072285258f6

F117 - LICENSE.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 4075 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 00d9148cf5d0db9ffb5a8f4d8095c0694b738531a17a5af2691a2dff01ad574d

F118 - LICENSE_NOTICE.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 1201 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 c7bd3acf5b8723aa558f653ba821a3aaa3d55450b84dff5c878f1618abd22895

F119 - NOTICE

S1 3bf52747bc94de847d390ffacabecae183aee588 / 737 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 cc2580944c52971c424165c3684acadcf8faa5385e73f3f67ac64dede34a28e9

F120 - LICENSE-CODE-APACHE-2.0.txt

S1 3bf52747bc94de847d390ffacabecae183aee588 / 734 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 2f3e4ba2e1025aa5282ae21a8850863c1a55b7aae983919b51986918a0533d56

F121 - LICENSE-CODE-POLYFORM-NONCOMMERCIAL-1.0.0.txt

S1 3bf52747bc94de847d390ffacabecae183aee588 / 929 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 c4c7f5443b9a1dc32a0d701d3ff2531d3c8be5e2cef81f2444e1e8b3dcf5ade0

F122 - LICENSE-DOCS-CC-BY-NC-4.0.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 788 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 0fe6ad304320316fbfcc40c50c8d92e24fc03240d11a281a1067e7769c27b06c

F123 - LICENSE-DOCS-CC-BY-NC-ND-4.0.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 725 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 2785c9eb33bebd00537011b8b6d017a5c4ad4d271db59af8889087fceaab3387

F124 - docs/publications/2026-07-22-qik-vrt-status-clarification/LICENSE_NOTICE.md

S1 3bf52747bc94de847d390ffacabecae183aee588 / 3930 Byte / CURRENT_S1_SOURCE_OR_LICENSE

SHA-256 59358ce10f9d2e85849580b7fb15c7952f48ae4830bbd6b3b5362184ee5a31d5

F125 - next/core/Makefile

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1538 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 5fcddc08c8771c7bd713232429f2ac70ffa4067dc8bce22a88f46aaf5d7a3f76

F126 - next/core/adapters/stdio.c

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3190 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 172c6f1756e330a52d589c7952254002a9cd8a81eceada9204c3142bd862e495

F127 - next/core/include/qikvrt_bus.h

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1697 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 82b96016f65087b11802b119ce2b712c40e35a2aa7cb2bdd6e597fa31d6b8f1a

F128 - next/core/include/qikvrt_types.h

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 690 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 e50e5f115651ae7ca922d9cc4e8dd79b608433e6d3aea2959d42a72846e4bc97

F129 - next/core/include/qikvrt_wire_v1.h

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1791 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 6a7e9d0ba1c3852f3900dd608744429f066ad4972fef181f517217a81f8f556f

F130 - next/core/src/qikvrt_bus.c

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4915 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 2af20b548426d31ed98e2dc3a656bd81649d2b5a636203921e686c75619e83fb

F131 - next/core/src/qikvrt_exchange.c

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 5656 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 eea181a8a0d396cc81ebf8ca38f06960429539f8dfe79b611eb21a98d1e7f6f9

F132 - next/core/src/qikvrt_kernel.c

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 2019 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 6b57865464a9ff9acf0f55daa29516163396b6f156582026ad162f9ac87b46d2

F133 - next/core/src/qikvrt_sha256_v1.c

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 2758 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 2d64f219003a97fe70f6a8e83678df6c73ee35d44dd0d23c00d99730b8a0220b

F134 - next/core/src/qikvrt_wire_v1.c

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4207 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 5adb12d1a66005b35b03776c415780a18c5bb563eec7c390e56a7779d9e53db4

F135 - next/core/tests/test_core.c

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4879 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 4669df19386dc2203e5e732d0e3ec9cd09f3e535ca0737d3acadfb62de4c2e86

F136 - spec/temdd/README.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1122 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 a3f51a8b3361c438ecc6f05caf8b13125ca3de6f16b8885123ce06ccf02a45ca

F137 - spec/temdd/TEMDD_ARCHITECTURE_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 722 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 5524cd716cf4b52ee0d35eaf111c282fbaf9efc0facb9c8793a97ab7e8158723

F138 - spec/temdd/TEMDD_AUTHORITY_MODEL_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 331 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 cb690af7b9debb36681af7dfb6dfe8d5ef7d13bb794cfa0b8c17cd4582670b3b

F139 - spec/temdd/TEMDD_BACKEND_CONTRACT_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 532 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 e4b7fd4a313f6643286b41802f0dd82d4255a49c4815c3280cc42541bd9d47dd

F140 - spec/temdd/TEMDD_BOOTSTRAP_DOD_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 651 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 9d8dd8d09ab3f7fd5e358126e1c2db360ffee67119e940bd2513a23d4d341959

F141 - spec/temdd/TEMDD_BOOTSTRAP_MANIFEST_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 626 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 59d34152d9b712c39f74fdddd3719600f32ce438cfc759fa2c7ed0d10b95f540

F142 - spec/temdd/TEMDD_BOOTSTRAP_RECEIPT_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 720 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 5875b76eefdcf29b6f160d2d24d23e853b3cfa5d143dc1f13d4d646dc2fc420a

F143 - spec/temdd/TEMDD_CD_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 522 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 e26ce065595411a3d426bc9aaf90f97f47b0178896de8cc6ac47f1c4add317a6

F144 - spec/temdd/TEMDD_COMPILER_CONTRACT_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 383 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 62fa7d01f65cab82f1c7aa986245f3ebcb9e62d60bb8bd72414ca04612d44719

F145 - spec/temdd/TEMDD_CONFORMANCE_MATRIX_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 561 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 18173fbdc40c77af2a2c4b1145f802dd2fd4b2e345a3647b47476588235c2be9

F146 - spec/temdd/TEMDD_CONFORMANCE_MATRIX_V1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 987 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 8698ce0e33cd89c4062e044f93747aaa5cbfe48ef89f8a237638b3863d95dc94

F147 - spec/temdd/TEMDD_CONFORMANCE_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1615 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 a74cdebb0b0803d1b34358c71aaaed1888317193888ec3bf4ac1fc7dd3abcaaa

F148 - spec/temdd/TEMDD_CONFORMANCE_V1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1203 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 68c345367c2f3e40eb1d0b2a1419b21b932b8ba9c5a11f7f1b4c96c08c268b95

F149 - spec/temdd/TEMDD_DESIGN_RATIONALE_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 600 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 8f408f79f4853e225c61eaa54762ee6f3f733be85d0b182e25febc674df85f74

F150 - spec/temdd/TEMDD_DOGFOOD_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 478 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 6535abd308d389c68891fe0e4c91b703a803e77ec6b156e60f44a5bfce3bb7e1

F151 - spec/temdd/TEMDD_EFFECT_MODEL_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 440 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 8ccbe1083222aadfaefc2b12d665863169ed02b05c22866ff79698522648c4bd

F152 - spec/temdd/TEMDD_EVENT_MODEL_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 382 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 d910000ec00e8b9afe6f3a759d9cafcb2c2e1b45e049d885f0e5cd2d8bd56ee1

F153 - spec/temdd/TEMDD_EVIDENCE_MODEL_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 313 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 19efb9e04173d5c2d5254ef46f7b8ed470d033b13ddbd1802522c6edca45c75c

F154 - spec/temdd/TEMDD_GLOSSARY_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 735 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 4bc52c643f499ba397472a6c10f64aa674ace6f6481884b3a30ab073fd2243ab

F155 - spec/temdd/TEMDD_IDE_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 955 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 1b5b151c090508999c1fa3d5edd0d64711149a794e8e18a9fbeec29c5dd94214

F156 - spec/temdd/TEMDD_IMPLEMENTATION_MAP_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 586 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 0b08882dc225ebf9d15f0dc823b81bda72476f46a843776d8536760eaeaa7b3f

F157 - spec/temdd/TEMDD_INTEROP_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 471 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 435996f35834f77fed0dfc44ace8032cd6896fd807ff6925a8ca94f796d69618

F158 - spec/temdd/TEMDD_LICENSE_AND_PROVENANCE_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 463 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 48f53f9a2f9a145ac911b3f750da27cebb1227da59a74be01d2c8e967d2c0316

F159 - spec/temdd/TEMDD_MIGRATION_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 572 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 a98b752b3f62a98ad146522b7c9ab0e8ba785da84a955f9d140438164bd021b3

F160 - spec/temdd/TEMDD_OBSERVABILITY_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 366 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 4015574f1a9abe0d4eeb1ebf827b4826f483343a3daccfc6c6cd014f15069567

F161 - spec/temdd/TEMDD_PERFORMANCE_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 498 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 d5dac4dac992daa5c68e5c401c7a4dd89fffe5f47cf55179f76dcb73196bada9

F162 - spec/temdd/TEMDD_PERSISTENT_EVENT_LEDGER_V1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 7530 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 e477d0f21845cd5e4b11aa4e824adb18a0426c94c77a3ed5c3c0c3e232864c65

F163 - spec/temdd/TEMDD_PORTABILITY_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 527 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 9e5a935725de963b07b7f8ed32231722401c265260bbf5e5415ef5815c55a674

F164 - spec/temdd/TEMDD_RELEASE_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 570 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 02f07166a7b588ba8f992a0d8d3f9c4c4c335c83399c1b54f5c45020f4d716df

F165 - spec/temdd/TEMDD_ROADMAP_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 545 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 4a3576e39b1cf859b089994ccf4a00c6be2d64b3703b1eb8019b1eee545acda1

F166 - spec/temdd/TEMDD_SECURITY_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 623 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 482cfb7f288e78b6e4b035982156ea614f9bf02802138e79639725c718cafd55

F167 - spec/temdd/TEMDD_SELF_IMPROVEMENT_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 498 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 e34b5aacd56be3cae2d2ab60a50ef9b125d2ef6b3f2a20c7bbde6ef2932e0901

F168 - spec/temdd/TEMDD_STATUS_MODEL_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 413 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 511f3eddd6dff9aec4b7cf8558b2ff85ed0d4c1cb062cc02b7302efeb4594703

F169 - spec/temdd/TEMDD_Syntax_V0_1.ebnf

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1567 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 dbfe62dc5e9fce8b91d06cc984ba9760fa90a67fff8961c5f86b763606148b6a

F170 - spec/temdd/TEMDD_TESTING_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 527 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 fb3822e3063319c870404d1b6f325f08a1390e210020c6a9d0a1f5c62ef58605

F171 - spec/temdd/TEMDD_VALIDATION_PLAN_V0_1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 449 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 ea9d16b50376535d03635778b73b6b2f7015c6c63f27798fe8be229fc6ef8a15

F172 - spec/temdd/TEMDD_VERSIONING_V0_1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 549 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 929e08eb6bf3a509ab0b1287a7f24e8162ebbd718508c2e3dae6c679b06ebce1

F173 - hardware/vhdl/qikvrt_metatransistor.vhd

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1647 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 dc5c6f40dea9d70bf21fa320dbc5f775756d681f80cf7c4e68adf5133fd2e886

F174 - hardware/vhdl/qikvrt_metatransistor_pkg.vhd

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1619 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 d4abe16e65600c4b4a54b6dbaab9412ad03b1df231da1e44a4e4ccfd98e5cb49

F175 - hardware/vhdl/tb_qikvrt_metatransistor.vhd

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4987 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 ba92c7cf916598be8303382cd37abf2dff2ad76d5aba77db480e1030406df3c9

F176 - tools/qikvrt_temdd.py

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 2042 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 6d086aad67b10fe5878bad6b13beacf5f4f51e21f457d7e678816ea64bf8e667

F177 - src/temdd/v1_adapter.py

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1356 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 7f6248d5ac21c79875b985ba31fb03d86108b68797a75756cc02c3113b38ee0e

F178 - docs/terminal/QIKVRT_CLOUD_TRANSPUTER_MATERIALIZATION_V1.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3282 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 da82c8748a04f6e2935c14de473d8c7d4ab6043435254af3f874a010ba7e625c

F179 - state/work_units/UNIVERSAL_TRANSPUTER_C90_BUS_20260921_V1.json

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 5057 Byte / HISTORICAL_SOURCE_SPECIFICATION_OR_TEST

SHA-256 8d56240fc55084e7da39d15976a6e597b65d73724f8abfef5a2d3f22af56beea

F180 - LICENSE

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 3279 Byte / HISTORICAL_LICENSE_OR_NOTICE

SHA-256 b72bb73f40c5b63f77e4efa80a408d52b64c98ae4f0cb4cc99b86072285258f6

F181 - LICENSE.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 4075 Byte / HISTORICAL_LICENSE_OR_NOTICE

SHA-256 00d9148cf5d0db9ffb5a8f4d8095c0694b738531a17a5af2691a2dff01ad574d

F182 - LICENSE_NOTICE.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 1201 Byte / HISTORICAL_LICENSE_OR_NOTICE

SHA-256 c7bd3acf5b8723aa558f653ba821a3aaa3d55450b84dff5c878f1618abd22895

F183 - NOTICE

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 737 Byte / HISTORICAL_LICENSE_OR_NOTICE

SHA-256 cc2580944c52971c424165c3684acadcf8faa5385e73f3f67ac64dede34a28e9

F184 - LICENSE-CODE-POLYFORM-NONCOMMERCIAL-1.0.0.txt

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 929 Byte / HISTORICAL_LICENSE_OR_NOTICE

SHA-256 c4c7f5443b9a1dc32a0d701d3ff2531d3c8be5e2cef81f2444e1e8b3dcf5ade0

F185 - LICENSE-DOCS-CC-BY-NC-ND-4.0.md

historisch c6a53e200c4835a4f871d8496feb81d953610831 / 725 Byte / HISTORICAL_LICENSE_OR_NOTICE

SHA-256 2785c9eb33bebd00537011b8b6d017a5c4ad4d271db59af8889087fceaab3387

ANHANG A - Vollständige zitierte Primärtexte

Die folgenden Lesefassungen enthalten den gesamten Text der bezeichneten Primärdateien. Zeilennummern und Druckumbrüche dienen ausschließlich der Lesbarkeit. Eventuell im Original enthaltene externe Links bleiben als Bestandteil des zitierten Dokuments erkennbar; sie sind keine neue wissenschaftliche Ersatzbibliographie. Das ZIP enthält die unveränderten Datei-Bytes.

A1 - S01 / runtime/self-host/RECOVERED_TERMINAL.md

Archivkennung F089; Ursprungspfad runtime/self-host/RECOVERED_TERMINAL.md

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 43b677d3044a71cccc2a0b9a19a1fd2741191a69; SHA-256 17cce25cdc39af59608f1bfe872c2b0a9f766a56ffd146b31ea84baf274456d4; 5569 Byte

```
<!-- SPDX-License-Identifier: CC-BY-NC-ND-4.0 -->
# Recovered historical TEMDD and Firefox/noVNC carrier

`architecture/universal-transputer-c90-v1` retains exact snapshot commit
`c6a53e200c4835a4f871d8496feb81d953610831`, tree
`f1640191ec25495c90e997b38e77490995808b1c`. Its commit declares Authority source
`b1971ca5003f5ac63fb8baf0bc663195f7ebcb78`. The snapshot commit/tree and restored
blob bytes were retrieved; the declared Authority commit is not independently
reachable through this connection. Neither equality with today's Railway build
nor completeness of later Authority changes is inferred.

`evidence/self_host/SOURCE_RECOVERY_20261005.json` binds every restored path by
size, SHA-256 and original Git blob. The native daemon, IDE, deployment recipes,
original tests and original Firefox extension source are historical original
bytes. The extension is preserved separately under `original-firefox/`; it is
not silently installed or substituted for the currently selected client.

The historical Dockerfile/entrypoints assume the full original `/opt/qikvrt`
checkout and its broader Cloud Transputer dependencies. They are preserved as
source evidence, not invoked against the narrower S1 export. The new portable
adapter in `tools/qikvrt_self_host.py` replaces only the Git/layout binding and
child startup. It reuses the unmodified original `Ledger`, HTTP handler and
owner-only Unix ingress. These adapter changes are **new implementation**, not
restored original source. No replacement ledger, scheduler or effect executor
is introduced.

## Native portable profile

The example config now selects `terminal_profile: temdd`, an explicit
`source_repository` and `subject_pr`. Startup and every native operation verify
the independently supplied manifest pin, exact exported source, executable
hashes, private config and volume binding. This path needs Node 24 and Python
3.12, but neither Git nor gh nor a provider network. Legacy config without an
explicit profile retains the process-local `reference` profile.

The existing durable CLI accepts `--manifest-sha256 <independent-pin>` with its
unchanged input, preparation/hash, repository/PR/head/tree and state-directory
arguments. This explicit path verifies the sealed export instead of a Git
checkout. Prepare still inserts no event; Commit uses the existing mode-0600
Unix ingress and owner UID; receipt verification uses a fresh read-only SQLite
connection. The recovered daemon keeps original native-ID conflict detection,
WAL/FULL commit, epoch, event bytes and SSE replay. HTTP writes are refused.

The public monitor `/api/terminal` reads and validates the native subject and
epoch, returning sanitized metadata only. Private event bodies and native SSE
remain on the loopback owner interface. Public routing and access to personal
browser state require the host's separately admitted authenticated HTTPS route.

## Firefox/noVNC profile

`runtime/self-host/Dockerfile` provisions a compatible dependency image from
digest-pinned official Python 3.12 and Node 24 bases and Debian packages. A
builder with these tools can freeze a browser variant using
`pack --browser-assets-root /usr/share/novnc`. Its manifest binds every admitted
noVNC source file and all six executable hashes. Third-party distribution
copyright metadata is retained. No binaries, credentials or live profiles enter
the source package. Package assembly is not a distribution-wide reproducible
APT-build claim: the actual build package versions and image are retained in CI.

Set `terminal_profile: firefox`, explicit separate `novnc_port`, `vnc_port`,
`display` (for example `:99`) and an owner-only `browser_password_file` containing
exactly eight printable ASCII bytes. The adapter starts Xvfb, x11vnc,

websockify and Firefox on the native IDE. VNC authentication is mandatory; every
VNC/noVNC listener remains loopback. Both x11vnc and LibVNCServer IPv6
listeners are disabled; the configured IPv4 port is the only VNC listener. No unsigned extension/signing
exception is
```

```
enabled. Bounded startup checks require a real Firefox Navigator window and
byte-exact noVNC HTTP response. Linux pidfds supervise actual child exits without
runtime polling. Profile/state/logs are private and outside the export.

The existing terminal workflow builds the dependency image and runs the actual
candidate export in `docker run --init --network none`, with a private temporary
volume. It checks Firefox/X11, noVNC WebSocket/RFB authentication negotiation,
native Prepare/Commit/SQLite/SSE and crash/restart preservation. The container
must provide a real init reaper; operators use their existing host supervisor
with process-group termination and orphan reaping. Kernel PID and TCP-listener
readbacks verify crashed Xvfb removal and only four declared loopback ports.
Artifacts bind
the actual candidate head/tree, package, binary/source hashes and fresh runtime
readbacks. A missing dependency or failed runtime is an error, not a skipped
acceptance case.

## Remaining acceptance

Historical source recovery closes the claim that **no native source carrier is
reachable** for this bounded snapshot. Fresh CI runtime readback must still be
observed on the successor. The independent-host identity, persistent-device
attestation, authenticated public HTTPS routing and deployed restart remain
separate acceptance obligations. No deployment, public-runtime acceptance,
S1 DONE, Authority/Mirror equality or `EFFECT_ACK_DONE` follows from recovery or
CI. The configured local filesystem is not an attestation of power-loss safety.
```

A2 - S02 / state/work_units/S1_PRODUCTIVE_TERMINAL_SOURCE_RECOVERY_20261005.json

Archivkennung F090; Ursprungspfad state/work_units/S1_PRODUCTIVE_TERMINAL_SOURCE_RECOVERY_20261005.json

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 8ae56034298cd526b61de2a98b4f363dff6c7d9c; SHA-256 faba273fe232e6b2b78f0a31ab68e058e0ffdd20bcbd3fd748ebb5ff149138b9; 27195 Byte


```
0001  {
0002    "artificial_contributor": {
0003      "identity": "OpenAI Codex",
0004      "model_family": "GPT-6",
0005      "model_or_build_identity": "UNAVAILABLE",
0006      "retention": "METADATA_ONLY",
0007      "role": "history/ref/PR/artifact search, exact original-source recovery, sealed-source/start
      adapters, runtime tests and candidate verification"
0008    },
0009    "authorization": {
0010      "candidate_implementation_and_push": "CURRENT_OWNER_DIRECTIVE",
0011      "hosting_account_creation_or_spending": false,
0012      "main_merge": "NOT_PERFORMED; MAIN_HISTORY_ONLY_MERGED_INTO_CANDIDATE",
0013      "platform_account_or_protection_changes": false
0014    },
0015    "boundaries": {
0016      "authority_effect": false,
0017      "complete_original_repository_restoration": false,
0018      "deployment_performed": false,
0019      "effect_ack_done": false,
0020      "historical_snapshot_path_equality": true,
0021      "host_persistence_attested": false,
0022      "host_restart_verified": false,
0023      "ordinary_release": false,
0024      "predecessor_evidence_transfer": false,
0025      "provider_independent_public_runtime_verified": false,
0026      "public_readback_verified": false,
0027      "public_url": null,
0028      "source_recovery_proved_unrecoverable": false
0029    },
0030    "deployment_hold": {
0031      "authorized_deployment_identity": null,
0032      "bound_persistent_mount_or_device": null,
0033      "callable_own_host_operation": null,
0034      "code": "HOLD_OWN_HOST_IDENTITY_STORAGE_ROUTING_UNAVAILABLE",
0035      "public_route": null,
0036      "retry_condition": "Authorized callable host, exact identity, persistent device and authenticated
      public HTTPS, then exact install/use/readback/deployed restart"
0037    },
0038    "exact_head_gates": {
0039      "firefox_runtime_command": "docker run --network none ... python3 -B -m unittest -v
      tests.test_self_host_firefox",
0040      "readback_artifact": "qikvrt-productive-terminal-readback-<actual-candidate-head>",
0041      "run_binding_location": "Native GitHub Actions metadata/checks and exact-head readback artifacts; no
      self-reference of containing commit",
0042      "skipping_or_gate_weakening": false,
0043      "state": "REQUIRED_FRESH_ON_COMMITTED_SUCCESSOR; NO_INHERITED_PASS",
0044      "test_command": "make test repository-monitor-test self-host-test"
0045    },
0046    "human_principal": {
0047      "instruction_date": "2026-10-05",
0048      "instruction_source": "CURRENT_USER_ACTION",
0049      "name": "Ingolf Lohmann",
0050      "purpose": "Search all reachable source history; recover original productive TEMDD/Firefox/noVNC
      blobs with provenance or explicitly reconstruct only if unavailable; validate provider-independent start;
       no deployment or EFFECT_ACK_DONE without fresh runtime readback.",
0051      "roles": [
0052        "creator",
0053        "product_owner",
0054        "code_owner"
0055      ]
0056    },

0057    "implementation_files": [
0058      {
```

```
      "bytes": 8111,
      "git_blob_sha1": "2e1cf1b67fc740681e7cd7a4e348f6da299f974c",
      "path": ".github/workflows/qikvrt_effect_ack_http_terminal.yml",
      "sha256": "eede5fcbc774baf03a7188261776651416b5c3cab51efcee6426ad8ecd87925c"
    },
    {
      "bytes": 3058,
      "git_blob_sha1": "cffcb872da82ac29d6b6f312d255d7a38948506f",
      "path": ".github/workflows/qikvrt_temdd.yml",
      "sha256": "90278614a3befd6353181998cd97a55b5102c1f863e4633706088354cf14b653"
    },
    {
      "bytes": 11818,
      "git_blob_sha1": "fcd50c15f4e882913370a41d8fcd841c5e2b4a66",
      "path": "Makefile",
      "sha256": "3ebd7151ba55dd2b5a39380403054481a9c04292d573e62fa7db82633fd6530a"
    },
    {
      "bytes": 4957,
      "git_blob_sha1": "b793a5ba0cb966520f681399d8f2a61ceb5fe883",
      "path": "deploy/universal-terminal/Dockerfile",
      "sha256": "97a03ff7b88218050b4f4295bcf6369ba03a402388364666cca32a0673d4b4f7"
    },
    {
      "bytes": 5001,
      "git_blob_sha1": "c2099bf301108389192c72497a3637acae392d7a",
      "path": "deploy/universal-terminal/cloud-entrypoint.sh",
      "sha256": "97b2507ed2ac3955ed80682403ed482334201511849285c881e5c8739bfc6b9c"
    },
    {
      "bytes": 5661,
      "git_blob_sha1": "f807391adabc9046fa37ba847f572acbd1709e46",
      "path": "deploy/universal-terminal/compose.yaml",
      "sha256": "4c2ca8207b24fcf0a88d55ddb46eac31117e636aaafb5a74cfbd91b078cfbaa1"
    },
    {
      "bytes": 314,
      "git_blob_sha1": "1411d7f9ab41a18fd28b1209683ac9a1e232066f",
      "path": "deploy/universal-terminal/db.qik-vrt.mesh",
      "sha256": "2b76539a1a5828fead91905b46dd62113b0a3e97985c5339d0e2077f95a28f95"
    },
    {
      "bytes": 7470,
      "git_blob_sha1": "3db27a8487e67b2beac596e1f576afe1cf95b21e",
      "path": "deploy/universal-terminal/entrypoint.sh",
      "sha256": "c0bae910a3220b79d283943294b0ace65b8787c8f95b275a75ffc73d65fb139b"
    },
    {
      "bytes": 1604,
      "git_blob_sha1": "9b980bdf846cdfe2f96e50f9ec9c349c3198ef50",
      "path": "deploy/universal-terminal/mesh-index.html",
      "sha256": "f1d52c2edee53ebcb546803eb549aa0027121dc7b43fd3f861827612f0f162c6"
    },
    {
      "bytes": 1900,
      "git_blob_sha1": "1a778891a805825bb82d7e475c97d9b02c3e6a6e",
      "path": "deploy/universal-terminal/mirror-bootstrap.sh",
      "sha256": "f6889cda54e74b6fa3bc2faf9f73fbe9aca48c394d461769d1f91e5a24202edc"
    },
    {
      "bytes": 357,
      "git_blob_sha1": "17b250ab096a70c188913d9368e0493c6ffd0b87",
      "path": "deploy/universal-terminal/named.conf",
      "sha256": "5df23f5a9d0217cbd38a1d86dd6b661f9cbeea72489a577ef1349948165c6d90"
    },
    {
      "bytes": 4026,
      "git_blob_sha1": "4e13a623d72896cae563d57b710766af3744a27a",
      "path": "deploy/universal-terminal/nginx.conf",
      "sha256": "583491a285112a900cedb5b1e4dc001ddc1df1efd90d13f1f234127f0758489b"
    },
    {
      "bytes": 3992,
      "git_blob_sha1": "06a952c1e61ce94d1b569d6171b1853e37dc30dc",
      "path": "deploy/universal-terminal/qikvrt_ip_bootstrap.c",
      "sha256": "73c9c63e410e4e3f0802627473eac9ae19d6c89ca85a07e75a0bd63136482a0e"
    },
```

```
    {
      "bytes": 4396,
      "git_blob_sha1": "454606e89d09f845ed7470b9aed764dfb1ba884e",
      "path": "deploy/universal-terminal/qikvrt_smtpd.py",
      "sha256": "9b68ef0b402ac269418571eebfc9aee5fd33eece849dbe259bd69b73a1e3ce29"
    },
    {
      "bytes": 1548,
      "git_blob_sha1": "3bec1909edc484bb15116a0a23c27f5b4e20a3e9",
      "path": "deploy/universal-terminal/runtime-health.sh",
      "sha256": "731d9e2adddf83999623424f9a306a6dd1ff2b142ec6a179cfd2c9ae12f13bf3"
    },
    {
      "bytes": 5285,
      "git_blob_sha1": "54c672e48c52b3d5b94a47d73a76137e67fb4662",
      "path": "deploy/universal-terminal/service-entrypoint.sh",
      "sha256": "57c34784cf4a82a93a7639cf7917f37bb75e1a4b404e41243f890c5a85f06054"
    },
    {
      "bytes": 151,
      "git_blob_sha1": "bd555a057536c9f913fbdcc2a1a9ad3b42da28ab",
      "path": "deploy/universal-terminal/snmpd.conf",
      "sha256": "3d7caeb6b2f8e0a75f6c66a01b6520ba2403c5ac0b9f64837a11ee2e61e17528"
    },
    {
      "bytes": 468,
      "git_blob_sha1": "e06dd174e059b3e81c0706278cfb5c269dbb380c",
      "path": "deploy/universal-terminal/sshd_config",
      "sha256": "2effb563d95bdb35c9cb22ed144b718dd5ba6df509c70e3db8c4cb44f9b839b3"
    },
    {
      "bytes": 12482,
      "git_blob_sha1": "919b735bfee5dfa68b94725453a397c7439a882e",
      "path": "docs/QIKVRT_PRODUCTIZATION_ROADMAP.md",
      "sha256": "cd8a7ce888ab768fa0e659a59c1f38cc0afce8a3d5618bf519e10af00a3eb430"
    },
    {
      "bytes": 11922,
      "git_blob_sha1": "b1e06afc86a04d9989150b3c1690389ac5723631",
      "path": "docs/monitor/README.md",
      "sha256": "5afc63b464462d7f6eb094e23d59adf328df22de937a03ddfcf8bd884bbf01f0"
    },
    {
      "bytes": 9774,
      "git_blob_sha1": "afb34dc921ae63fc3c1bd1c24d775b4ad7312bf9",
      "path": "docs/monitor/self-host.mjs",
      "sha256": "5620d5840849f6e96f97a04bf5b631be4a534c2b4ccaead0767442f254cd845f"
    },
    {
      "bytes": 10647,
      "git_blob_sha1": "2c32118f30c37d8393ae174b06aedb3ece1fe4c0",
      "path": "docs/terminal/temdd/index.html",
      "sha256": "847109600bdc487b3ba18d92393aa03370386780d8fdca2bd4529a7269d55819"
    },
    {
      "bytes": 241,
      "git_blob_sha1": "fa01b335fe878c663475c441e4a5fd8d8e938c20",
      "path": "evidence/node_health/37235824372-1.json",
      "sha256": "0be70fd732958190edaab039617b00a26e244ff0d34f88931d240b9e204f69d6"
    },
    {
      "bytes": 241,
      "git_blob_sha1": "fa01b335fe878c663475c441e4a5fd8d8e938c20",
      "path": "evidence/node_health/LATEST.json",
      "sha256": "0be70fd732958190edaab039617b00a26e244ff0d34f88931d240b9e204f69d6"
    },
    {
      "bytes": 262,
      "git_blob_sha1": "c872f704c36085ddffe8b2539675463bb6fa3b26",
      "path": "evidence/node_registration_renewal/37235824372-1.json",
      "sha256": "893f5c4388631bd4ea2ac2c3f247b2060f2c2701fc123a9c56b5486909655c40"
    },
    {
      "bytes": 262,
      "git_blob_sha1": "c872f704c36085ddffe8b2539675463bb6fa3b26",
      "path": "evidence/node_registration_renewal/LATEST.json",
      "sha256": "893f5c4388631bd4ea2ac2c3f247b2060f2c2701fc123a9c56b5486909655c40"
```

```
    },
    {
      "bytes": 120508,
      "git_blob_sha1": "dfe1c78ead2edff550895e3fb32200e8892bce5d",
      "path": "evidence/self_host/SOURCE_RECOVERY_20261005.json",
      "sha256": "ac8e345f2b62a562d65cc670274959d41f1df4dbbde4229c2c4af61de815dd7e"
    },
    {
      "bytes": 2366,
      "git_blob_sha1": "87d19ab6bee748a222c64df164f59c2df928c4df",
      "path": "evidence/self_host/SOURCE_RECOVERY_LOCAL_20261005.json",
      "sha256": "c829c61c6f6ec6f9dfe2261b6cf1ca24ac46afc9a0ea0e31085109ab5981e78c"
    },
    {
      "bytes": 563,
      "git_blob_sha1": "93da558c120c34ca4ba6925c932e77017f519566",
      "path": "qikvrt/runtime/onboarding/NODE_HEALTH.json",
      "sha256": "01bbc8663e865f4fbdb2dfc97b73e82ec1b14a5e5e8ffa559ebf1bff1a2b01f3"
    },
    {
      "bytes": 542,
      "git_blob_sha1": "c34906996cbeda09c41b9c58da7dff17c21d7c30",
      "path": "qikvrt/runtime/onboarding/NODE_REGISTRATION_RENEWAL.json",
      "sha256": "88d1cb6956b40414fcdcc546751fbf88d93ae7f43987d8e3be032f9b8843797c"
    },
    {
      "bytes": 422,
      "git_blob_sha1": "eb661b8dfcaf696f129070f8a0429d1a4ce57446",
      "path": "runtime/self-host/CONFIG.example.json",
      "sha256": "c9ce35bb8897d8ccd3e5b726422b2e09bb534ef6519ddf90f0fe6dbce3ffb424"
    },
    {
      "bytes": 1208,
      "git_blob_sha1": "08a0a094ecdbec08bf60ea4ec13dfec0f1dde34e",
      "path": "runtime/self-host/Dockerfile",
      "sha256": "d393d3eb7eaae2c1dcd4404ef6cf86600ea794a068ff32cf3a793d3abb0fec3b"
    },
    {
      "bytes": 3910,
      "git_blob_sha1": "bff9c80bc29d276b9ce8e050093646c6fc6aaedf",
      "path": "runtime/self-host/PACKAGE.json",
      "sha256": "44b675f6fcf1abb653dafd79197da9f77a2077a7e3fe7fd658503208a878fb91"
    },
    {
      "bytes": 8829,
      "git_blob_sha1": "0f7fff71870dd68707e75325ae7acbd56901842a",
      "path": "runtime/self-host/README.md",
      "sha256": "862acc5d841f76be8dc587a1dfa0f1610ef274717c5e0696f8d324a7a05aed35"
    },
    {
      "bytes": 5569,
      "git_blob_sha1": "43b677d3044a71cccc2a0b9a19a1fd2741191a69",
      "path": "runtime/self-host/RECOVERED_TERMINAL.md",
      "sha256": "17cce25cdc39af59608f1bfe872c2b0a9f766a56ffd146b31ea84baf274456d4"
    },
    {
      "bytes": 6577,
      "git_blob_sha1": "1eefd2549e6866fe0e2187174c55060b2f115ca3",
      "path": "runtime/self-host/original-firefox/README.md",
      "sha256": "f71fbf101d2d3a365fc094cd5697493826459688b377a67ca575f01499d2d9c6"
    },
    {
      "bytes": 3955,
      "git_blob_sha1": "6e3d44df2928052e4239c6cd53163455d23453a4",
      "path": "runtime/self-host/original-firefox/_locales/ar/messages.json",
      "sha256": "42ec0581a07b76d2c25d227c39151af2caaba7b8883551840db9497207a1672c"
    },
    {
      "bytes": 3670,
      "git_blob_sha1": "11fd70240efd73878f652ab3c212bfe8dcf910f5",
      "path": "runtime/self-host/original-firefox/_locales/de/messages.json",
      "sha256": "56986ec3e64d8e2a6f070bc438bd4c4e0cba1c3b53ccb8e6dc7f453a01abb4cb"
    },
    {
      "bytes": 3411,
      "git_blob_sha1": "50b8040e5ec48b17ddfa702c4aab61da23086b32",
      "path": "runtime/self-host/original-firefox/_locales/en/messages.json",
```

```
        "sha256": "ad4403073b812bce47d215ea936b010df68cd8cedaab9f7cc74fddf44c659a6d"
      },
      {
        "bytes": 3639,
        "git_blob_sha1": "d3e32c91067f40a7a66e1e1bbf22293d54f6fb7f",
        "path": "runtime/self-host/original-firefox/_locales/es/messages.json",
        "sha256": "1e3037677d0abf91dfc9fb33c81fe81313e745f891e2143e46351e5886246090"
      },
      {
        "bytes": 3765,
        "git_blob_sha1": "de4d3108fea6ae9a9f4a843e9795dd594a9be0e3",
        "path": "runtime/self-host/original-firefox/_locales/fr/messages.json",
        "sha256": "4d2265145bcc645b899b9d43b73e9fd002133ce99a80e78def9792c990fcf225"
      },
      {
        "bytes": 5834,
        "git_blob_sha1": "34baa5c2a8312e3a5f47d08f1190ec9246c970d9",
        "path": "runtime/self-host/original-firefox/_locales/hi/messages.json",
        "sha256": "231458fdd2cd0a8d43da6382e304a12e548da389767de53f9eb1e4c2b7f13eb0"
      },
      {
        "bytes": 3542,
        "git_blob_sha1": "ad54ebfde1856ddd1bddd6507b1edddd55e633b8",
        "path": "runtime/self-host/original-firefox/_locales/id/messages.json",
        "sha256": "09fb19851b9e9370524244db6bbc84931a354d942325ac49a6660a4e15f12d95"
      },
      {
        "bytes": 3716,
        "git_blob_sha1": "18ddf3373a94b8562778cecea654d113a6d3ce28",
        "path": "runtime/self-host/original-firefox/_locales/it/messages.json",
        "sha256": "499086963505889846a271b5f0415d9b52a5884028634086019b88dc6cc10d34"
      },
      {
        "bytes": 4021,
        "git_blob_sha1": "11bce6bf298aaa2e4a5ed95392ea3b5afb721ef0",
        "path": "runtime/self-host/original-firefox/_locales/ja/messages.json",
        "sha256": "9e38ac2cdcf822cd21b0d7fdc9c3b915b94aa6770272780450d4a871bdf7ce3b"
      },
      {
        "bytes": 3787,
        "git_blob_sha1": "f6b1d88576b2ccdc8d44bb21f27fd5d35b7acd1f",
        "path": "runtime/self-host/original-firefox/_locales/ko/messages.json",
        "sha256": "5dbfbf3ec9a2a2f7b9ebb217ad26221bbef89ebb203bf7c95be5c88646ca37c3"
      },
      {
        "bytes": 3629,
        "git_blob_sha1": "2d5d1c499251a3f75346b8b2b649e3d88df929ab",
        "path": "runtime/self-host/original-firefox/_locales/pt_BR/messages.json",
        "sha256": "f6c849089370d4a13b73cef843fcb39a43c7fa7a402dbfbc16fbd7a8a572b3e5"
      },
      {
        "bytes": 4706,
        "git_blob_sha1": "7aee4bec673294353d2276d77723767611a5a587",
        "path": "runtime/self-host/original-firefox/_locales/ru/messages.json",
        "sha256": "f006010304185d32003f0bed98ff42a76a3ad4d5f97b38371b4cb524ab10aea2"
      },
      {
        "bytes": 3513,
        "git_blob_sha1": "d72d5265e778cb265bf266c06e3816085ab86c59",
        "path": "runtime/self-host/original-firefox/_locales/tr/messages.json",
        "sha256": "6f442192d324f983ede84f9735ae681e51e6235a6ab7b4dd61cdd9a53daf8775"
      },
      {
        "bytes": 3350,
        "git_blob_sha1": "602cb0c77a192d8de51f8cba8161209516295eaa",
        "path": "runtime/self-host/original-firefox/_locales/zh_CN/messages.json",
        "sha256": "e71fa8c582e6f67de441d2128ab4e63d8ad9a2e12c896837010bc618d9c0d8d4"
      },
      {
        "bytes": 15796,
        "git_blob_sha1": "7900e7c9f89a870772471dcd5be22aef171cbc75",
        "path": "runtime/self-host/original-firefox/authenticated_delivery.js",
        "sha256": "8b62b0dd7beadc24b5d037102d10b3590714e446298e833a5ae41eeeea74ff26"
      },
      {
        "bytes": 15060,
        "git_blob_sha1": "96831eafc0182599400bf2af3497ded47376d16d",
```

```
        "path": "runtime/self-host/original-firefox/background.js",
        "sha256": "d76e058245d216d28c78065031006377b80e6396f102356c3c359e3635961cc3"
      },
      {
        "bytes": 9424,
        "git_blob_sha1": "dc3bc7d525a762f318174b1e8854b5c6df3050e8",
        "path": "runtime/self-host/original-firefox/content.js",
        "sha256": "f41f930c6e0600956d1436e7f25c100c8d55200ca6b20e3581bf3aab4f4bcbae"
      },
      {
        "bytes": 1578,
        "git_blob_sha1": "5af919973660056c11d1cb94dcfa2092956d4192",
        "path": "runtime/self-host/original-firefox/manifest.json",
        "sha256": "f17889e6922a1c0eb84f949c49c1ef20ba73e80f10ca8e36c0c23f585d0bed65"
      },
      {
        "bytes": 1634,
        "git_blob_sha1": "71a869d548cdfec3f546b905246d67811e3ee601",
        "path": "runtime/self-host/original-firefox/options.html",
        "sha256": "5c972e6528d43608f5ecf754054ddaa22992e53d9a49d93c3d258b0b7e88c5b3"
      },
      {
        "bytes": 1154,
        "git_blob_sha1": "92bdd1cb494267b69d4fa43ec97a8025d9da26fa",
        "path": "runtime/self-host/original-firefox/options.js",
        "sha256": "a20467bc0d329369b562a02909412bbbc6596cd9e5e3f27530fcfa4373669f4d"
      },
      {
        "bytes": 5377,
        "git_blob_sha1": "03da429fe609603783f4829bb952ffd3c21dce6a",
        "path": "runtime/self-host/original-firefox/review_effect.js",
        "sha256": "a905219e62db17e5a9f355bbcb10e64efdeb9c02472572967cb53dafa0e114be"
      },
      {
        "bytes": 3536,
        "git_blob_sha1": "abf1f6ffae3a4760652214e73b46b4ed107ad521",
        "path": "runtime/self-host/original-firefox/terminal.css",
        "sha256": "769dffd97cb95d995a4b72b073cbef669391421b0c4acabf07f9ef79cb1f99ed"
      },
      {
        "bytes": 13389,
        "git_blob_sha1": "c0dbf5d880d146c23645d8de971c34ae64da6906",
        "path": "runtime/self-host/original-workflows/qikvrt_temdd.yml",
        "sha256": "5ecd9a8470f02da7e32da16812fe61f2fe42e214b00bec2d08ee139daa5e99c8"
      },
      {
        "bytes": 5414,
        "git_blob_sha1": "302786ada70bf9f11a9a1d1acaa12a94f9e95cb5",
        "path": "runtime/toolchains/CACHE_COVERAGE.json",
        "sha256": "f1968265f61fcb42e38ea43777fdb782ec741d08f744f0e520c2a315db2bd0b7"
      },
      {
        "bytes": 19032,
        "git_blob_sha1": "f094577ae3012a157e930600aeada09e65c9443f",
        "path": "runtime/toolchains/CACHE_REGISTRY.json",
        "sha256": "37c36e12e675290fee02c2e5bcc992ac22f4254eb9bba17b95d6167144d507ee"
      },
      {
        "bytes": 9805,
        "git_blob_sha1": "514f7011120ff6c2c82d27da0e886497375ac96e",
        "path": "runtime/toolchains/PROVENANCE.md",
        "sha256": "a410c5c4a29a5f16a66c9dd6633c9d18fa9bd9ff3f63f78e70cdd4aa7699f3cc"
      },
      {
        "bytes": 4169,
        "git_blob_sha1": "b350a1977a9779aad3b811da06dd730216e9fcd5",
        "path": "runtime/toolchains/TOOLCHAIN.lock.tsv",
        "sha256": "d852c87a440cc1185ebe54829c8a36cae275dd91c880eb9871e40d529d08acdb"
      },
      {
        "bytes": 30823,
        "git_blob_sha1": "30ff5f58318a54f06efb86f437dea1f7239c9c00",
        "path": "src/qikvrt_effect_ack_http_terminal.py",
        "sha256": "039b7c34fe4ae8766ae0b141734ef91131377e5e58fedb7746320242543da2d8"
      },
      {
        "bytes": 21653,
```

```
        "git_blob_sha1": "6322bd45edb9de7d2aa07dee6fbc12307148475c",
        "path": "src/qikvrt_temdd_event_ledger.py",
        "sha256": "94908e6a50ce107a79965055f3ecabae10e7589d96f7a26496d2a4956b068b13"
      },
      {
        "bytes": 5812,
        "git_blob_sha1": "4ddea0669f9a420b9664301cf025ef6e579abdc6",
        "path": "state/work_units/PROVIDER_INDEPENDENT_MESH_ROADMAP_20261004.json",
        "sha256": "e4cdc28dbc9886fa5cdb9100b5d340a2b0c6f5f014bc6a467d155d312c90e6ea"
      },
      {
        "bytes": 27261,
        "git_blob_sha1": "f92602d4a8698c7d3b0547c585b0144e39fea8ab",
        "path": "state/work_units/SELF_HOSTED_NODE_PACKAGE_S1_20261004.json",
        "sha256": "60a326d5ebe23f5fce93320e3de39d325b0cecadbf3050adb111f5a41d5c0e53"
      },
      {
        "bytes": 8652,
        "git_blob_sha1": "1891b1e2edcdbb2b0c5ebeb73c787e3d7ab15564",
        "path": "tests/test_qikvrt_temdd_public_proxy.py",
        "sha256": "505d1608109f75caac698e2af97278a58431b7a2235f8f2ce03ed98cb3a962ee"
      },
      {
        "bytes": 20746,
        "git_blob_sha1": "68b5aac5419c0c99fd19c73595f6c8edf05a933c",
        "path": "tests/test_self_host.py",
        "sha256": "372d2265978332beb02da44bd277d0d78924f418fe0a369a94b99844a1f15e22"
      },
      {
        "bytes": 10825,
        "git_blob_sha1": "286a15030c9a7bd8c1c04b9493264c0a9edcb5ab",
        "path": "tests/test_self_host_firefox.py",
        "sha256": "f4365f8c37b776be987e0f6cf08badec013c39e18751d42af24d659769a8fc29"
      },
      {
        "bytes": 10400,
        "git_blob_sha1": "0d444ae11b84baf790e00be062268346002b9ded",
        "path": "tests/test_self_host_temdd.py",
        "sha256": "eaabaa8149736bf638741168e26e7c3fbb8feec48c7391edeac04556d90c160b"
      },
      {
        "bytes": 21663,
        "git_blob_sha1": "187230e0bab7512811c0e9d71c4f6498be5f06ef",
        "path": "tests/test_temdd_event_ledger.py",
        "sha256": "662d41da4b1e15237ee73ce73347d090c32e6fa7dcaaea208e95938cf20b3966"
      },
      {

        "bytes": 3806,
        "git_blob_sha1": "58f5af0d026dfa9319f99b3beb8dcd6e30aec18d",
        "path": "tests/test_temdd_runtime_subject_binding.py",
        "sha256": "e6c1c98665c50fdf2c5de1ac29f17f5ccf78aefdcdbd9abb4e700aeb5ac6bc4a"
      },
      {
        "bytes": 14890,
        "git_blob_sha1": "171d755a36e273154625fbed6512fc79c998001b",
        "path": "tools/ai_runtime_bootloader.py",
        "sha256": "e15aaf5f416ff53098d677fb8286feb75fe1d20d58d52332a9d0d1550d217317"
      },
      {
        "bytes": 24266,
        "git_blob_sha1": "b6d8600b7894920971890ce06d7d2ec9da510668",
        "path": "tools/bootstrap-runtime.ps1",
        "sha256": "a304946b15f0a421de4247a36698091298e5fa68965e87f9c5d35031c13f7328"
      },
      {
        "bytes": 18480,
        "git_blob_sha1": "7f84753d3e9d258a68e98f25010d42710594a94d",
        "path": "tools/bootstrap-runtime.sh",
        "sha256": "4943adc0ff18dd83b507d64b29204452c46f7e6d3c483c31e88926a978a00d52"
      },
      {
        "bytes": 10198,
        "git_blob_sha1": "94f553fe54038389c19bc8b95a0a69c86961a9bf",
        "path": "tools/qikvrt_mesh_monitor_readback.mjs",
        "sha256": "e2c674f221255e9ed2cf495bb897181a8352420acb656bd7fb7e0e8193ced1dd"
      },
      {
```

```
      "bytes": 28982,
      "git_blob_sha1": "1788bab299d3fcaf07af99c57a2d1f2471636d99",
      "path": "tools/qikvrt_self_host.py",
      "sha256": "75d334b48eefb837ad02b285765764b5a0912d6b0bc2596284a7ae2ba02c1345"
    }
  ],
  "local_validation": {
    "bytes": 2366,
    "git_blob_sha1": "87d19ab6bee748a222c64df164f59c2df928c4df",
    "path": "evidence/self_host/SOURCE_RECOVERY_LOCAL_20261005.json",
    "sha256": "c829c61c6f6ec6f9dfe2261b6cf1ca24ac46afc9a0ea0e31085109ab5981e78c"
  },
  "main_continuation_parent": "0a44160d16c3937dc5888652791f4bd1a7136af7",
  "milestone": "S1",
  "milestone_state": "OPEN",
  "pr": 457,
  "reuse_evidence": {
    "bytes": 120508,
    "git_blob_sha1": "dfe1c78ead2edff550895e3fb32200e8892bce5d",
    "path": "evidence/self_host/SOURCE_RECOVERY_20261005.json",
    "sha256": "ac8e345f2b62a562d65cc670274959d41f1df4dbbde4229c2c4af61de815dd7e"
  },
  "runtime_gate_repairs": [
    {
      "effect_ack_done": false,
      "executed_scope": "55 native/source/original-subject tests executed; single original Makefile-admission failure; GUI not started",
      "failed_head": "ccf46e64ebd54fcdb1e064cc4c04d4fe0cbb8b47",
      "failed_tree": "ebc16c3b12864b917a6de715b7d7347969513546",
      "failure": "Original test_repository_entrypoint_and_test_admission requires real Makefile test: temdd-event-ledger-test dependency",
      "focused_local_regression": {
        "command": "python3 -B -m unittest -v tests.test_self_host tests.test_self_host_temdd.SourceRecoveryTests tests.test_temdd_runtime_subject_binding tests.test_temdd_event_ledger.ProducerTests.test_repository_entrypoint_and_test_admission",
        "exit_code": 0,
        "scope": "LOCAL_FIXTURE_AND_ORIGINAL_ADMISSION; NATIVE_GUI_READBACK_STILL_REQUIRED_ON_SUCCESSOR",
        "tests": 25
      },
      "fresh_runtime_gates": "REQUIRED_ON_SUCCESSOR; NO_INHERITED_PASS",
      "job_id": 111591981593,
      "repair": "Restore the original named test target as an unconditional dependency of self-host-test and an explicit S1-profile dependency of make test; original tests remain byte-identical; preserve generic portable platform contract; preserve original broad TEMDD workflow unchanged as archived source and add a real focused S1 native workflow instead of invoking the unavailable historical compiler/backend closure",
      "run_id": 37255635654
    },
    {
      "cause_inference": "The launcher inherits the deliberately dead HTTP_PROXY fixture; its default urllib opener routes the loopback noVNC startup read through that proxy although child browser env is clean",
      "effect_ack_done": false,
      "failed_head": "fe06b23d43b08a9da6c6f22bc2d8c20f8fc9d799",
      "failed_tree": "f6291fdb27492cff65b8d1dca0cfb80b0275dd56",
      "failure": "BROWSER_STARTUP_READBACK_TIMEOUT; seven actual Firefox positive-start cases fail; native cases pass; no positive browser runtime receipt",
      "fresh_runtime_gates": "REQUIRED_ON_SUCCESSOR; NO_INHERITED_PASS",
      "job_id": 111593667702,
      "local_causal_regression": {
        "actual_control": "Default opener fails against the dead inherited proxy; explicit no-proxy opener returns byte-exact actual loopback HTTP and rejects different local asset bytes",
        "command": "python3 -B -m unittest -v tests.test_self_host_temdd.SourceRecoveryTests",
        "exit_code": 0,
        "tests": 3
      },
      "repair": "Explicit no-proxy loopback noVNC opener; name each readiness edge; retain bounded synthetic CI component failure logs; no readiness timeout enlargement or gate removal",
      "run_id": 37256188418
    },
    {
      "effect_ack_done": false,
      "exact_artifact": {
        "downloaded_and_hash_verified": true,
        "id": 11323107645,
        "sha256": "ce06993d25efc019d582c0f8f46050e1b0643bdcb284efa540fece8091f54e74"
      },
```

```
      "failed_head": "799b46c37a31e88204dd3ab76e873a6cb84fa6bd",
      "failed_tree": "97631ef4ad91fa149ca7862b16de64e4712d7b4c",
      "failure": "Cold Firefox/native/noVNC readback passes; browser crash restart and later cases fail BROWSER_CHILD_EXITED",
      "fresh_runtime_gates": "REQUIRED_ON_SUCCESSOR; PREDECESSOR_COLD_START_NOT_TRANSFERRED",
      "job_id": 111595645773,
      "observed_additional_socket_boundary": "LibVNCServer logs an undeclared default IPv6 listener on 5900 even with configured IPv4 port; disable both x11vnc IPv6 and library IPv6 default; assert actual kernel listeners equal the four declared IPv4 loopback ports",
      "observed_cause": "xvfb.log reports Server is already active for display 101 and /tmp/.X101-lock after process-group SIGKILL; test Python was namespace PID1 without an orphan reaper",
      "repair": "Use Docker platform --init, bind the actual reaper executable and engine version, assert old Xvfb PID removal and fresh acknowledged profile/SQLite readback",
      "run_id": 37256843587
    }
  ],
  "schema": "qikvrt-attributable-work-unit/v1",
  "source_classification": {
    "authority_origin_commit_directly_retrieved": false,
    "latest_railway_production_equality": "UNVERIFIED",
    "new_code": "S1_PACKAGE_BINDING_AND_PORTABLE_BROWSER_START_ADAPTER; NOT_RESTORED_ORIGINAL_SOURCE",
    "originals": "44 BYTE_EXACT_HISTORICAL_FILES; SEE SOURCE_RECOVERY",
    "reconstructed_daemon": false
  },
  "source_parent_head": "dfddd74b031867c5113827780ad937379c85914f",
  "source_parent_tree": "480a1275729bc96e3b71566b24fbaca41e948787",
  "source_repository": "ingolf-lohmann/qik-vrt",
  "state": "ORIGINAL_SOURCE_RECOVERED_AND_S1_ADAPTER_IMPLEMENTED; FRESH_RUNTIME_GATES_REQUIRED",
  "work_unit_id": "S1_PRODUCTIVE_TERMINAL_SOURCE_RECOVERY_20261005"
}
```

A3 - S04 / .github/copilot-instructions.md

Archivkennung F091; Ursprungspfad .github/copilot-instructions.md

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 31922701cb2ce18d849ca4a14bfb23714b9de6e7; SHA-256 ea5f6b0c20fb403a3da94f4687e2a3448ca0dd43060495e5cfda5afea63a01fb; 509 Byte

```
0001  # QIK-VRT Copilot instructions
0002
0003  Before generating or modifying project content, read the root file `AI`, parse
0004  `AI_CONTEXT.json`, and follow its `required_read_order`.
0005
0006  Repository evidence is authoritative over chat or model memory. Preserve
0007  provenance, effect boundaries, tests, hashes, and the explicit licensing split:
0008  freely available architecture does not automatically license the concrete
0009  implementation as open source. Treat symmetric canonicality as verified only
0010  for byte-equivalent declared scope.
```

A4 - S05 / spec/temdd/TEMDD_LANGUAGE_SPEC_V0_1.md

Archivkennung F092; Ursprungspfad spec/temdd/TEMDD_LANGUAGE_SPEC_V0_1.md

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 633d82a4cdef8e4346393fbc1bedf8d3b205e0b3; SHA-256 8bbc29511a6d09ac58754234f14f09752fc53482ce10e8a3051d4581e1afdf59; 3330 Byte

```
0001  # TEMDD Language Specification v0.1
0002
0003  Status: executable bootstrap candidate. Authority: Goldkelch/qik-vrt.
0004
0005  TEMDD (Tested Event Model Driven Development) is the executable metaprogramming language of QIK-VRT. Its
      lifecycle is `REQUEST -> EXECUTE -> FOLLOW -> LEARN -> SUCCESSOR -> ... -> DONE`.
0006
0007  ## Normative invariants
0008  T01 Evidence is exact-subject typed and cannot transfer to another head/tree/scope. T02 Mutation creates
      a successor and invalidates dependent evidence. T03 TRANSPORT_ACK != EFFECT_ACK. T04 RESULT != EFFECT.
      T05 HOLD/CONTINUE are nonterminal and never NOOP. T06 Open productive work excludes DONE. T07 External
      effects require exact-scope authority. T08 Committed effects require fresh readback. T09
      Missing/stale/conflicting/malformed/indeterminate critical evidence fails closed. T10 Unknown critical
      effect state blocks. T11 Learning constructs a successor and cannot self-certify. T12 DONE requires the
      complete declared DoD from fresh bound evidence.
0009
0010  ## Semantic objects
0011  `authority`, `subject`, `request`, `event`, `test`, `evidence`, `model`, `effect`, `hold`, `learn`,
      `successor`, `done`. Models are not evidence. Effects have prepare/authorize/commit/readback boundaries.
0012
0013  ## Execution
0014  TEMDD is event-driven. Scheduling and polling are outer-layer transports, not kernel semantics. A runtime
       consumes one event, performs a finite transition, emits evidence/effect/successor intents and becomes
      quiescent. WAIT/HOLD, queued work, open PRs, unmerged productive branches, failed gates, missing releases
       and missing effect readback cannot become NOOP or DONE.
0015
0016  ## Compilation
0017  `TEMDD source -> parser -> typed AST/IR -> conformance -> backend`. Initial backends are Python
      reference, Smalltalk live objects, C90 deterministic runtime, M68000 fixed relations and Lean
      obligations. Translators preserve subject identity, provenance, authority, status, effect state and
      readback obligations. Semantic loss or backend disagreement is BLOCK.
0018
0019  ## Self-improvement
0020  `S --learn--> S'`. S' is a new exact subject. Evidence for S is not validation for S'. Fresh conformance
      and deployment evidence is mandatory before adoption.
0021
0022  ## Continuous deployment
0023  `parse -> elaborate -> conformance -> differential backends -> formal obligations -> reproducible build
      -> candidate deploy/readback -> authorized promotion -> production readback`. No stage implies a later
      stage.
0024
0025  ## Canonical QIK-VRT DoD
0026  `ZERO_BUGS && ALL_PULL_REQUESTS_REGARDED && ALL_BRANCHES_REGARDED && ALL_PRODUCTIVE_BRANCHES_MERGED &&
      FRESH_EXACT_MAIN_VALIDATION_PASS && FRESH_EFFECT_READBACK`.
0027
0028  ## Bootstrap boundary
0029  v0.1 distinguishes BOOTSTRAP_CONFORMANT from STABLE_LANGUAGE. Stable status requires executable Smalltalk
       and M68000 gates, compiled Lean obligations, reproducible release, Main adoption and fresh production
      effect readback. Version labels and successful transport/build jobs cannot manufacture these claims.
      Intermediate states are observable but are not a return condition; the return condition is the full
      declared DoD.
0030
0031  ## Conformance keywords
0032  MUST/MUST NOT define normative requirements; SHOULD is a strong recommendation overridable only with
      recorded evidence; MAY is optional. Any implementation claiming TEMDD v0.1 conformance MUST satisfy the
      executable positive/negative corpus and T01-T12 checks for the features it claims.
```

A5 - S06 / spec/temdd/TEMDD_NORMATIVE_CORE_V1.md

Archivkennung F093; Ursprungspfad spec/temdd/TEMDD_NORMATIVE_CORE_V1.md

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 1af763493808fc5f192e5cd7536b1b51e437804d; SHA-256 4b4c32b98c7482a570b4b8cb85b86af053cc3ecf3567f8f03442f3487e7d3d8b; 4247 Byte

```
0001  # TEMDD Normative Semantic Core v1
0002
0003  Status: normative successor contract.
0004  Authority repository: Goldkelch/qik-vrt.
0005  Predecessor evidence transfer: DENY.
0006
0007  TEMDD v1 is an event- and evidence-oriented control language. It does not
0008  retroactively redefine TEMDD v0.1 receipts. A subject mutation creates a new
0009  subject and invalidates subject-bound conformance evidence until fresh
0010  validation is produced.
0011
0012  ## Seven-layer product contract
0013
0014  1. Language — grammar plus operational Request -> Execute -> Follow -> Learn semantics.
0015  2. IR — canonical deterministic machine-readable representation.
0016  3. IDE — authoring, diagnostics, IR inspection, event graph, ledger and conformance projection.
0017  4. Ledger — append-only event history with exact subject, cause and evidence bindings.
0018  5. Formalization — machine-checked invariants for subject, causality, evidence and effect
      acknowledgement.
0019  6. Tests — parser, semantic, IR, ledger, provenance, property and end-to-end negative/positive vectors.
0020  7. Conformance — an implementation-independent runner and exact-subject machine-readable report.
0021
0022  No layer is evidence for a later layer merely because it executed successfully.
0023
0024  ## Result domain
0025
0026  ```text
0027  RESULT := PASS | FAIL | CONTINUE | HOLD_UNVERIFIED
0028  TRANSPORT_ACK != EFFECT_ACK
0029  RESULT        != EFFECT
0030  SEQUENCE      != CAUSALITY
0031  PREDECESSOR_EVIDENCE_TRANSFER = false
0032  ```
0033
0034  `CONTINUE` and `HOLD_UNVERIFIED` are language-level outcomes. They are not
0035  CI presentation conventions and they never imply `DONE`.
0036
0037  ## T13 — CAUSAL_BINDING
0038
0039  Event order is not causal order. Causality exists only through explicit
0040  `cause_event_ids` bound to immutable event identities.
0041
0042  ```text
0043  sequence(e1,e2) MUST NOT imply cause(e1,e2)
0044  cause(e1,e2) iff e1.event_id in e2.cause_event_ids
0045  ```
0046
0047  A timestamp, ledger sequence number, PR number or file order MUST NOT manufacture
0048  a causal edge.
0049
0050  ## T14 — EVIDENCE_NON_TRANSFER
0051
0052  Evidence is valid only for the exact subject it observed.
0053
0054  ```text
0055  mutation(S0,S1)
0056    => S0 != S1
0057    => Evidence(S0) MUST NOT validate S1
0058  ```
0059
0060  Subject identity includes repository, logical subject identifier, HEAD and TREE.
0061  A successor MAY preserve predecessor evidence as immutable history, but MUST NOT

0062  relabel it as fresh evidence for the successor.
0063
0064  ## T15 — EFFECT_CONSTRUCTION
0065
```

Transport success, computation result and commit acknowledgement are not effect acknowledgement.

```text
EffectAck(a)
  iff Authority(a)
  and Commit(a)
  and FreshReadback(a)
  and ExactSubject(a)
  and Match(Expected(a), Observed(a))
```

Therefore `PUT_SUCCESS != EFFECT_ACK`, `COMMIT != EFFECT_ACK`, and `RESULT != EFFECT`. Missing, stale, ambiguous or mismatching readback is `HOLD_UNVERIFIED`.

T16 — CONFORMANCE_BINDING

A TEMDD v1 PASS is valid only when the conformance report binds all of:

- implementation repository;
- exact implementation HEAD;
- exact implementation TREE;
- implementation digest;
- normative suite version and digest;
- all required semantic layers;
- positive and negative vector results.

The conformance runner MUST be separated from the reference implementation and MUST invoke the implementation through an adapter/process boundary rather than importing implementation internals as its oracle.

```text
TEMDD_CONFORMANT(S) :=
  LANGUAGE_CONFORMANT(S)
  and IR_CONFORMANT(S)
  and EVENT_SEMANTICS_CONFORMANT(S)
  and LEDGER_CONFORMANT(S)
  and EVIDENCE_BINDING_CONFORMANT(S)
  and FORMAL_INVARIANTS_SATISFIED(S)
  and TEST_SUITE_PASS(S)
  and NEGATIVE_VECTORS_PASS(S)
  and REPORT.subject == S
  and REPORT.suite == normative_suite
```

Canonical pipeline

```text
TEMDD_PROGRAM
  -> parse
AST
  -> elaborate/type/effect check
TEMDD_IR
  -> execute
EVENTS
  -> observe + bind
LEDGER
  -> evaluate contracts
RESULT
```

The IDE is a projection over this pipeline:

```text
SOURCE <-> IR <-> EVENT GRAPH <-> LEDGER <-> CONFORMANCE
```

`IDE_ANALYSIS != EXECUTION`, `IDE_RUNS != CONFORMANCE`, `LEDGER_ROW != EVIDENCE_TRUTH`, `CONFORMANCE != P3`, `CONFORMANCE != MAIN`, and `CONFORMANCE != EFFECT`.

DONE

`DONE` requires fresh, exact-subject-bound required evidence and successful conformance. Repository review, protected promotion, exact-Main reobservation and external EFFECT readback remain separate later gates.

A6 - S07 / next/core/include/qikvrt_transputer.h

Archivkennung F094; Ursprungspfad next/core/include/qikvrt_transputer.h

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 6436fe95e18ad912fbc01899957698edd8e98d55; SHA-256 298d87d7fb87ffe82b08a7820635a4427cd3ce7035ccb8fa4a09146cc9bded81; 3989 Byte

```
0001  /* SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0002   * Copyright 2026 Ingolf Lohmann. Implementation: OpenAI Codex. */
0003  #ifndef QIKVRT_TRANSPUTER_H
0004  #define QIKVRT_TRANSPUTER_H
0005  #include "qikvrt_wire_v1.h"
0006  #include <stddef.h>
0007
0008  #define QIKVRT_OBSERVE 0U
0009  #define QIKVRT_HOLD 1U
0010  #define QIKVRT_CONTINUE 2U
0011  #define QIKVRT_ROUTE_BYTES 172U
0012  #define QIKVRT_CHUNK_BYTES (QIKVRT_WIRE_MAX_PAYLOAD-QIKVRT_ROUTE_BYTES)
0013  #define QIKVRT_MAX_CHUNKS 16U
0014  #define QIKVRT_MAX_MESSAGE (QIKVRT_MAX_CHUNKS*QIKVRT_CHUNK_BYTES)
0015  #define QIKVRT_CODEC_COMMAND 1U
0016  #define QIKVRT_CODEC_OBJECT 2U
0017  #define QIKVRT_CODEC_RECEIPT 3U
0018
0019  typedef struct qikvrt_input_s {
0020      qikvrt_u32 a, b;
0021      unsigned char lut, requested, binding, authority, distinction, drift;
0022  } qikvrt_input;
0023  typedef struct qikvrt_output_s {
0024      qikvrt_u32 value;
0025      unsigned char state, value_valid;
0026  } qikvrt_output;
0027  qikvrt_u32 qikvrt_boolean_lut(qikvrt_u32 a, qikvrt_u32 b, unsigned char lut);
0028  qikvrt_output qikvrt_evaluate(qikvrt_input input);
0029  /* Stable octet ABI: input = BE32(a), BE32(b), six controls; output =
0030   * BE32(value), state, valid. No native struct layout crosses a language. */
0031  void qikvrt_evaluate_bytes(const unsigned char *input, unsigned char *output);
0032  int qikvrt_effect_to_wire(unsigned char internal, unsigned char *wire);
0033  int qikvrt_effect_from_wire(unsigned char wire, unsigned char *internal);
0034
0035  /* QXT2 route inside the unchanged QVRT v1 payload. Layer IDs 0..7,
0036   * source/destination/exact-subject SHA256, whole-message SHA256 and size.
0037   * Digests identify bytes; they are NOT credentials or effect approval. */
0038  int qikvrt_route(unsigned char *route,
0039      const unsigned char *source, const unsigned char *destination,
0040      const unsigned char *subject, unsigned char source_layer,
0041      unsigned char destination_layer, unsigned char codec,
0042      const unsigned char *body, unsigned int length);
0043  /* A caller-owned transport callback must copy/consume a complete frame.
0044   * Backpressure/failure returns zero; no retry, discard, or DONE inference. */
0045  typedef int (*qikvrt_emit_fn)(void *context, const unsigned char *frame, unsigned int length);
0046  int qikvrt_send(const unsigned char *route,
0047      const unsigned char *body, unsigned int length, const qikvrt_wire_frame *key,
0048      unsigned char *frame_workspace, unsigned int capacity,
0049      qikvrt_emit_fn emit, void *context);
0050
0051  /* Incremental receiver. Storage belongs to its caller. One transfer at a
0052   * time, any chunk order; same-byte retransmission is harmless; conflicts
0053   * poison this receiver until the caller explicitly initializes a new one. */
0054  typedef struct qikvrt_receiver_s {
0055      unsigned char expected[96], source_layer, destination_layer;
0056      unsigned char route[QIKVRT_ROUTE_BYTES];
0057      unsigned char *storage;
0058      unsigned int capacity, length;
0059      qikvrt_u32 seen;
0060      qikvrt_wire_frame key;
0061      int started, failed, complete;
0062  } qikvrt_receiver;

0063  int qikvrt_receiver_init(qikvrt_receiver *r, unsigned char *storage,
0064      unsigned int capacity, const unsigned char *source,
0065      const unsigned char *destination, const unsigned char *subject,
0066      unsigned char source_layer, unsigned char destination_layer);
```

```
/* 0=HOLD, 1=partial, 2=complete (bytes checked), 3=duplicate. Never effect ACK. */
int qikvrt_receive(qikvrt_receiver *r, const unsigned char *frame, unsigned int length);
size_t qikvrt_receiver_size(void);
size_t qikvrt_receiver_alignment(void);
unsigned int qikvrt_receiver_length(const qikvrt_receiver *r);
unsigned char qikvrt_receiver_codec(const qikvrt_receiver *r);
void qikvrt_receiver_correlation(const qikvrt_receiver *r,unsigned char out[32]);

/* Single-frame octet FFI, avoiding native header layout across languages. */
int qikvrt_pack_bytes(const unsigned char *metadata, const unsigned char *payload,
    unsigned int length, unsigned char *out, unsigned int capacity, unsigned int *written);
int qikvrt_unpack_bytes(const unsigned char *frame, unsigned int length,
    unsigned char *metadata, unsigned char *payload, unsigned int capacity);
#endif
```

A7 - S08 / next/hardware/metatransistor_tile.vhd

Archivkennung F095; Ursprungspfad next/hardware/metatransistor_tile.vhd

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob f123dd4f14e0002da5210a37b8bd1a071e6fffce; SHA-256 e77da78cafae0f16a080705c6cb51a76c45080ddf70cbd4d11bd62b3dafd7058; 2896 Byte

```
0001  -- SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0002  -- Copyright 2026 Ingolf Lohmann. Generated implementation: OpenAI Codex.
0003  library ieee;
0004  use ieee.std_logic_1164.all;
0005  use ieee.numeric_std.all;
0006  use work.qikvrt_metatransistor_pkg.all;
0007
0008  -- A composable one-entry ready/valid tile with 32 parallel programmable
0009  -- Boolean lanes. Tags and results remain stable under backpressure.
0010  entity metatransistor_tile is
0011    port (
0012      clk_i, reset_i, in_valid_i, out_ready_i : in std_logic;
0013      in_ready_o, out_valid_o : out std_logic;
0014      a_i, b_i, tag_i : in std_logic_vector(31 downto 0);
0015      lut_i : in std_logic_vector(3 downto 0);
0016      requested_i : in qikvrt_state_t;
0017      binding_i, authority_i, distinction_i, drift_i : in std_logic;
0018      value_o, tag_o : out std_logic_vector(31 downto 0);
0019      state_o : out qikvrt_state_t;
0020      value_valid_o : out std_logic
0021    );
0022  end entity;
0023
0024  architecture rtl of metatransistor_tile is
0025    signal valid_q : std_logic := '0';
0026    signal ready : std_logic;
0027    signal state_q : qikvrt_state_t := QIKVRT_OBSERVE;
0028    signal value_q, tag_q : std_logic_vector(31 downto 0) := (others=>'0');
0029    function known(v:std_logic_vector) return boolean is
0030    begin
0031      for n in v'range loop
0032        if v(n)/='0' and v(n)/='1' then return false; end if;
0033      end loop;
0034      return true;
0035    end function;
0036  begin
0037    ready <= '1' when reset_i='0' and (valid_q='0' or out_ready_i='1') else '0';
0038    in_ready_o <= ready;
0039    out_valid_o <= valid_q;
0040    state_o <= state_q;
0041    value_o <= value_q;
0042    tag_o <= tag_q;
0043    value_valid_o <= '1' when valid_q='1' and state_q=QIKVRT_CONTINUE else '0';
0044    process(clk_i)
0045      variable normalized:qikvrt_state_t;
0046      variable v:std_logic_vector(31 downto 0);
0047      variable pair:std_logic_vector(1 downto 0);
0048    begin
0049      if rising_edge(clk_i) then
0050        if reset_i/='0' then
0051          valid_q<='0';state_q<=QIKVRT_OBSERVE;
0052          value_q<=(others=>'0');tag_q<=(others=>'0');
0053        elsif ready='1' then
0054          valid_q<='0';
0055          if in_valid_i='1' then
0056            normalized:=normalize_state(requested_i,binding_i,authority_i,distinction_i,drift_i);
0057            -- Preserve the source package bytes while closing its unknown-drift case.
0058            if drift_i/='0' or not known(a_i) or not known(b_i)
0059              or not known(lut_i) or not known(tag_i) then normalized:=QIKVRT_HOLD; end if;
0060            v:=(others=>'0');
0061            if normalized=QIKVRT_CONTINUE then
0062              for n in 0 to 31 loop

0063                pair:=a_i(n)&b_i(n);
0064                case pair is
0065                  when "00"=>v(n):=lut_i(0);
0066                  when "01"=>v(n):=lut_i(1);
```

```
                  when "10"=>v(n):=lut_i(2);
                  when "11"=>v(n):=lut_i(3);
                  when others=>v(n):='0';
                end case;
              end loop;
            end if;
            state_q<=normalized;value_q<=v;tag_q<=tag_i;valid_q<='1';
          end if;
        end if;
      end if;
    end process;
  end architecture;
```

A8 - S09 / docs/publications/2026-07-22-qik-vrt-status-clarification/PATENT_DISCLOSURE_BOUNDARY.md

Archivkennung F096; Ursprungspfad docs/publications/2026-07-22-qik-vrt-status-clarification/PATENT_DISCLOSURE_BOUNDARY.md

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob ae302b26a5b0e2ad092c9d6fddc4a462fff29dff; SHA-256 e665eb02d53dd6038a8754246dcf97ee27d91d1247e49dbe2a1e58cf96151f55; 6404 Byte

```
0001  <!--
0002  SPDX-License-Identifier: CC-BY-NC-ND-4.0
0003  Copyright 2026 Ingolf Lohmann.
0004  -->
0005
0006  # Patent disclosure boundary
0007
0008  ## Zweck und Nichtberatungs-Hinweis
0009
0010  Dieses Dokument grenzt die technische Veröffentlichung von möglichen
0011  patentrechtlichen Aussagen ab. Es ist keine Rechts- oder Patentanwaltsberatung,
0012  keine Patentanmeldung, kein Claim Chart und kein Rechtsgutachten. Maßgeblich
0013  sind die betroffenen Rechtsordnungen, die konkreten Ansprüche, die vollständige
0014  Chronologie und die Prüfung durch dafür qualifizierte Berater und Behörden.
0015
0016  Die [WIPO](https://www.wipo.int/en/web/patents) beschreibt ein Patent als ein
0017  für eine Erfindung erteiltes Ausschließlichkeitsrecht und verweist für Schutz
0018  auf Anmeldungen bei nationalen oder regionalen Ämtern. Eine technische
0019  Veröffentlichung, ein DOI, ein Git-Tag oder ein Internet-Draft ist daher nicht
0020  selbst die Erteilung eines solchen Rechts.
0021
0022  Für eine europäische Neuheitsprüfung erfasst [Artikel 54
0023  EPÜ](https://www.epo.org/en/legal/epc/2020/a54.html) grundsätzlich den vor dem
0024  Anmelde- oder Prioritätstag öffentlich zugänglich gemachten Stand der Technik.
0025  Welche Wirkung eine konkrete Offenlegung in welcher Rechtsordnung hat, kann
0026  dieses Dokument nicht entscheiden. Insbesondere wird keine Schonfrist,
0027  Prioritätswirkung oder Unschädlichkeit einer Veröffentlichung zugesagt.
0028
0029  Für europäische computerimplementierte Erfindungen behandelt das
0030  [Europäische Patentamt Programme für
      Datenverarbeitungsanlagen](https://www.epo.org/en/legal/guidelines-epc/2026/g_ii_3_6.html)
0031  nicht schon wegen ihrer Ausführung auf einem Computer als patentfähig; die
0032  Prüfung verlangt einen weiteren technischen Effekt beziehungsweise technischen
0033  Beitrag im konkreten Anspruch. Auch dessen Vorliegen würde Neuheit und
0034  erfinderische Tätigkeit nicht vorwegnehmen.
0035
0036  ## Öffentlich dokumentiert
0037
0038  Am 22. Juli 2026 waren jedenfalls folgende technische Primärartefakte
0039  öffentlich adressierbar:
0040
0041  - das [QIK-VRT/EFFECT_ACK Working
0042    Paper](https://doi.org/10.5281/zenodo.21498773);
0043  - der [versionierte Software-Snapshot](https://doi.org/10.5281/zenodo.21498774);
0044  - der [Internet-Draft
0045    `draft-lohmann-qikvrt-effect-ack-01`](https://datatracker.ietf.org/doc/draft-lohmann-qikvrt-effect-ack/
      01/);
0046  - die veröffentlichten Quellstände in den verlinkten GitHub-Repositories.
0047
0048  Diese Liste belegt nur die benannten öffentlichen Fundstellen. Sie ist keine
0049  vollständige Prior-Art-Suche und trifft keine Aussage darüber, ob gleiche oder
0050  ähnliche Inhalte früher, anderswo oder durch Dritte offengelegt wurden.
0051
0052  ## Keine positive Patentstatus-Aussage
0053
0054  Aus den veröffentlichten Artefakten und dieser Statusklärung folgt keine
0055  Garantie oder Feststellung zu:
0056
0057  - Anmeldung, Anhängigkeit, Erteilung oder territorialer Reichweite eines
0058    Patents;
0059  - Anmelde-, Prioritäts-, Veröffentlichungs- oder Erfindungsdatum im
0060    patentrechtlichen Sinn;

0061  - Erfindereigenschaft, Berechtigung, Inhaberschaft oder wirksamer Übertragung;
```

```
- Neuheit, erfinderischer Tätigkeit, gewerblicher Anwendbarkeit oder
  Patentfähigkeit;
- ausreichender Offenbarung oder Ausführbarkeit im patentrechtlichen Sinn;
- Rechtsbeständigkeit, Anspruchsauslegung oder Durchsetzbarkeit;
- Freedom to Operate oder Nichtverletzung fremder Patent-, Urheber-, Marken-,
  Geschäftsgeheimnis- oder sonstiger Rechte.

Auch ein positives Ergebnis einer Repository-Prüfung, ein DOI-Status oder
`EFFECT_ACK_DONE` wäre kein Ersatz für diese eigenständigen rechtlichen
Prüfungen.

## Technische Offenlegungsgrenze dieses Bundles

Dieses Statusbundle fasst bereits öffentlich dokumentierte mathematische und
prozessuale Grenzen zusammen. Es fügt bewusst keine neuen enabling
Hardwaredetails hinzu. Nicht beschrieben werden insbesondere:

- Schaltpläne, Masken-, ASIC-, FPGA- oder Leiterplattenentwürfe;
- konkrete Sensor-, Aktor-, Verriegelungs- oder Redundanzarchitekturen;
- Schlüsselablage, Hardware-Root-of-Trust oder sichere Provisionierung;
- produktspezifische Zeit-, Leistungs-, Toleranz- oder Ausfallparameter;
- Fertigungs-, Kalibrierungs-, Zertifizierungs- oder Gefährdungsverfahren;
- eine konkrete Implementierung, die aus abstrakten Bedingungen ein
  betriebsfähiges cyberphysisches Produkt macht.

Die Begriffe „treuer Executor“, „vollständige Mediation“ und „empirisch
validiertes Systemmodell“ sind notwendige Bedingungsgrenzen. Sie sind keine
Behauptung, dass eine bestimmte Hardware diese Bedingungen erfüllt, und keine
Anleitung, wie sie vollständig zu realisieren wäre.

## Keine Totalisierungsbrücke zum Patent

Das mathematische Faser-Kriterium und die endliche Modellprüfung begründen
weder eine universelle technische Lösung noch einen automatischen
Patentanspruch. Insbesondere darf die wissenschaftliche Kennzeichnung
`PASS_WITH_EXPLICIT_BOUNDARIES` nicht in eine positive Garantie für
Patentfähigkeit, Schutzumfang oder Verwertbarkeit umgedeutet werden.

## Copyright-Lizenz ist keine Patentlizenz

Die Texte dieses Bundles stehen unter CC BY-NC-ND 4.0. Nach Abschnitt 2(b)(2)
des [offiziellen
Creative-Commons-Lizenztexts](https://creativecommons.org/licenses/by-nc-nd/4.0/legalcode.en)
werden Patent- und Markenrechte durch diese Lizenz nicht lizenziert. Die
Copyright-Erlaubnisse dürfen deshalb nicht als Patentlizenz, Zusicherung der
Nichtverletzung oder Verzicht auf sonstige Rechte gelesen werden.

## Erforderliche getrennte Prüfung

Wer eine Anmeldung, Lizenzierung, Verwertung oder ein Produkt erwägt, muss vor
einer Wirkung mindestens getrennt prüfen lassen:

1. vollständige Offenlegungs- und Prioritätschronologie;
2. Erfinder-, Inhaber- und Übertragungskette;
3. Anspruchsfassung und jurisdictionsspezifische Patentfähigkeit;
4. einschlägigen Stand der Technik und Freedom to Operate;
5. Software-, Standardisierungs-, Lizenz- und Drittanbieterrechte;
6. Safety-, Security-, Export-, Produkt- und Zulassungsanforderungen.

Bis dahin bleibt jede positive Patent- oder Verwertungsaussage ausdrücklich
offen.

Ein internationales Verfahren nach dem PCT ist kein „Weltpatent“; die
Erteilung bleibt Sache der zuständigen nationalen oder regionalen Ämter. Auch
ein erteiltes europäisches Patent ist nicht „unangreifbar“, sondern kann unter
den Voraussetzungen der [Artikel 99](https://www.epo.org/en/legal/epc/2020/a99.html)
und [100 EPÜ](https://www.epo.org/en/legal/epc/2020/a100.html) Gegenstand eines
Einspruchs sein.
```

A9 - S10 / runtime/self-host/README.md

Archivkennung F097; Ursprungspfad runtime/self-host/README.md

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 0f7fff71870dd68707e75325ae7acbd56901842a; SHA-256 862acc5d841f76be8dc587a1dfa0f1610ef274717c5e0696f8d324a7a05aed35; 8829 Byte

```
<!-- SPDX-License-Identifier: CC-BY-NC-ND-4.0 -->
# S1 standalone node package, version 1.1.0

This package reuses the existing Node monitor and reference terminal. It now
also includes the recovered original productive TEMDD ledger/Unix daemon and
Firefox/noVNC deployment sources from snapshot
`c6a53e200c4835a4f871d8496feb81d953610831`. Exact source provenance and the new
portable S1 adapter are described in [RECOVERED_TERMINAL.md](RECOVERED_TERMINAL.md).
The recovered originals remain byte-identical. No second ledger or executor is
introduced. Select `temdd` (the example default), `firefox`, or the compatibility
`reference` profile explicitly. Missing profile retains the existing reference
behavior for prior private configurations.
Linux x86-64/ARM64 operators provision Node 24.x and Python 3.12.x. The builder
freezes their actual patch versions, architectures and executable SHA-256;
startup rejects any substitution. Python's renderer pin remains unchanged.

`tools/bootstrap-runtime.sh --check-only --profile self-host` performs no
GitHub CLI check or installation. Its default adapter is `none`. Selecting
`--adapter github` retains the locked GitHub CLI bootstrap. Every pre-existing
profile still requires that bootstrap and refuses `--adapter none`.
The PowerShell checker carries the same adapter selection; the actual service
launcher is Linux-only. The profile checker is readiness, not deployment proof.

## Freeze, transfer and start

In a clean reviewed checkout run:

```sh
python3 -B tools/qikvrt_self_host.py pack --root . \
  --expected-head <exact-commit> --expected-tree <exact-tree> \
  --output /absolute/new/package-directory
```

The output is a directory and deterministic uncompressed `.tar` of explicitly
allowlisted **committed** blobs with exact sizes, hashes and Git source binding.
It contains neither credentials nor `.git` nor private state. An independent
receiver must obtain the printed manifest pin through a trusted channel, verify
the archive bytes, and safely extract regular files. No online installer, gh,
Railway, npm install or mutable remote checkout is used to start the export.
The declared tools are operator-provisioned, not included binary payloads.

Copy `CONFIG.example.json` outside the package, replace all placeholders and
bind an authorized node identity, absolute volume path, ports and source pair.
Config and secret files must belong to the runtime UID and be mode 0600; the
volume must be mode 0700. Supply a private 32–256-character ASCII terminal token
from the host's existing secret provisioner. Never put a token in argv, source,
the package, caches or receipts. Default listener is loopback. Bind `0.0.0.0`
only behind the host's existing authorized HTTPS routing. No DNS, account,
firewall, billing or routing changes are made by this package.

```sh
python3 -B tools/qikvrt_self_host.py verify --root <package-directory> \
  --manifest-sha256 <independently-obtained-pin>
python3 -B tools/qikvrt_self_host.py run --root <package-directory> \
  --manifest-sha256 <same-pin> --config <absolute-private-config>
```

The launcher and monitor share an OS file lock for the entire service lifetime,
starts the selected original native or reference terminal on loopback and the
existing Node monitor, and shuts both down on termination. A failed bind never becomes a
ready receipt. Configure restart in the host's existing supervisor, using this
exact command and unchanged private config. No second supervision/execution

system or platform service is created. An isolated launcher crash leaves the
surviving monitor holding the lock; restart is refused until it also stops.
Kernel lock release after all holders exit permits crash restart.
```

The separately mounted volume layout is:

Path relative to configured volume	Contract
`binding.json`	fsynced node/package/config/source binding; drift refuses startup
`node.lock`	owner-only OS lock; a second live writer is refused
`monitor/node.json`	existing exact journal and checkpoint; fsync/rename semantics unchanged
`temdd/`	existing owner-only SQLite ledger/Unix ingress location; no substitute ledger is created
`receipts/`	reserved private operator receipts; not a cache or public document route

A different package/config against an existing volume requires separately reviewed migration. Do not delete or relabel acknowledged data to defeat the binding. A real persistent mount/device identity, capacity and power-loss guarantees remain host admission evidence; an ordinary local directory is not proof of host persistence. The runtime binds only its configured filesystem.

Network, authentication and readback

Adapter `none` observes its local packaged source, never GitHub. GitHub ingress and live-run routes are refused. The outbound provider wrapper rejects Railway and every external provider before transport; only loopback is allowed. With the explicitly selected GitHub adapter, HTTPS `api.github.com` is permitted and its webhook requires the existing HMAC boundary. Startup requires an owner-only `github_webhook_secret_file` containing a 32–256-byte secret. The host must apply its own OS egress policy; the application wrapper is not a process sandbox.

Public GET/HEAD routes reuse monitor activity/events/SSE and the read-only terminal at `/AI/` or `/terminal/`. Terminal repository reads resolve only allowlisted public exported files; the immutable package commit is explicitly distinguished from a current remote `main`. No public terminal effect or owner Unix operation is exposed. `/api/terminal` returns sanitized runtime metadata, not private input, record bodies, tokens or secret-file paths. Reference HTTP endpoints require the configured bearer token. Native subject and SSE reads retain the recovered loopback/Host/same-Origin boundary. Private native events are never proxied through the public monitor. Owner durable Prepare/Commit remain the existing Unix peer-UID/socket/SQLite client contract.

Run the **existing independent monitor client** in a separate process:

```sh
node tools/qikvrt_mesh_monitor_readback.mjs --self-host <package-directory> \
  <exact-origin-url> <manifest-pin> <config-sha256> <expected-node-id> none
```

It verifies runtime/config/source pins, terminal binding, exact client assets, snapshot, original journal bytes/digest chain and a final fresh node readback. It uses the exported original client bytes, with no Git or provider dependency. The original Railway plan path remains compatible when its existing contract is installed; self-host selects an explicit separate plan. No credentials are needed in the readback process. A local URL proves only local HTTP readback.

Executable S1 validation

On the declared Linux runtime run `make test repository-monitor-test self-host-test`. The existing full repository suite stays intact; the two explicit S1 targets add the original monitor controls and standalone package controls. CI, repository evidence, global completion, collective review and Batch-003 remaining subject disposition require all three targets after provisioning the S1 runtime. The existing terminal workflow additionally requires all 31 HTTP/owner-Unix controls and freezes the actual source package. A profile-specific Linux test is not imposed on unrelated portable callers.

Precise remaining acceptance

The former productive source gap is resolved for the exact recovered historical snapshot. The original Goldkelch commit declared by that snapshot could not be retrieved directly, and equality with a later Railway production build remains unverified. The selected native daemon runs its original SQLite/WAL and Unix implementation through the new sealed-source adapter. Reference HTTP remains process-local and does not supply durable native evidence.

The native suite exercises actual owner Prepare/Commit, fresh SQLite/SSE, wrong subject, drift, duplicate commit and crash restart. The terminal workflow also builds the declared Linux Firefox carrier and runs the actual candidate export with only loopback networking, no Git/gh in its runtime PATH, real X11 window and VNC WebSocket readbacks, plus native/browser crash restart. These

```
0143  checks establish only the execution recorded by the immutable workflow head.
0144
0145  No callable own-host deployment operation, verified host identity, durable mount
0146  or public route is available in this task. The deployment work unit records
0147  `HOLD_OWN_HOST_IDENTITY_STORAGE_ROUTING_UNAVAILABLE` and references the historical source recovery
      separately. Container and local
0148  restart tests do not close the host admission condition.
0149  S1 remains OPEN until host admission, exact deployment,
0150  fresh public use/readback and deployed restart acceptance are established.
0151  No public URL, S1 DONE or EFFECT_ACK_DONE is asserted.
```

ANHANG CI - Vollständige Runtime- und Restart-Receipts

FIREFOX_NATIVE_READBACK.json


```
{
  "browser_executables": {
    "Xvfb": "6eda36f0c84969a0fbe3f411c4a16521620db8a16c13defbee690cad86bd9ef8",
    "firefox-esr": "617a7a905680b94f8d00e9bf92b8f03705389094e43edb9c7363756ca4b25327",
    "websockify": "1a14abe56973818410c2a504f66496645dcb0df1b5a3d499a7b6f0cf0c848388",
    "x11vnc": "84af9b01b1961384e4f6e3f9d41bee5adb54c304fe81da2b8db2e61b22ad0652",
    "xdpyinfo": "e7e1a29eb970f5eb5c0592a4fc4dd4c6cafd5d79358dfb7b27a81b74d060ccca",
    "xwininfo": "fcd45fa8cf04e0c62086dbbb3e9d4762b0080cc41622cf53a5bddea92b7be84c"
  },
  "config_sha256": "43f82aeb139f1c6fb071e6e993a4a40890493d04763fdc40a03ffcf272399905",
  "container_init": {
    "executable_sha256": "5fb35102eb8606dfe20aa755229becc81e6fdf3ba587d64b58dca51b79c0b36a",
    "name": "docker-init"
  },
  "deployment_performed": false,
  "effect_ack_done": false,
  "firefox_navigator_window_observed": true,
  "firefox_version": "Mozilla Firefox 153.4.0esr",
  "gh_in_runtime_path": false,
  "git_in_runtime_path": false,
  "independent_host_identity_verified": false,
  "ledger_id": "ab3f7945028a4d2d8fa3e69060b88a64",
  "manifest_sha256": "a8e2330eb2c9421d1686486f3a96f593dcaceff908f55a90f4013e6f7a541c76",
  "native_durable_event": {
    "dod": false,
    "evidence_transfer": "DENY",
    "id": "ab3f7945028a4d2d8fa3e69060b88a64:1",
    "kind": "OBSERVE",
    "ledger_digest": "8a08962baff2098d603f0db561382af3b35bdf29aea2f11933731afc5c0a0432",
    "message": "Universal Terminal: hash-bound durable input observation",
    "observed_at": "2026-10-05T03:07:12Z",
    "payload": {
      "adapter": "QIKVRT_OWNER_UNIX_TERMINAL_V1",
      "effect_ack_done": false,
      "input_hash": "sha256:8c4a446c1b863d0d1f85428cf821761a60961229015fe551091851f28a6371d7",
      "terminal_input": {
        "schema": "qikvrt_terminal_input_v1",
        "text": "private-test-input: Grüße Ω"
      }
    },
    "payload_digest": "e63304ea095e913e2f26b6135d1ece0edae6b98cb74447fa8a5e6ac57879535a",
    "provenance": {
      "native_event_id": "terminal:10db997e36dff12aa2b2cdd2773d8763d2db3f401ac9e3a3",
      "source": "transputer"
    },
    "recorded_at": "2026-10-05T03:07:13.090853Z",
    "schema": "qikvrt_temdd_event_v1",
    "subject": {
      "head": "3bf52747bc94de847d390ffacabecae183aee588",
      "pr": 457,
      "repository": "ingolf-lohmann/qik-vrt",
      "tree": "96c7bb691675dcd15bb5045871201adb400074f0"
    }
  },
  "native_subject": {
    "head": "3bf52747bc94de847d390ffacabecae183aee588",
    "pr": 457,
    "repository": "ingolf-lohmann/qik-vrt",
    "tree": "96c7bb691675dcd15bb5045871201adb400074f0"
  },
  "native_terminal_document_title_observed": true,
  "observed_at": "2026-10-05T03:07:13.280286+00:00",
  "observed_listeners": [
    {
      "address": "127.0.0.1",
      "port": 34035
    },
    {
      "address": "127.0.0.1",
      "port": 45667
    },
```

```
    {
      "address": "127.0.0.1",
      "port": 48925
    },
    {
      "address": "127.0.0.1",
      "port": 58213
    }
  ],
  "observed_network_interfaces": [
    "lo"
  ],
  "outbound_network": "DOCKER_NETWORK_NONE",
  "public_https_readback_verified": false,
  "schema": "qikvrt-self-host-firefox-runtime-readback/v1",
  "scope": "ACTUAL_EXACT_CANDIDATE_RUNTIME_IN_CI_CONTAINER",
  "source_head": "3bf52747bc94de847d390ffacabecae183aee588",
  "source_tree": "96c7bb691675dcd15bb5045871201adb400074f0",
  "transport": {
    "rfb_version": "RFB 003.008",
    "vnc_authentication_required": true,
    "websocket_status": 101
  }
}
```


FIREFOX_NATIVE_RESTART_READBACK.json


```
{
  "crashed_xvfb_pids": [
    1002
  ],
  "crashed_xvfb_reaped": true,
  "effect_ack_done": false,
  "manifest_sha256": "a8e2330eb2c9421d1686486f3a96f593dcaceff908f55a90f4013e6f7a541c76",
  "native_durable_event": {
    "dod": false,
    "evidence_transfer": "DENY",
    "id": "27099ef895c14f30968edc8e30f156df:1",
    "kind": "OBSERVE",
    "ledger_digest": "768b774a52e36ce9fb9b875f7fbdbc12eb7add49c09aae66064869778a120d9c",
    "message": "Universal Terminal: hash-bound durable input observation",
    "observed_at": "2026-10-05T03:07:17Z",
    "payload": {
      "adapter": "QIKVRT_OWNER_UNIX_TERMINAL_V1",
      "effect_ack_done": false,
      "input_hash": "sha256:8c4a446c1b863d0d1f85428cf821761a60961229015fe551091851f28a6371d7",
      "terminal_input": {
        "schema": "qikvrt_terminal_input_v1",
        "text": "private-test-input: Grüße Ω"
      }
    },
    "payload_digest": "e63304ea095e913e2f26b6135d1ece0edae6b98cb74447fa8a5e6ac57879535a",
    "provenance": {
      "native_event_id": "terminal:91a9aad1632fdb8e07efa96bf78f026fe2a6e5dcf18dc6c4",
      "source": "transputer"
    },
    "recorded_at": "2026-10-05T03:07:17.608013Z",
    "schema": "qikvrt_temdd_event_v1",
    "subject": {
      "head": "3bf52747bc94de847d390ffacabecae183aee588",
      "pr": 457,
      "repository": "ingolf-lohmann/qik-vrt",
      "tree": "96c7bb691675dcd15bb5045871201adb400074f0"
    }
  },
  "native_original_rows_preserved": true,
  "observed_listeners_after_restart": [
    {
      "address": "127.0.0.1",
      "port": 34211
    },
    {
      "address": "127.0.0.1",
      "port": 36105
    },
    {
      "address": "127.0.0.1",
      "port": 37415
```

```
    },
    {
      "address": "127.0.0.1",
      "port": 50617
    }
  ],
  "profile_marker_preserved": true,
  "public_https_readback_verified": false,
  "schema": "qikvrt-self-host-firefox-restart-readback/v1",
  "source_head": "3bf52747bc94de847d390ffacabecae183aee588",
  "source_tree": "96c7bb691675dcd15bb5045871201adb400074f0"

}
```


CI_BINDING_VERIFICATION.json


```
{
  "schema": "qikvrt-source-band-ci-binding/v1",
  "observation_date": "2026-10-05",
  "run_id": 37257971138,
  "artifact_id": 11323124612,
  "run_head": "3bf52747bc94de847d390ffacabecae183aee588",
  "run_tree": "96c7bb691675dcd15bb5045871201adb400074f0",
  "run_status": "completed",
  "run_conclusion": "success",
  "run_created_at": "2026-10-05T03:05:17Z",
  "run_completed_updated_at": "2026-10-05T03:08:26Z",
  "artifact_zip_sha256": "fe3b6daca01f188642de02e2b6c8d59a5ff217db4c4682dba7a3b5c6306a569b",
  "artifact_zip_bytes": 18173,
  "artifact_api_digest_matches": true,
  "archive_members": [
    {
      "path": "evidence/native/DISTRIBUTION_PACKAGES.tsv",
      "sha256": "e4e58084ac858a625c180d5ce0c0996f301b0cc7b7dd0acfd3e2014247538415",
      "bytes": 8926
    },
    {
      "path": "evidence/native/DOCKER_ENGINE_VERSION.json",
      "sha256": "bb01e98a1e16e838dd91bea46c0ab4bf0911443351a65b541a65cf7548ea7313",
      "bytes": 9
    },
    {
      "path": "evidence/native/FIREFOX_NATIVE_READBACK.json",
      "sha256": "61da00092d448ede9acba5de5063522a3cbd7739570e748f93308101d9cad0bc",
      "bytes": 3479
    },
    {
      "path": "evidence/native/FIREFOX_NATIVE_RESTART_READBACK.json",
      "sha256": "529714f8dec53a3949d4805d9e68065b29febaba6afe352b0aa436a682355e11",
      "bytes": 2003
    },
    {
      "path": "evidence/native/MANIFEST.json",
      "sha256": "a8e2330eb2c9421d1686486f3a96f593dcaceff908f55a90f4013e6f7a541c76",
      "bytes": 43084
    },
    {
      "path": "evidence/native/OCI_IMAGE_ID.json",
      "sha256": "b304fd3c7495499bb24af07000e06d2c448272127a503aaac6581bfe0d9d5d73",
      "bytes": 74
    }
  ],
  "record_payload_and_ledger_digest_verified": true,
  "current_source_read_does_not_reexecute_ci": true,
  "scope": "ACTUAL_EXACT_CANDIDATE_RUNTIME_IN_CI_CONTAINER",
  "deployment_performed": false,
  "public_https_readback_verified": false,
  "independent_host_identity_verified": false,
  "physical_hardware_verified": false,
  "effect_ack_done": false
}
```

ANHANG B - Vollständige zentrale Implementierungs- und Testquellen

Diese Listings decken die unmittelbar behandelten Eingabe-, Speicher-, Start-, Sprach-, Nachrichten- und RTL-Mechanismen ab. Weitere ergänzende Dateien und die vollständigen Lizenztexte werden im ZIP unverändert bereitgestellt. Ein Listing ist Quellenbeleg und wird nicht mit einem frischen Laufbeleg gleichgesetzt.

B1 - src/qikvrt_temdd_event_ledger.py

Archivkennung F082; Ursprungspfad src/qikvrt_temdd_event_ledger.py

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 6322bd45edb9de7d2aa07dee6fbc12307148475c; SHA-256 94908e6a50ce107a79965055f3ecabae10e7589d96f7a26496d2a4956b068b13; 21653 Byte

```
0001  #!/usr/bin/env python3
0002  # SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0003  """Synchronous TEMDD ledger; socket/SSE transport lives outside the kernel.
0004
0005  No scheduler, repository polling, review, promotion, or DoD inference. The
0006  owner-only Unix socket is a local native-producer trust boundary, not a GitHub
0007  webhook authenticator. A successful append means durable ledger readback only.
0008  """
0009  from __future__ import annotations
0010
0011  import argparse
0012  import contextlib
0013  from datetime import datetime, timezone
0014  import fcntl
0015  import hashlib
0016  import json
0017  import os
0018  from pathlib import Path
0019  import re
0020  import selectors
0021  import socket
0022  import socketserver
0023  import sqlite3
0024  import stat
0025  import subprocess
0026  import sys
0027  import threading
0028  from urllib.parse import parse_qs, urlsplit
0029  import uuid
0030
0031  # Also supports loading as src.qikvrt_temdd_event_ledger in repository tests.
0032  try:
0033      from . import qikvrt_effect_ack_http_terminal as terminal
0034  except ImportError:
0035      import qikvrt_effect_ack_http_terminal as terminal
0036
0037  MAX_EVENT = 65536
0038  KINDS = frozenset({'OBSERVE', 'CLASSIFY', 'ACTION', 'EFFECT', 'READBACK',
0039                     'SUCCESSOR', 'HOLD'})
0040  SCHEMA = 'qikvrt_temdd_native_event_v1'
0041  HEX40 = re.compile(r'[0-9a-f]{40}\Z')
0042  CURSOR = re.compile(r'([0-9a-f]{32}):([1-9][0-9]{0,18})\Z')
0043
0044
0045  class Hold(ValueError):
0046      """Fail-closed observation/ingress failure; never repository NOOP."""
0047
0048
0049  def canonical(value):
0050      return json.dumps(value, ensure_ascii=False, sort_keys=True,
0051                        separators=(',', ':'), allow_nan=False).encode('utf-8')
0052
0053
0054  def digest(value):
0055      return hashlib.sha256(canonical(value)).hexdigest()
0056
0057
0058  def utc():
0059      return datetime.now(timezone.utc).isoformat().replace('+00:00', 'Z')
0060
0061
0062  def subject(root, repository, pr):

0063      """Observe one clean immutable checkout, without a floating tree lookup."""
0064      def git(*args):
0065          return subprocess.check_output(['git', '-C', str(root), *args],
0066              stderr=subprocess.DEVNULL, timeout=5, text=True).strip()
```

```
    try:
        head = git('rev-parse', '--verify', 'HEAD^{commit}')
        tree = git('rev-parse', '--verify', head + '^{tree}')
        if not HEX40.fullmatch(head) or not HEX40.fullmatch(tree):
            raise Hold('EXACT_SUBJECT_UNOBSERVABLE')
        git('diff', '--quiet', head, '--')
        if git('rev-parse', '--verify', 'HEAD^{commit}') != head:
            raise Hold('EXACT_SUBJECT_CHANGED')
    except (OSError, subprocess.SubprocessError) as exc:
        raise Hold('EXACT_SUBJECT_UNOBSERVABLE_OR_DIRTY') from exc
    if not re.fullmatch(r'[A-Za-z0-9_.-]+/[A-Za-z0-9_.-]+', repository):
        raise Hold('REPOSITORY_REQUIRED')
    if type(pr) is not int or pr < 1:
        raise Hold('PR_REQUIRED')
    return {'repository': repository, 'pr': pr, 'head': head, 'tree': tree}


class Ledger:
    """One process owns the durable store; append is a finite transaction."""
    def __init__(self, state_dir, exact_subject):
        self.subject = dict(exact_subject)
        self.binding = digest(self.subject)
        self.condition = threading.Condition(threading.RLock())
        self.stopped = False
        self.reason = ''
        self.listeners = set()
        self.directory = Path(state_dir) / 'temdd'
        if self.directory.is_symlink():
            raise Hold('UNSAFE_LEDGER_DIRECTORY')
        self.directory.mkdir(mode=0o700, parents=True, exist_ok=True)
        info = self.directory.stat()
        if info.st_uid != os.geteuid() or stat.S_IMODE(info.st_mode) & 0o077:
            raise Hold('LEDGER_DIRECTORY_MUST_BE_OWNER_ONLY')
        self.lock_fd = os.open(self.directory / 'owner.lock',
            os.O_CREAT | os.O_RDWR | os.O_NOFOLLOW, 0o600)
        try:
            fcntl.flock(self.lock_fd, fcntl.LOCK_EX | fcntl.LOCK_NB)
            for name in ('events.sqlite3', 'events.sqlite3-wal', 'events.sqlite3-shm'):
                if (self.directory / name).is_symlink():
                    raise Hold('UNSAFE_LEDGER_FILE')
            self.db = sqlite3.connect(self.directory / 'events.sqlite3',
                                      check_same_thread=False, timeout=5)
            self.db.execute('PRAGMA journal_mode=WAL')
            self.db.execute('PRAGMA synchronous=FULL')
            with self.db:
                self.db.execute('CREATE TABLE IF NOT EXISTS meta (key TEXT PRIMARY KEY, value TEXT NOT
NULL)')
                self.db.execute('CREATE TABLE IF NOT EXISTS events ('
                    'seq INTEGER PRIMARY KEY AUTOINCREMENT, binding TEXT NOT NULL, '
                    'source TEXT NOT NULL, native_id TEXT NOT NULL, native_digest TEXT NOT NULL, '
                    'body TEXT NOT NULL, body_digest TEXT NOT NULL, UNIQUE(source,native_id))')
                self.db.execute('CREATE INDEX IF NOT EXISTS events_binding ON events(binding,seq)')
                self.db.execute('INSERT OR IGNORE INTO meta VALUES (?,?)', ('schema', '1'))
                self.db.execute('INSERT OR IGNORE INTO meta VALUES (?,?)', ('epoch', uuid.uuid4().hex))
            meta = dict(self.db.execute('SELECT key,value FROM meta'))
            if meta.get('schema') != '1' or not re.fullmatch(r'[0-9a-f]{32}', meta.get('epoch', '')):
                raise Hold('UNKNOWN_LEDGER_SCHEMA')
            self.epoch = meta['epoch']

            if self.db.execute('PRAGMA quick_check').fetchone() != ('ok',):
                raise Hold('CORRUPT_LEDGER')
        except BaseException:
            if hasattr(self, 'db'):
                self.db.close()
            os.close(self.lock_fd)
            raise

    def _validate(self, event):
        required = {'schema', 'kind', 'subject', 'provenance', 'observed_at', 'message', 'payload'}
        if not isinstance(event, dict) or set(event) != required or event['schema'] != SCHEMA:
            raise Hold('INVALID_NATIVE_EVENT')
        if canonical(event['subject']) != canonical(self.subject):
            raise Hold('EXACT_SUBJECT_MISMATCH')
        if not isinstance(event['kind'], str) or event['kind'] not in KINDS:
            raise Hold('INVALID_EVENT_KIND')
        provenance = event['provenance']
        if not isinstance(provenance, dict) or set(provenance) != {'source', 'native_event_id'}:
            raise Hold('NATIVE_PROVENANCE_REQUIRED')
```

```
0143          if provenance['source'] not in ('repository', 'transputer'):
0144              raise Hold('UNADMITTED_PRODUCER')
0145          native_id = provenance['native_event_id']
0146          if not isinstance(native_id, str) or not re.fullmatch(r'[^\x00-\x20\x7f]{1,256}', native_id):
0147              raise Hold('INVALID_NATIVE_EVENT_ID')
0148          if not isinstance(event['message'], str) or len(event['message']) > 4096:
0149              raise Hold('INVALID_EVENT_MESSAGE')
0150          if not isinstance(event['payload'], dict):
0151              raise Hold('PAYLOAD_OBJECT_REQUIRED')
0152          observed = event['observed_at']
0153          try:
0154              if not isinstance(observed, str) or not observed.endswith('Z'):
0155                  raise ValueError()
0156              datetime.fromisoformat(observed[:-1] + '+00:00')
0157          except ValueError as exc:
0158              raise Hold('UTC_OBSERVATION_REQUIRED') from exc
0159          if len(canonical(event)) > MAX_EVENT:
0160              raise Hold('EVENT_TOO_LARGE')
0161
0162      def _decode(self, row):
0163          seq, binding, text, expected = row
0164          body = json.loads(text)
0165          if binding != self.binding or body.get('subject') != self.subject or digest(body) != expected:
0166              raise Hold('LEDGER_READBACK_MISMATCH')
0167          return dict(body, id=f'{self.epoch}:{seq}', ledger_digest=expected)
0168
0169      def append(self, event):
0170          # Freeze caller-owned objects before validation / hashing / persistence.
0171          event = json.loads(canonical(event))
0172          self._validate(event)
0173          native = digest(event)
0174          provenance = event['provenance']
0175          with self.condition:
0176              if self.stopped:
0177                  raise Hold(self.reason or 'LEDGER_STOPPED')
0178              old = self.db.execute('SELECT seq,native_digest FROM events WHERE source=? AND native_id=?',
0179                  (provenance['source'], provenance['native_event_id'])).fetchone()
0180              if old is not None:
0181                  if old[1] != native:
0182                      raise Hold('NATIVE_EVENT_ID_CONFLICT')
0183                  row = self.db.execute('SELECT seq,binding,body,body_digest FROM events WHERE seq=?',
      (old[0],)).fetchone()
0184                  return self._decode(row)

0185              body = dict(event, schema='qikvrt_temdd_event_v1', recorded_at=utc(),
0186                          payload_digest=digest(event['payload']), evidence_transfer='DENY', dod=False)
0187              text = canonical(body).decode('utf-8')
0188              with self.db:
0189                  result = self.db.execute('INSERT INTO
      events(binding,source,native_id,native_digest,body,body_digest) '
0190                      'VALUES (?,?,?,?,?,?)', (self.binding, provenance['source'],
      provenance['native_event_id'], native, text, digest(body)))
0191                  seq = result.lastrowid
0192              # No publish before COMMIT and exact durable readback.
0193              row = self.db.execute('SELECT seq,binding,body,body_digest FROM events WHERE seq=?',
      (seq,)).fetchone()
0194              readback = self._decode(row)
0195              self._notify()
0196              return readback
0197
0198      def cursor(self, value):
0199          if not value:
0200              return 0
0201          if not isinstance(value, str):
0202              raise Hold('INVALID_REPLAY_CURSOR')
0203          match = CURSOR.fullmatch(value)
0204          if not match or match[1] != self.epoch or int(match[2]) > 9223372036854775807:
0205              raise Hold('INVALID_REPLAY_CURSOR')
0206          seq = int(match[2])
0207          with self.condition:
0208              row = self.db.execute('SELECT seq,binding,body,body_digest FROM events WHERE seq=?',
      (seq,)).fetchone()
0209              if row is None or row[1] != self.binding:
0210                  raise Hold('STALE_OR_UNKNOWN_REPLAY_CURSOR')
0211              self._decode(row)
0212          return seq
0213
0214      def replay(self, after):
```

```
        with self.condition:
            rows = self.db.execute('SELECT seq,binding,body,body_digest FROM events '
                'WHERE binding=? AND seq>? ORDER BY seq LIMIT 128', (self.binding, after)).fetchall()
            return [self._decode(row) for row in rows]

    def _notify(self):
        self.condition.notify_all()
        for writer in tuple(self.listeners):
            with contextlib.suppress(BlockingIOError, OSError):
                writer.send(b'1')  # coalesced, nonblocking, at most 32 observers

    def subscribe(self):
        with self.condition:
            reader, writer = socket.socketpair()
            reader.setblocking(False)
            writer.setblocking(False)
            self.listeners.add(writer)
            writer.send(b'1')  # closes the replay/subscribe race
            return reader, writer

    def unsubscribe(self, reader, writer):
        with self.condition:
            self.listeners.discard(writer)
            reader.close()
            writer.close()
            self.condition.notify_all()

    def stop(self, reason='PRODUCER_STOPPED'):

        with self.condition:
            self.stopped = True
            self.reason = reason
            self._notify()

    def close(self):
        self.stop()
        with self.condition:
            self.db.close()
            os.close(self.lock_fd)


class Runtime:
    def __init__(self, root, state_dir, repository, pr):
        self.root = Path(root).resolve()
        self.repository, self.pr = repository, pr
        self.subject = subject(self.root, repository, pr)
        self.ledger = Ledger(state_dir, self.subject)
        self.subscribers = threading.BoundedSemaphore(32)

    def ensure_subject(self):
        try:
            if subject(self.root, self.repository, self.pr) != self.subject:
                raise Hold('EXACT_SUBJECT_CHANGED_RESTART_REQUIRED')
            if self.ledger.stopped:
                raise Hold(self.ledger.reason)
        except Hold as exc:
            self.ledger.stop(str(exc))
            raise
        return self.subject

    def append(self, event):
        self.ensure_subject()
        return self.ledger.append(event)


def make_handler(runtime):
    class Handler(terminal.Handler):
        protocol_version = 'HTTP/1.1'

        def _hold(self, reason, code=503):
            self._json(code, {'state': 'HOLD', 'reason': reason, 'dod': False,
                              'ordinary_release': False, 'evidence_transfer': 'DENY'})

        def _frame(self, value, name=None, event_id=None):
            frame = (f'event: {name}\n' if name else '')
            frame += (f'id: {event_id}\n' if event_id else '')
            self.wfile.write((frame + 'data: ' + canonical(value).decode('utf-8') +
    '\n\n').encode('utf-8'))
```

```
0291              self.wfile.flush()
0292
0293          def do_GET(self):
0294              path = urlsplit(self.path).path
0295              if path not in ('/AI', '/AI/', '/api/temdd/subject', '/api/temdd/events'):
0296                  return super().do_GET()
0297              host = self.headers.get('Host', '')
0298              allowed_hosts = {f'127.0.0.1:{self.server.server_port}',
      f'localhost:{self.server.server_port}'}
0299              if host not in allowed_hosts:
0300                  return self._hold('LOOPBACK_HOST_REQUIRED', 403)
0301              if self.headers.get('Origin') not in (None, 'http://' + host):
0302                  return self._hold('SAME_ORIGIN_REQUIRED', 403)

0303              try:
0304                  exact = runtime.ensure_subject()
0305                  if path in ('/AI', '/AI/'):
0306                      data = (runtime.root / 'docs/terminal/temdd/index.html').read_bytes()
0307                      self.send_response(200)
0308                      self.send_header('Content-Type', 'text/html; charset=utf-8')
0309                      self.send_header('Content-Length', str(len(data)))
0310                      self.send_header('Cache-Control', 'no-store')
0311                      self.send_header('X-Content-Type-Options', 'nosniff')
0312                      self.end_headers()
0313                      self.wfile.write(data)
0314                      return
0315                  if path == '/api/temdd/subject':
0316                      return self._json(200, {'subject': exact, 'ledger_id': runtime.ledger.epoch,
0317                          'state': 'HOLD_UNVERIFIED', 'dod': False, 'evidence_transfer': 'DENY'})
0318                  query = parse_qs(urlsplit(self.path).query, keep_blank_values=True)
0319                  if set(query) - {'after'} or len(query.get('after', [''])) != 1:
0320                      raise Hold('INVALID_REPLAY_QUERY')
0321                  # Native reconnect headers supersede the original URL's cursor.
0322                  after = runtime.ledger.cursor(self.headers.get('Last-Event-ID') or query.get('after',
      [''])[0])
0323                  if not runtime.subscribers.acquire(blocking=False):
0324                      return self._hold('SUBSCRIBER_LIMIT')
0325              except (ValueError, OSError, sqlite3.Error) as exc:
0326                  return self._hold(str(exc), 409 if isinstance(exc, Hold) else 503)
0327              reader, writer = runtime.ledger.subscribe()
0328              try:
0329                  self.connection.settimeout(5)  # bounded writes, not a polling timer
0330                  self.send_response(200)
0331                  self.send_header('Content-Type', 'text/event-stream; charset=utf-8')
0332                  self.send_header('Cache-Control', 'no-store')
0333                  self.send_header('X-Accel-Buffering', 'no')
0334                  self.send_header('X-Content-Type-Options', 'nosniff')
0335                  self.end_headers()
0336                  self._frame({'subject': exact, 'ledger_id': runtime.ledger.epoch,
0337                      'dod': False, 'evidence_transfer': 'DENY'}, name='subject')
0338                  with selectors.DefaultSelector() as ready:
0339                      ready.register(reader, selectors.EVENT_READ)
0340                      ready.register(self.connection, selectors.EVENT_READ)
0341                      while True:  # outer I/O adapter; no timed status queries
0342                          for key, _ in ready.select():
0343                              if key.fileobj is self.connection:
0344                                  return  # disconnect or unsupported pipelined input
0345                          with contextlib.suppress(BlockingIOError):
0346                              reader.recv(65536)
0347                          while True:
0348                              with runtime.ledger.condition:
0349                                  if runtime.ledger.stopped:
0350                                      self._frame({'state': 'HOLD', 'reason': runtime.ledger.reason, 'dod':
       False}, name='hold')
0351                                      return
0352                                  batch = runtime.ledger.replay(after)
0353                              runtime.ensure_subject()
0354                              for event in batch:
0355                                  self._frame(event, event_id=event['id'])
0356                                  after = int(event['id'].split(':')[1])
0357                              if len(batch) < 128:
0358                                  break
0359              except (OSError, ValueError, sqlite3.Error):
0360                  # Observer disconnect never changes repository execution state.
0361                  with contextlib.suppress(OSError):
0362                      self._frame({'state': 'HOLD', 'reason': 'STREAM_READBACK_UNAVAILABLE', 'dod': False},

       name='hold')
0363              finally:
```

```
                self.close_connection = True
                runtime.ledger.unsubscribe(reader, writer)
                runtime.subscribers.release()

        def do_POST(self):
            if urlsplit(self.path).path.startswith('/api/temdd/'):
                return self._hold('NATIVE_UNIX_INGRESS_REQUIRED', 405)
            return super().do_POST()
    return Handler


def make_ingress(runtime):
    class Ingress(socketserver.StreamRequestHandler):
        def handle(self):
            self.connection.settimeout(5)
            try:
                raw = self.rfile.readline(MAX_EVENT + 2)
                if len(raw) > MAX_EVENT + 1 or not raw.endswith(b'\n'):
                    raise Hold('BOUNDED_EVENT_LINE_REQUIRED')
                event = runtime.append(json.loads(raw))
                result = {'state': 'PERSISTED', 'event': event, 'authority_effect': False, 'dod': False}
            except (OSError, ValueError, sqlite3.Error, RecursionError) as exc:
                result = {'state': 'HOLD', 'reason': str(exc), 'dod': False}
            with contextlib.suppress(OSError):
                self.wfile.write(canonical(result) + b'\n')
    path = runtime.ledger.directory / 'ingress.sock'
    if path.exists() or path.is_symlink():
        if path.is_symlink() or not stat.S_ISSOCK(path.stat().st_mode):
            raise Hold('UNSAFE_INGRESS_PATH')
        path.unlink()  # store-owner lock excludes a live competing producer
    server = socketserver.UnixStreamServer(str(path), Ingress)
    os.chmod(path, 0o600)
    return server


def submit(state_dir, event):
    raw = canonical(event)
    if len(raw) > MAX_EVENT:
        raise Hold('EVENT_TOO_LARGE')
    with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as client:
        client.settimeout(10)
        client.connect(str(Path(state_dir) / 'temdd/ingress.sock'))
        client.sendall(raw + b'\n')
        with client.makefile('rb') as response:
            result = json.loads(response.readline(MAX_EVENT * 2))
    if result.get('state') != 'PERSISTED':
        raise Hold(result.get('reason', 'LEDGER_READBACK_UNAVAILABLE'))
    return result


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('operation', choices=('serve', 'append'))
    parser.add_argument('--state-dir', required=True)
    parser.add_argument('--host', default='127.0.0.1')
    parser.add_argument('--port', type=int, default=8771)
    parser.add_argument('--root', default=str(Path(__file__).resolve().parents[1]))
    parser.add_argument('--repository', default='Goldkelch/qik-vrt')
    parser.add_argument('--pr', type=int, default=1103)
    args = parser.parse_args()

    try:
        if args.operation == 'append':
            raw = sys.stdin.buffer.read(MAX_EVENT + 1)
            if len(raw) > MAX_EVENT:
                raise Hold('EVENT_TOO_LARGE')
            print(canonical(submit(args.state_dir, json.loads(raw))).decode('utf-8'))
            return 0
        if args.host != '127.0.0.1':
            raise Hold('LOOPBACK_ONLY')
        runtime = Runtime(args.root, args.state_dir, args.repository, args.pr)
        http = terminal.ThreadingHTTPServer((args.host, args.port), make_handler(runtime))
        ingress = make_ingress(runtime)
        thread = threading.Thread(target=ingress.serve_forever, daemon=True)
        thread.start()  # outer I/O adapter, not a repository job scheduler
        try:
            runtime.append({'schema': SCHEMA, 'kind': 'OBSERVE', 'subject': runtime.subject,
                'provenance': {'source': 'transputer', 'native_event_id': 'listener-bound:' +
```

```
      uuid.uuid4().hex},
0441                 'observed_at': utc(), 'message': 'Local TEMDD listeners bound; no DoD inference',
0442                 'payload': {'pid': os.getpid(), 'http_address': list(http.server_address)}})
0443             print(canonical({'state': 'LISTENING', 'subject': runtime.subject, 'dod': False}).decode(),
      flush=True)
0444             http.serve_forever()
0445         finally:
0446             runtime.ledger.stop()
0447             ingress.shutdown()
0448             ingress.server_close()
0449             http.server_close()
0450             thread.join(timeout=6)
0451             runtime.ledger.close()
0452         return 0
0453     except (OSError, ValueError, sqlite3.Error, RecursionError) as exc:
0454         print(canonical({'state': 'HOLD', 'reason': str(exc), 'dod': False}).decode(), file=sys.stderr)
0455         return 2
0456
0457
0458 if __name__ == '__main__':
0459     raise SystemExit(main())
```

B2 - src/qikvrt_effect_ack_http_terminal.py

Archivkennung F098; Ursprungspfad src/qikvrt_effect_ack_http_terminal.py

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 30ff5f58318a54f06efb86f437dea1f7239c9c00; SHA-256 039b7c34fe4ae8766ae0b141734ef91131377e5e58fedb7746320242543da2d8; 30823 Byte

```
0001  #!/usr/bin/env python3
0002  # SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0003  """Loopback-only reference backend for the QIKVRT Firefox terminal.
0004
0005  Experimental HTTP-profile demonstrator. It proves capability discovery,
0006  Structured-Field prepare/commit, exact-bound single-use commit, and post-effect
0007  reobservation without granting repository, publication, deployment, or other
0008  external-effect capability.
0009
0010  The separate durable-* CLI operations are owner-local clients of the existing
0011  TEMDD Unix ingress. They add no HTTP writer and never infer product completion.
0012  """
0013  from __future__ import annotations
0014
0015  import argparse
0016  import base64
0017  import contextlib
0018  import hashlib
0019  import hmac
0020  import json
0021  import os
0022  from pathlib import Path
0023  import re
0024  import secrets
0025  import socket
0026  import sqlite3
0027  import stat
0028  import struct
0029  import subprocess
0030  import threading
0031  import time
0032  from dataclasses import dataclass
0033  from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
0034  from typing import Any
0035
0036  MAX_BODY = 2 * 1024 * 1024
0037  TOKEN_TTL_SECONDS = 120
0038  HOST = "127.0.0.1"
0039  DEFAULT_PORT = 8771
0040  SF_KEY = re.compile(r"^[a-z*][a-z0-9_.*-]*$")
0041  MAX_NATIVE_EVENT = 65536
0042  NATIVE_SCHEMA = "qikvrt_temdd_native_event_v1"
0043
0044
0045  def canonical_json(value: Any) -> bytes:
0046      return json.dumps(value, ensure_ascii=False, sort_keys=True, separators=(",", ":"),
      allow_nan=False).encode("utf-8")
0047
0048
0049  def sha256(value: bytes) -> str:
0050      return hashlib.sha256(value).hexdigest()
0051
0052
0053  def sf_bytes(raw: bytes) -> str:
0054      return ":" + base64.b64encode(raw).decode("ascii") + ":"
0055
0056
0057  def parse_sf_bytes(value: str) -> bytes:
0058      if len(value) < 2 or not (value.startswith(":") and value.endswith(":")):
0059          raise ValueError("Structured Field Byte Sequence required")
0060      try:
0061          return base64.b64decode(value[1:-1], validate=True)
0062      except (ValueError, base64.binascii.Error) as exc:
0063          raise ValueError("malformed Structured Field Byte Sequence") from exc
0064
0065
```

```
def parse_effect_ack_request(raw: str | None) -> dict[str, Any]:
    if raw is None or not raw.strip():
        raise ValueError("Effect-Ack-Request required")
    members: dict[str, str] = {}
    for item in raw.split(","):
        if "=" not in item:
            raise ValueError("malformed Effect-Ack-Request dictionary")
        key, value = item.split("=", 1)
        key = key.strip().lower()
        value = value.strip()
        if not SF_KEY.fullmatch(key) or key in members:
            raise ValueError("invalid or duplicate Effect-Ack-Request member")
        members[key] = value
    if set(members) - {"v", "mode", "token", "hash"}:
        raise ValueError("unknown Effect-Ack-Request member in version 1")
    try:
        version = int(members.get("v", ""))
    except ValueError as exc:
        raise ValueError("Effect-Ack-Request v must be an integer") from exc
    if version != 1:
        raise ValueError("unsupported Effect-Ack-Request version")
    mode = members.get("mode")
    if mode not in {"prepare", "commit"}:
        raise ValueError("Effect-Ack-Request mode must be prepare or commit")
    if mode == "prepare":
        if set(members) != {"v", "mode"}:
            raise ValueError("prepare must not carry token or hash")
        return {"v": 1, "mode": "prepare"}
    if set(members) != {"v", "mode", "token", "hash"}:
        raise ValueError("commit requires token and hash")
    token_bytes = parse_sf_bytes(members["token"])
    try:
        token = token_bytes.decode("ascii")
    except UnicodeDecodeError as exc:
        raise ValueError("commit token must be ASCII") from exc
    hash_bytes = parse_sf_bytes(members["hash"])
    if len(hash_bytes) != 32:
        raise ValueError("commit hash must contain exactly 32 octets")
    return {"v": 1, "mode": "commit", "token": token, "hash": hash_bytes.hex()}


def git_read(*args: str) -> str | None:
    try:
        return subprocess.check_output(["git", *args], text=True, stderr=subprocess.DEVNULL,
timeout=2).strip()
    except (OSError, subprocess.SubprocessError):
        return None


class DurableHold(ValueError):
    """The owner-local persistence effect is not verified."""


def durable_subject(root: str, repository: str, pr: int, head: str, tree: str,
                    manifest_pin: str | None = None) -> dict[str, Any]:
    """Require a clean, exact checkout; never relabel the running carrier."""
    if not re.fullmatch(r"[A-Za-z0-9_.-]+/[A-Za-z0-9_.-]+", repository):
        raise DurableHold("REPOSITORY_REQUIRED")
    if type(pr) is not int or pr < 1 or not all(re.fullmatch(r"[0-9a-f]{40}", x or "") for x in (head,
tree)):
        raise DurableHold("EXACT_SUBJECT_REQUIRED")
    if manifest_pin is not None:
        # Explicit sealed-export path. It neither probes Git nor changes the
        # existing clean-checkout path when no independent pin is supplied.
        import importlib.util
        spec = importlib.util.spec_from_file_location("qikvrt_owner_self_host", Path(root) /
"tools/qikvrt_self_host.py")
        adapter = importlib.util.module_from_spec(spec)
        spec.loader.exec_module(adapter)
        return adapter.packaged_subject(Path(root), manifest_pin, repository, pr, head, tree)
    def git(*args: str) -> str:
        return subprocess.check_output(["git", "-C", root, *args], text=True,
                                       stderr=subprocess.DEVNULL, timeout=5).strip()
    try:
        if git("rev-parse", "HEAD^{commit}") != head or git("rev-parse", head + "^{tree}") != tree:
            raise DurableHold("EXACT_SUBJECT_MISMATCH")
        if git("remote", "get-url", "origin") not in {f"https://github.com/{repository}",
```

```
    f"https://github.com/{repository}.git"}:
0140             raise DurableHold("CHECKOUT_REPOSITORY_MISMATCH")
0141         git("diff", "--quiet", head, "--")
0142         if git("rev-parse", "HEAD^{commit}") != head:
0143             raise DurableHold("EXACT_SUBJECT_CHANGED")
0144     except (OSError, subprocess.SubprocessError) as exc:
0145         raise DurableHold("EXACT_SUBJECT_UNOBSERVABLE_OR_DIRTY") from exc
0146     return {"repository": repository, "pr": pr, "head": head, "tree": tree}
0147
0148
0149 def _durable_directory(state_dir: str) -> Path:
0150     path = Path(state_dir)
0151     if ".." in path.parts:
0152         raise DurableHold("UNSAFE_STATE_PATH")
0153     directory = path.absolute() / "temdd"
0154     for component in (*reversed(directory.parents), directory):
0155         if component.is_symlink():
0156             raise DurableHold("SYMLINK_STATE_PATH")
0157     info = directory.lstat()
0158     if not stat.S_ISDIR(info.st_mode) or info.st_uid != os.geteuid() or stat.S_IMODE(info.st_mode) &
    0o077:
0159         raise DurableHold("LEDGER_DIRECTORY_MUST_BE_OWNER_ONLY")
0160     return directory
0161
0162
0163 def _durable_file(directory: Path, name: str, *, is_socket: bool = False) -> Path:
0164     path = directory / name
0165     info = path.lstat()
0166     valid_type = stat.S_ISSOCK(info.st_mode) if is_socket else stat.S_ISREG(info.st_mode)
0167     if not valid_type or info.st_uid != os.geteuid() or info.st_mode & 0o022:
0168         raise DurableHold("UNSAFE_DURABLE_PATH")
0169     if is_socket and stat.S_IMODE(info.st_mode) != 0o600:
0170         raise DurableHold("INGRESS_MUST_REMAIN_OWNER_ONLY_0600")
0171     return path
0172
0173
0174 def _durable_read(state_dir: str, event: dict[str, Any] | None = None) -> tuple[str, dict[str, Any] |
    None]:
0175     """Fresh read-only SQLite connection to the existing ledger, never a writer."""
0176     directory = _durable_directory(state_dir)
0177     database = _durable_file(directory, "events.sqlite3")
0178     for name in ("events.sqlite3-wal", "events.sqlite3-shm"):
0179         if (directory / name).exists() or (directory / name).is_symlink():

0180             _durable_file(directory, name)
0181     with contextlib.closing(sqlite3.connect(database.as_uri() + "?mode=ro", uri=True, timeout=5)) as db:
0182         db.execute("PRAGMA query_only=ON")
0183         db.execute("BEGIN")
0184         meta = dict(db.execute("SELECT key,value FROM meta"))
0185         epoch = meta.get("epoch", "")
0186         if meta.get("schema") != "1" or not re.fullmatch(r"[0-9a-f]{32}", epoch):
0187             raise DurableHold("UNKNOWN_LEDGER_SCHEMA")
0188         if event is None:
0189             return epoch, None
0190         provenance = event["provenance"]
0191         row = db.execute("SELECT seq,binding,native_digest,body,body_digest FROM events WHERE source=?
    AND native_id=?",
0192                          (provenance["source"], provenance["native_event_id"])).fetchone()
0193         if row is None:
0194             return epoch, None
0195         seq, binding, native_digest, text, body_digest = row
0196         body = json.loads(text)
0197         if not isinstance(body, dict):
0198             raise DurableHold("DURABLE_BODY_OBJECT_REQUIRED")
0199         expected = dict(event, schema="qikvrt_temdd_event_v1", recorded_at=body.get("recorded_at"),
0200                         payload_digest=sha256(canonical_json(event["payload"])),
    evidence_transfer="DENY", dod=False)
0201         if (type(seq) is not int or seq < 1 or binding != sha256(canonical_json(event["subject"]))
0202                 or native_digest != sha256(canonical_json(event)) or canonical_json(body) !=
    canonical_json(expected)
0203                 or not isinstance(body.get("recorded_at"), str)
0204                 or body_digest != sha256(canonical_json(body))):
0205             raise DurableHold("DURABLE_READBACK_MISMATCH")
0206         return epoch, dict(body, id=f"{epoch}:{seq}", ledger_digest=body_digest)
0207
0208
0209 def _durable_input(body: dict[str, Any]) -> dict[str, Any]:
0210     body = json.loads(canonical_json(body))  # Freeze caller-owned objects.
```

```
    if (not isinstance(body, dict) or body.get("schema") != "qikvrt_terminal_input_v1"
            or set(body) - {"schema", "text", "audio", "video", "page", "submitted_at"}
            or not isinstance(body.get("text"), str) or not 1 <= len(body["text"]) <= 4096
            or body.get("audio") is not None or body.get("video") is not None
            or len(canonical_json(body)) > MAX_NATIVE_EVENT // 2):
        raise DurableHold("BOUNDED_TEXT_INPUT_REQUIRED_MEDIA_NOT_ADMITTED")
    return body


def durable_prepare(body: dict[str, Any], subject: dict[str, Any], state_dir: str) -> dict[str, Any]:
    """Prepare an owner-local observation; insert no event and expose no HTTP route."""
    body = _durable_input(body)
    subject = json.loads(canonical_json(subject))
    if (set(subject) != {"repository", "pr", "head", "tree"} or type(subject["pr"]) is not int or
subject["pr"] < 1
            or not re.fullmatch(r"[A-Za-z0-9_.-]+/[A-Za-z0-9_.-]+", subject["repository"])
            or not all(re.fullmatch(r"[0-9a-f]{40}", subject[x]) for x in ("head", "tree"))):
        raise DurableHold("EXACT_SUBJECT_REQUIRED")
    directory = _durable_directory(state_dir)
    _durable_file(directory, "ingress.sock", is_socket=True)
    epoch, _ = _durable_read(state_dir)
    input_hash = "sha256:" + sha256(canonical_json(body))
    event = {"schema": NATIVE_SCHEMA, "kind": "OBSERVE", "subject": subject,
             "provenance": {"source": "transputer", "native_event_id": "terminal:" +
secrets.token_hex(24)},
             "observed_at": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
             "message": "Universal Terminal: hash-bound durable input observation",
             "payload": {"adapter": "QIKVRT_OWNER_UNIX_TERMINAL_V1", "terminal_input": body,

                         "input_hash": input_hash, "effect_ack_done": False}}
    plan = {"schema": "qikvrt_owner_unix_terminal_preparation_v1", "subject": subject, "event": event,
            "input_hash": input_hash, "ledger_id": epoch, "state_directory": str(directory.parent),
            "expires_at": int(time.time()) + TOKEN_TTL_SECONDS}
    return {"state": "PREPARED", "preparation": plan, "prepare_hash": "sha256:" +
sha256(canonical_json(plan)),
            "ordinary_release": False, "EFFECT_ACK_DONE": False}


def _durable_plan(prepared: dict[str, Any], expected_hash: str, body: dict[str, Any],
                  subject: dict[str, Any], state_dir: str) -> dict[str, Any]:
    if (set(prepared) != {"state", "preparation", "prepare_hash", "ordinary_release", "EFFECT_ACK_DONE"}
            or prepared["state"] != "PREPARED" or prepared["ordinary_release"] is not False
            or prepared["EFFECT_ACK_DONE"] is not False):
        raise DurableHold("INVALID_PREPARATION")
    plan = prepared["preparation"]
    digest = "sha256:" + sha256(canonical_json(plan))
    if not hmac.compare_digest(digest, expected_hash) or prepared["prepare_hash"] != digest:
        raise DurableHold("PREPARE_HASH_MISMATCH")
    if (set(plan) != {"schema", "subject", "event", "input_hash", "ledger_id", "state_directory",
"expires_at"}
            or plan["schema"] != "qikvrt_owner_unix_terminal_preparation_v1"
            or canonical_json(plan["subject"]) != canonical_json(subject)
            or plan["state_directory"] != str(_durable_directory(state_dir).parent)):
        raise DurableHold("PREPARED_SUBJECT_OR_ROUTE_MISMATCH")
    frozen = _durable_input(body)
    event = plan["event"]
    if (plan["input_hash"] != "sha256:" + sha256(canonical_json(frozen))
            or event.get("subject") != subject or event.get("schema") != NATIVE_SCHEMA or
event.get("kind") != "OBSERVE"
            or event.get("payload") != {"adapter": "QIKVRT_OWNER_UNIX_TERMINAL_V1", "terminal_input":
frozen,
                                        "input_hash": plan["input_hash"], "effect_ack_done": False}
            or event.get("provenance", {}).get("source") != "transputer"
            or not re.fullmatch(r"terminal:[0-9a-f]{48}", event.get("provenance",
{}).get("native_event_id", ""))
            or len(canonical_json(event)) > MAX_NATIVE_EVENT):
        raise DurableHold("PREPARED_INPUT_MISMATCH")
    return plan


def durable_readback(prepared: dict[str, Any], expected_hash: str, body: dict[str, Any],
                     subject: dict[str, Any], state_dir: str) -> dict[str, Any]:
    """May recover an ambiguous commit after expiry/restart, without resubmission."""
    plan = _durable_plan(prepared, expected_hash, body, subject, state_dir)
    epoch, event = _durable_read(state_dir, plan["event"])
    if epoch != plan["ledger_id"] or event is None:
        raise DurableHold("DURABLE_EVENT_NOT_OBSERVED_IN_PREPARED_LEDGER")
    return {"state": "EFFECT_ACK_CONTINUE", "durable_persisted": True, "durable_readback": event,
```

```
            "prepare_hash": expected_hash, "ordinary_release": False, "EFFECT_ACK_DONE": False,
            "public_readback_verified": False, "authority_effect": False}


def durable_commit(prepared: dict[str, Any], expected_hash: str, body: dict[str, Any],
                   subject: dict[str, Any], state_dir: str) -> dict[str, Any]:
    plan = _durable_plan(prepared, expected_hash, body, subject, state_dir)
    if type(plan["expires_at"]) is not int or plan["expires_at"] < time.time():
        raise DurableHold("EXPIRED_PREPARATION")
    epoch, old = _durable_read(state_dir, plan["event"])
    if epoch != plan["ledger_id"]:
        raise DurableHold("PREPARED_LEDGER_CHANGED")
    if old is not None:
        raise DurableHold("PREPARATION_ALREADY_COMMITTED_USE_READBACK")
    path = _durable_file(_durable_directory(state_dir), "ingress.sock", is_socket=True)
    if not hasattr(socket, "SO_PEERCRED"):
        raise DurableHold("UNIX_PEER_UID_VERIFICATION_REQUIRED")
    with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as client:
        client.settimeout(5)
        client.connect(str(path))
        _, uid, _ = struct.unpack("3i", client.getsockopt(socket.SOL_SOCKET, socket.SO_PEERCRED,
struct.calcsize("3i")))
        if uid != os.geteuid():
            raise DurableHold("INGRESS_PEER_OWNER_MISMATCH")
        client.sendall(canonical_json(plan["event"]) + b"\n")
        with client.makefile("rb") as response:
            raw = response.readline(MAX_NATIVE_EVENT * 2 + 2)
        if len(raw) > MAX_NATIVE_EVENT * 2 + 1 or not raw.endswith(b"\n"):
            raise DurableHold("AMBIGUOUS_COMMIT_USE_READBACK_DO_NOT_RESUBMIT")
        reply = json.loads(raw)
    if not isinstance(reply, dict):
        raise DurableHold("INGRESS_REPLY_OBJECT_REQUIRED_USE_READBACK")
    if reply.get("state") != "PERSISTED" or reply.get("dod") is not False or
reply.get("authority_effect") is not False:
        raise DurableHold("INGRESS_DID_NOT_QUIT_DURABLE_EFFECT")
    receipt = durable_readback(prepared, expected_hash, body, subject, state_dir)
    if canonical_json(reply.get("event")) != canonical_json(receipt["durable_readback"]):
        raise DurableHold("INGRESS_REPLY_DIFFERS_FROM_FRESH_DURABLE_READBACK")
    return receipt


@dataclass
class Prepared:
    token: str
    input_hash: str
    record_hash: str
    expires_at: float
    used: bool = False


class State:
    def __init__(self) -> None:
        self.secret = secrets.token_bytes(32)
        self.lock = threading.Lock()
        self.prepared: dict[str, Prepared] = {}
        self.records: dict[str, dict[str, Any]] = {}
        self.events: list[dict[str, Any]] = []

    def record(self, *, state: str, input_hash: str, ordinary_release: bool, reason: str) -> tuple[str,
dict[str, Any]]:
        body = {
            "schema": "qikvrt_effect_ack_http_terminal_record_v1",
            "wire_version": 1,
            "message_type": "effect-ack-record",
            "state": state,
            "input_hash": "sha256:" + input_hash,
            "policy_id": "QIKVRT_LOOPBACK_TERMINAL_V1",
            "policy_version": 1,
            "policy_allows_release": ordinary_release,
            "ordinary_release": ordinary_release,
            "responsibility_owner": "LOCAL_INTERACTIVE_USER",
            "reason": reason,
            "created_utc": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
            "external_effect": "NONE",
        }

        digest = sha256(canonical_json(body))
        body["record_hash"] = "sha256:" + digest
```

```
0355          self.records[digest] = body
0356          return digest, body
0357
0358      def make_token(self, input_hash: str, record_hash: str) -> str:
0359          nonce = secrets.token_bytes(24)
0360          expires = int(time.time()) + TOKEN_TTL_SECONDS
0361          payload = nonce + expires.to_bytes(8, "big") + bytes.fromhex(input_hash) +
      bytes.fromhex(record_hash)
0362          mac = hmac.new(self.secret, payload, hashlib.sha256).digest()
0363          token = base64.urlsafe_b64encode(payload + mac).decode("ascii").rstrip("=")
0364          self.prepared[token] = Prepared(token, input_hash, record_hash, float(expires))
0365          return token
0366
0367
0368  STATE = State()
0369
0370
0371  class Handler(BaseHTTPRequestHandler):
0372      server_version = "QIKVRTEffectAckTerminal/1.0"
0373
0374      def _authenticated(self) -> bool:
0375          # The standalone launcher sets this boundary; legacy loopback tests
0376          # retain their original scope. No token is emitted in state or receipts.
0377          token = getattr(self.server, "terminal_auth_token", None)
0378          if token is None:
0379              return True
0380          if hmac.compare_digest(self.headers.get("Authorization", "").encode(), ("Bearer " +
      token).encode()):
0381              return True
0382          self._json(401, {"state": "HOLD", "reason": "local terminal authentication required",
      "ordinary_release": False})
0383          return False
0384
0385      def _json(
0386          self,
0387          code: int,
0388          body: dict[str, Any],
0389          *,
0390          state: str | None = None,
0391          record_hash: str | None = None,
0392          commit_token: str | None = None,
0393      ) -> None:
0394          payload = json.dumps(body, ensure_ascii=False, sort_keys=True, indent=2).encode("utf-8")
0395          self.send_response(code)
0396          self.send_header("Content-Type", "application/json; charset=utf-8")
0397          self.send_header("Content-Length", str(len(payload)))
0398          self.send_header("Cache-Control", "no-store")
0399          self.send_header("X-Content-Type-Options", "nosniff")
0400          self.send_header("Access-Control-Allow-Origin", "https://github.com")
0401          self.send_header("Access-Control-Expose-Headers", "Effect-Ack, Link")
0402          self.send_header("Link", "</.well-known/effect-ack>; rel=\"effect-ack\";
      type=\"application/json\"")
0403          if state and record_hash:
0404              state_token = {
0405                  "EFFECT_NACK": "nack",
0406                  "EFFECT_ACK_CONTINUE": "continue",
0407                  "EFFECT_ACK_DONE": "done",
0408                  "EFFECT_ACK_ISOLATE": "isolate",
0409                  "EFFECT_ACK_BLOCK": "block",
0410              }[state]

0411              value = f"v=1, state={state_token}, hash={sf_bytes(bytes.fromhex(record_hash))}"
0412              if commit_token is not None:
0413                  value += f", token={sf_bytes(commit_token.encode('ascii'))}"
0414              self.send_header("Effect-Ack", value)
0415          self.end_headers()
0416          self.wfile.write(payload)
0417
0418      def _read_body(self) -> dict[str, Any]:
0419          if self.headers.get("Transfer-Encoding"):
0420              raise ValueError("Transfer-Encoding unsupported")
0421          raw_length = self.headers.get("Content-Length")
0422          if raw_length is None:
0423              raise ValueError("Content-Length required")
0424          length = int(raw_length)
0425          if length < 1 or length > MAX_BODY:
0426              raise ValueError("body outside bounded size")
0427          if self.headers.get("Content-Type", "").split(";", 1)[0].strip().lower() != "application/json":
```

```
0428              raise ValueError("application/json required")
0429          value = json.loads(self.rfile.read(length).decode("utf-8"))
0430          if not isinstance(value, dict):
0431              raise ValueError("JSON object required")
0432          return value
0433
0434      def do_OPTIONS(self) -> None:
0435          self.send_response(204)
0436          self.send_header("Access-Control-Allow-Origin", "https://github.com")
0437          self.send_header("Access-Control-Allow-Headers", "Content-Type, Effect-Ack-Request")
0438          self.send_header("Access-Control-Allow-Methods", "GET, POST, OPTIONS")
0439          self.send_header("Content-Length", "0")
0440          self.end_headers()
0441
0442      def do_GET(self) -> None:
0443          if not self._authenticated():
0444              return
0445          if self.path == "/.well-known/effect-ack":
0446              self._json(200, {
0447                  "schema": "qikvrt_effect_ack_http_capability_v1",
0448                  "versions": [1],
0449                  "modes": ["prepare", "commit"],
0450                  "protected_effects": ["terminal_input"],
0451                  "external_effects": "NONE",
0452                  "record_template": "/effect-ack/records/{sha256}",
0453              })
0454              return
0455          if self.path == "/terminal/state":
0456              head = git_read("rev-parse", "HEAD")
0457              tree = git_read("rev-parse", "HEAD^{tree}")
0458              with STATE.lock:
0459                  body = {
0460                      "schema": "qikvrt_terminal_backend_state_v1",
0461                      "events": len(STATE.events),
0462                      "last_event": STATE.events[-1] if STATE.events else None,
0463                      "repository_head": head,
0464                      "repository_tree": tree,
0465                      "external_effects": "NONE",
0466                      "runtime_binding": getattr(self.server, "runtime_binding", None),
0467                  }
0468              self._json(200, body)
0469              return
0470          prefix = "/effect-ack/records/"
0471          if self.path.startswith(prefix):
0472              digest = self.path[len(prefix):]

0473              with STATE.lock:
0474                  body = STATE.records.get(digest)
0475              if body is None:
0476                  self._json(404, {"state": "HOLD", "reason": "record not found"})
0477              else:
0478                  self._json(200, body, state=body["state"], record_hash=digest)
0479              return
0480          self._json(404, {"state": "HOLD", "reason": "not found"})
0481
0482      def do_POST(self) -> None:
0483          if not self._authenticated():
0484              return
0485          try:
0486              request_binding = parse_effect_ack_request(self.headers.get("Effect-Ack-Request"))
0487              body = self._read_body()
0488              if self.path == "/terminal/prepare":
0489                  if request_binding["mode"] != "prepare":
0490                      raise ValueError("prepare endpoint requires mode=prepare")
0491                  self._prepare(body)
0492                  return
0493              if self.path == "/terminal/commit":
0494                  if request_binding["mode"] != "commit":
0495                      raise ValueError("commit endpoint requires mode=commit")
0496                  self._commit(body, request_binding)
0497                  return
0498              self._json(404, {"state": "HOLD", "reason": "not found"})
0499          except (ValueError, UnicodeError, json.JSONDecodeError) as exc:
0500              self._json(400, {"state": "HOLD", "ordinary_release": False, "reason": str(exc)})
0501
0502      def _prepare(self, body: dict[str, Any]) -> None:
0503          if body.get("schema") != "qikvrt_terminal_input_v1":
0504              digest, record = STATE.record(
```

```
                state="EFFECT_ACK_BLOCK",
                input_hash=sha256(canonical_json(body)),
                ordinary_release=False,
                reason="unsupported terminal schema",
            )
            self._json(422, {"record": record, "record_hash": digest}, state=record["state"],
    record_hash=digest)
            return
        input_hash = sha256(canonical_json(body))
        with STATE.lock:
            digest, record = STATE.record(
                state="EFFECT_ACK_DONE",
                input_hash=input_hash,
                ordinary_release=True,
                reason="loopback terminal input satisfies bounded local policy",
            )
            token = STATE.make_token(input_hash, digest)
        response = {
            "state": "EFFECT_ACK_DONE",
            "ordinary_release": False,
            "commit_token": token,
            "record_hash": digest,
            "record_url": f"/effect-ack/records/{digest}",
            "expires_in_seconds": TOKEN_TTL_SECONDS,
            "external_effect": "NONE",
        }
        self._json(200, response, state="EFFECT_ACK_DONE", record_hash=digest, commit_token=token)

    def _commit(self, body: dict[str, Any], binding: dict[str, Any]) -> None:
        token = binding["token"]

        record_hash = binding["hash"]
        commit_input_hash = sha256(canonical_json(body))
        with STATE.lock:
            prepared = STATE.prepared.get(token)
            if prepared is None or prepared.used or prepared.expires_at < time.time() or not
    hmac.compare_digest(prepared.record_hash, record_hash):
                self._json(409, {"state": "HOLD", "ordinary_release": False, "reason": "invalid stale
    used or mismatched token"})
                return
            if not hmac.compare_digest(prepared.input_hash, commit_input_hash):
                self._json(409, {"state": "HOLD", "ordinary_release": False, "reason": "commit payload
    differs from exact prepared payload"})
                return
            prepared.used = True
            event = {
                "event_id": len(STATE.events) + 1,
                "kind": "TERMINAL_INPUT_ACCEPTED",
                "record_hash": record_hash,
                "input_hash": commit_input_hash,
                "text": str(body.get("text", ""))[:4096],
                "audio_present": body.get("audio") is not None,
                "video_present": body.get("video") is not None,
                "observed_at": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
                "external_effect": "NONE",
            }
            STATE.events.append(event)
            digest, record = STATE.record(
                state="EFFECT_ACK_DONE",
                input_hash=prepared.input_hash,
                ordinary_release=True,
                reason="single-use exact-bound loopback commit executed",
            )
        self._json(
            200,
            {"state": "EFFECT_ACK_DONE", "ordinary_release": True, "post_effect": event,
    "successor_record": record},
            state="EFFECT_ACK_DONE",
            record_hash=digest,
        )

    def log_message(self, fmt: str, *args: Any) -> None:
        return


def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("operation", nargs="?", default="serve", choices=("serve", "durable-prepare",
```

```
      "durable-commit", "durable-readback"))
0577      parser.add_argument("--host", default=HOST)
0578      parser.add_argument("--port", type=int, default=DEFAULT_PORT)
0579      parser.add_argument("--state-dir", default="/var/lib/qikvrt/state")
0580      parser.add_argument("--root", default=".")
0581      parser.add_argument("--repository")
0582      parser.add_argument("--pr", type=int)
0583      parser.add_argument("--expected-head")
0584      parser.add_argument("--expected-tree")
0585      parser.add_argument("--manifest-sha256", help="independent sealed S1 export pin; avoids Git only when
       explicitly supplied")
0586      parser.add_argument("--input", type=Path)
0587      parser.add_argument("--prepared", type=Path)
0588      parser.add_argument("--prepare-hash")
0589      args = parser.parse_args()

0590      if args.operation != "serve":
0591          try:
0592              if not all((args.repository, args.pr, args.expected_head, args.expected_tree, args.input)):
0593                  raise DurableHold("EXACT_SUBJECT_AND_INPUT_REQUIRED")
0594              subject = durable_subject(args.root, args.repository, args.pr, args.expected_head,
      args.expected_tree, args.manifest_sha256)
0595              def load(path: Path) -> dict[str, Any]:
0596                  with path.open("rb") as source:
0597                      raw = source.read(MAX_NATIVE_EVENT * 2 + 1)
0598                  if len(raw) > MAX_NATIVE_EVENT * 2:
0599                      raise DurableHold("BOUNDED_FILE_REQUIRED")
0600                  value = json.loads(raw)
0601                  if not isinstance(value, dict):
0602                      raise DurableHold("JSON_OBJECT_REQUIRED")
0603                  return value
0604              body = load(args.input)
0605              if args.operation == "durable-prepare":
0606                  result = durable_prepare(body, subject, args.state_dir)
0607              else:
0608                  if not args.prepared or not args.prepare_hash:
0609                      raise DurableHold("EXACT_PREPARE_HASH_REQUIRED")
0610                  operation = durable_commit if args.operation == "durable-commit" else durable_readback
0611                  result = operation(load(args.prepared), args.prepare_hash, body, subject, args.state_dir)
0612              print(json.dumps(result, ensure_ascii=False, sort_keys=True))
0613              return 0
0614          except (OSError, ValueError, sqlite3.Error, TypeError, KeyError, RecursionError) as exc:
0615              print(json.dumps({"state": "HOLD", "reason": str(exc), "EFFECT_ACK_DONE": False,
0616                                "ordinary_release": False, "next_action":
      "READBACK_BEFORE_ANY_COMMIT_RETRY"}, sort_keys=True))
0617              return 2
0618      if args.host not in {"127.0.0.1", "localhost"}:
0619          raise SystemExit("BLOCK: reference terminal bridge is loopback-only")
0620      server = ThreadingHTTPServer((args.host, args.port), Handler)
0621      print(json.dumps({"state": "READY", "host": args.host, "port": args.port, "external_effects":
      "NONE"}, sort_keys=True), flush=True)
0622      server.serve_forever()
0623      return 0
0624
0625
0626  if __name__ == "__main__":
0627      raise SystemExit(main())
```

B3 - tools/qikvrt_self_host.py

Archivkennung F099; Ursprungspfad tools/qikvrt_self_host.py

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 1788bab299d3fcaf07af99c57a2d1f2471636d99; SHA-256 75d334b48eefb837ad02b285765764b5a0912d6b0bc2596284a7ae2ba02c1345; 28982 Byte

```
0001  #!/usr/bin/env python3
0002  # SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0003  # Copyright 2026 Ingolf Lohmann. Implementation contribution: OpenAI Codex.
0004  """Freeze and start the existing monitor/terminal carriers; no job executor."""
0005  from __future__ import annotations
0006
0007  import argparse
0008  import fcntl
0009  import hashlib
0010  import importlib.util
0011  import json
0012  import os
0013  import platform
0014  import re
0015  import shutil
0016  import signal
0017  import socket
0018  import stat
0019  import struct
0020  import time
0021  import urllib.request
0022  import subprocess
0023  import sys
0024  import tarfile
0025  import threading
0026  from http.server import ThreadingHTTPServer
0027  from pathlib import Path
0028
0029  ROOT = Path(__file__).resolve().parents[1]
0030  DEFINITION = "runtime/self-host/PACKAGE.json"
0031  BROWSER_COMMANDS = ("firefox-esr", "Xvfb", "x11vnc", "websockify", "xdpyinfo", "xwininfo")
0032
0033
0034  def raw_json(value):
0035      return (json.dumps(value, sort_keys=True, indent=2, ensure_ascii=False) + "\n").encode()
0036
0037
0038  def digest(raw):
0039      return hashlib.sha256(raw).hexdigest()
0040
0041
0042  def git(root, *args):
0043      return subprocess.check_output(["git", "-C", str(root), *args], timeout=30,
      stderr=subprocess.DEVNULL).strip()
0044
0045
0046  def runtime(command):
0047      path = Path(shutil.which(command) or command).resolve(strict=True)
0048      version = subprocess.check_output([str(path), "--version"], timeout=5, stderr=subprocess.STDOUT,
0049                                        env={"PATH": os.environ.get("PATH", ""), "LANG":
      "C"}).decode().strip()
0050      return {"version": version, "executable_sha256": digest(path.read_bytes())}
0051
0052
0053  def browser_executables():
0054      # Distribution-built tools use an exact executable binding, not a made-up
0055      # universal version. Actual package versions are retained by the image build.
0056      return {name: digest(Path(shutil.which(name) or name).resolve(strict=True).read_bytes())
0057              for name in BROWSER_COMMANDS}
0058
0059
0060  def freeze(root, output, head, tree, browser_assets=None):

0061      """Export committed bytes only. Tar metadata is deterministic; no credentials."""
0062      if git(root, "rev-parse", "HEAD").decode() != head or git(root, "rev-parse", "HEAD^{tree}").decode()
      != tree:
0063          raise ValueError("EXACT_SOURCE_BINDING_MISMATCH")
```

```
    git(root, "diff", "--quiet", head, "--")
    definition = json.loads(git(root, "show", head + ":" + DEFINITION))
    node, python = runtime("node"), runtime(sys.executable)
    if not node["version"].startswith("v24.") or not python["version"].startswith("Python 3.12.") or sys.platform != "linux":
        raise ValueError("LINUX_NODE24_PYTHON312_REQUIRED")
    manifest = {"schema": "qikvrt-self-host-package/v1", "package_version": definition["version"],
                "source_repository": definition["source_repository"], "source_head": head, "source_tree": tree,
                "runtime": {"node": node, "python": python, "platform": platform.machine(), "system": "linux"},
                "files": {}, "effect_ack_done": False,
                "terminal_scope": "RECOVERED_HISTORICAL_TEMDD_DAEMON_AND_FIREFOX_NOVNC_SOURCES; SEALED_S1_ADAPTER",
                "native_terminal_daemon_included": definition.get("native_terminal_daemon_included", False)}
    if browser_assets is not None:
        manifest["browser_runtime"] = {"executables": browser_executables(), "files": [],
            "asset_origin": "OPERATOR_PROVISIONED_DEBIAN_NOVNC_SOURCE; EXACT_EXPORTED_BYTES",
            "firefox_version": subprocess.check_output([shutil.which("firefox-esr"), "--version"],
                timeout=10, stderr=subprocess.DEVNULL).decode().strip()}
        if not (browser_assets / "vnc.html").is_file() or not (browser_assets / "core/rfb.js").is_file():
            raise ValueError("COMPLETE_NOVNC_SOURCE_REQUIRED")
    output.mkdir(mode=0o700, parents=True, exist_ok=False)
    try:
        for name in sorted(definition["files"]):
            if name.startswith("/") or ".." in Path(name).parts:
                raise ValueError("UNSAFE_PACKAGE_PATH")
            data = subprocess.check_output(["git", "-C", str(root), "cat-file", "blob", head + ":" + name], timeout=30)
            entry = git(root, "ls-tree", head, "--", name).decode().split()[0]
            if entry not in {"100644", "100755"}:
                raise ValueError("REGULAR_COMMITTED_FILES_REQUIRED")
            path = output / name
            path.parent.mkdir(parents=True, exist_ok=True)
            path.write_bytes(data)
            path.chmod(0o755 if entry == "100755" else 0o644)
            manifest["files"][name] = {"bytes": len(data), "sha256": digest(data), "mode": entry}
        if browser_assets is not None:
            # Debian's noVNC tree contains distribution-managed JS links.
            # Freeze their resolved source bytes as regular files, with no key,
            # certificate, credential, VCS or executable payload admitted.
            suffixes = {".html", ".js", ".css", ".svg", ".png", ".jpg", ".jpeg", ".gif", ".ico",
                        ".woff", ".woff2", ".ttf", ".eot", ".oga", ".ogg", ".wav", ".mp3", ".json", ".txt", ".map"}
            allowed = (browser_assets.resolve(), Path('/usr/share/javascript'), Path('/usr/share/nodejs'))
            def admitted(source):
                resolved = source.resolve(strict=True)
                if not any(resolved == root or root in resolved.parents for root in allowed):
                    raise ValueError("NOVNC_DISTRIBUTION_SOURCE_LINK_ESCAPED")
            sources = []
            for parent, directories, filenames in os.walk(browser_assets, followlinks=True):
                admitted(Path(parent))
                if len(Path(parent).relative_to(browser_assets).parts) > 16:
                    raise ValueError("NOVNC_SOURCE_LINK_CYCLE_OR_DEPTH")
                directories[:] = sorted(p for p in directories if not p.startswith('.'))
                sources.extend(Path(parent) / name for name in filenames)

                if len(sources) > 4096: raise ValueError("BOUNDED_NOVNC_FILESET_REQUIRED")
            for source in sorted(sources):
                relative = source.relative_to(browser_assets)
                if any(p.startswith(".") for p in relative.parts) or not source.is_file() or source.suffix not in suffixes:
                    continue
                admitted(source)
                data = source.read_bytes()
                if len(data) > 8 * 1024 * 1024: raise ValueError("BOUNDED_NOVNC_ASSET_REQUIRED")
                name = "runtime/self-host/novnc/" + relative.as_posix()
                dest = output / name; dest.parent.mkdir(parents=True, exist_ok=True)
                dest.write_bytes(data); dest.chmod(0o644)
                manifest["files"][name] = {"bytes": len(data), "sha256": digest(data), "mode": "100644"}
                manifest["browser_runtime"]["files"].append(name)
            copyright_file = Path("/usr/share/doc/novnc/copyright")
            if not copyright_file.is_file(): raise ValueError("NOVNC_DISTRIBUTION_RIGHTS_REQUIRED")
            name = "runtime/self-host/novnc/DEBIAN_COPYRIGHT.txt"
            data = copyright_file.read_bytes(); (output / name).write_bytes(data); (output / name).chmod(0o644)
```

```
            manifest["files"][name] = {"bytes": len(data), "sha256": digest(data), "mode": "100644"}
            manifest["browser_runtime"]["files"].append(name)
            manifest["browser_runtime"]["files"].sort()
        data = raw_json(manifest)
        (output / "MANIFEST.json").write_bytes(data)
        (output / "MANIFEST.json").chmod(0o644)
        with tarfile.open(str(output) + ".tar", "w", format=tarfile.PAX_FORMAT) as archive:
            for path in sorted(p for p in output.rglob("*") if p.is_file()):
                info = archive.gettarinfo(str(path), path.relative_to(output).as_posix())
                info.uid = info.gid = info.mtime = 0
                info.uname = info.gname = ""
                with path.open("rb") as source:
                    archive.addfile(info, source)
        return {"state": "PACKAGE_FROZEN", "manifest_sha256": digest(data), "source_head": head,
                "source_tree": tree, "archive_sha256": digest(Path(str(output) + ".tar").read_bytes()),
                "effect_ack_done": False}
    except BaseException:
        shutil.rmtree(output)
        raise


def verify(package, expected):
    raw = (package / "MANIFEST.json").read_bytes()
    if not re.fullmatch(r"[a-f0-9]{64}", expected or "") or digest(raw) != expected:
        raise ValueError("PACKAGE_MANIFEST_PIN_MISMATCH")
    value = json.loads(raw)
    if value.get("schema") != "qikvrt-self-host-package/v1" or value.get("effect_ack_done") is not False:
        raise ValueError("PACKAGE_SCHEMA_MISMATCH")
    for key in ("source_head", "source_tree"):
        if not re.fullmatch(r"[a-f0-9]{40}", value.get(key, "")):
            raise ValueError("EXACT_SOURCE_BINDING_REQUIRED")
    actual = {p.relative_to(package).as_posix() for p in package.rglob("*") if p.is_file() or
p.is_symlink()}
    if actual != set(value["files"]) | {"MANIFEST.json"}:
        raise ValueError("PACKAGE_INVENTORY_MISMATCH")
    for name, entry in value["files"].items():
        if name.startswith("/") or ".." in Path(name).parts:
            raise ValueError("UNSAFE_PACKAGE_PATH")
        path = package / name
        if path.is_symlink() or any(p.is_symlink() for p in path.parents):
            raise ValueError("PACKAGE_SYMLINK_FORBIDDEN")
        raw = path.read_bytes()
        if (entry["bytes"] != len(raw) or entry["sha256"] != digest(raw)

                or stat.S_IMODE(path.stat().st_mode) != (0o755 if entry["mode"] == "100755" else 0o644)):
            raise ValueError("PACKAGE_ARTIFACT_MISMATCH")
    definition = json.loads((package / DEFINITION).read_bytes())
    browser = value.get("browser_runtime")
    if browser is not None and (not isinstance(browser, dict)
            or set(browser.get('executables', {})) != set(BROWSER_COMMANDS)
            or not isinstance(browser.get('files'), list)
            or len(browser['files']) != len(set(browser['files']))
            or not {'runtime/self-host/novnc/vnc.html', 'runtime/self-host/novnc/core/rfb.js',
                    'runtime/self-host/novnc/DEBIAN_COPYRIGHT.txt'} <= set(browser['files'])):
        raise ValueError("BROWSER_RUNTIME_SCHEMA_MISMATCH")
    extra = set(browser["files"]) if browser is not None else set()
    if any(not p.startswith("runtime/self-host/novnc/") for p in extra):
        raise ValueError("NOVNC_SOURCE_CONFINEMENT_REQUIRED")
    if set(value["files"]) != set(definition["files"]) | extra or definition["version"] !=
value["package_version"]:
        raise ValueError("PACKAGE_DEFINITION_MISMATCH")
    if browser is not None and browser["executables"] != browser_executables():
        raise ValueError("BROWSER_EXECUTABLE_BINDING_MISMATCH")
    if value["runtime"] != {"node": runtime("node"), "python": runtime(sys.executable),
                            "platform": platform.machine(), "system": sys.platform}:
        raise ValueError("RUNTIME_EXECUTABLE_BINDING_MISMATCH")
    return value


def private_path(path, directory=False):
    if not path.is_absolute() or ".." in path.parts or any(p.is_symlink() for p in (path,
*path.parents)):
        raise ValueError("PRIVATE_PATH_MUST_BE_ABSOLUTE_AND_NOT_SYMLINKED")
    info = path.lstat()
    if (info.st_uid != os.geteuid() or stat.S_IMODE(info.st_mode) != (0o700 if directory else 0o600)
            or not (stat.S_ISDIR(info.st_mode) if directory else stat.S_ISREG(info.st_mode))):
        raise ValueError("OWNER_ONLY_PRIVATE_PATH_REQUIRED")
```

```
0205
0206  def packaged_subject(package, pin, repository, pr, head, tree):
0207      """Verify an independently pinned export, without Git or a remote provider."""
0208      manifest = verify(package, pin)
0209      if (repository != manifest["source_repository"] or type(pr) is not int or pr < 1
0210              or head != manifest["source_head"] or tree != manifest["source_tree"]):
0211          raise ValueError("EXACT_PACKAGED_SUBJECT_MISMATCH")
0212      return {"repository": repository, "pr": pr, "head": head, "tree": tree}
0213
0214
0215  def load_source(package, name):
0216      spec = importlib.util.spec_from_file_location(name, package / "src" / (name + ".py"))
0217      module = importlib.util.module_from_spec(spec)
0218      sys.modules[name] = module
0219      spec.loader.exec_module(module)
0220      return module
0221
0222
0223  def native_carrier(package, pin, config, config_path, binding, token):
0224      """Reuse the recovered ledger/HTTP/Unix implementation; adapt only its source binding.
0225
0226      The original Runtime requires Git. This new S1 adapter binds the sealed
0227      package/config/volume instead. Original source bytes remain unmodified.
0228      """
0229      native = load_source(package, "qikvrt_temdd_event_ledger")
0230
0231      class SealedRuntime(native.Runtime):
0232          def __init__(self):

0233              self.root = package
0234              self.repository, self.pr = config["source_repository"], config["subject_pr"]
0235              self.subject = packaged_subject(package, pin, self.repository, self.pr,
0236                                              config["source_head"], config["source_tree"])
0237              self.ledger = native.Ledger(config["state_dir"], self.subject)
0238              self.subscribers = threading.BoundedSemaphore(32)
0239
0240          def ensure_subject(self):
0241              try:
0242                  packaged_subject(package, pin, self.repository, self.pr,
0243                                   self.subject["head"], self.subject["tree"])
0244                  private_path(config_path)
0245                  if digest(config_path.read_bytes()) != binding["config_sha256"]:
0246                      raise native.Hold("EXACT_CONFIGURATION_CHANGED")
0247                  volume_binding = Path(config["state_dir"]) / "binding.json"
0248                  private_path(volume_binding)
0249                  if json.loads(volume_binding.read_bytes()) != binding:
0250                      raise native.Hold("PERSISTENT_VOLUME_BINDING_MISMATCH")
0251                  if self.ledger.stopped:
0252                      raise native.Hold(self.ledger.reason)
0253              except (OSError, ValueError, KeyError, subprocess.SubprocessError) as exc:
0254                  self.ledger.stop(str(exc))
0255                  raise native.Hold(str(exc)) from exc
0256              return self.subject
0257
0258      runtime_instance = SealedRuntime()
0259      ingress = server = None
0260      try:
0261          original_handler = native.make_handler(runtime_instance)
0262
0263          class ReadOnlyHTTP(original_handler):
0264              def do_POST(self):
0265                  self._hold("NATIVE_UNIX_INGRESS_REQUIRED", 405)
0266
0267          ingress = native.make_ingress(runtime_instance)
0268          # Defense in depth for the existing mode-0600 owner-only ingress.
0269          def same_owner(request, _address):
0270              _, uid, _ = struct.unpack("3i", request.getsockopt(socket.SOL_SOCKET, socket.SO_PEERCRED,
      12))
0271              return uid == os.geteuid()
0272          ingress.verify_request = same_owner
0273          server = ThreadingHTTPServer(("127.0.0.1", config["terminal_port"]), ReadOnlyHTTP)
0274          server.terminal_auth_token = token
0275          server.runtime_binding = binding
0276          return runtime_instance, ingress, server
0277      except BaseException:
0278          if server is not None: server.server_close()
0279          if ingress is not None: ingress.server_close()
0280          runtime_instance.ledger.close()
```

```
        raise


def novnc_readback(package, port):
    # A loopback startup read must never inherit a provider/proxy route.
    opener = urllib.request.build_opener(urllib.request.ProxyHandler({}))
    with opener.open('http://127.0.0.1:' + str(port) + '/vnc.html', timeout=2) as response:
        return response.status == 200 and response.read() == (package / 'runtime/self-host/novnc/vnc.html').read_bytes()


def browser_carrier(package, manifest, config, volume):
    """New portable start adapter of the recovered Firefox/Xvfb/VNC recipe.

    No downloader, provider, unsigned-addon exception or public listener.
    Browser profiles remain private; the existing host owns HTTPS admission.
    """
    if "browser_runtime" not in manifest: raise ValueError("BROWSER_PACKAGE_REQUIRED")
    password = Path(config["browser_password_file"]); private_path(password)
    secret = password.read_bytes().rstrip(b"\n")
    if len(secret) != 8 or any(c < 33 or c > 126 for c in secret):
        raise ValueError("EIGHT_ASCII_BYTE_PRIVATE_VNC_PASSWORD_REQUIRED")
    directory = volume / "browser"
    directory.mkdir(mode=0o700, exist_ok=True); private_path(directory, directory=True)
    for name in ("profile", "home", "cache", "logs"):
        (directory / name).mkdir(mode=0o700, exist_ok=True); private_path(directory / name, directory=True)
    env = {"PATH": os.environ.get("PATH", ""), "LANG": "C.UTF-8", "DISPLAY": config["display"],
        "HOME": str(directory / "home"), "XDG_CACHE_HOME": str(directory / "cache"),
        "XDG_CONFIG_HOME": str(directory / "home")}
    processes, logs = [], []
    def spawn(name, argv):
        path = directory / "logs" / (name + ".log")
        if path.exists() or path.is_symlink(): private_path(path)
        fd = os.open(path, os.O_CREAT | os.O_WRONLY | os.O_APPEND | os.O_NOFOLLOW, 0o600)
        log = os.fdopen(fd, "wb"); logs.append(log)
        p = subprocess.Popen(argv, env=env, cwd=directory, stdout=log, stderr=log)
        processes.append(p); return p
    def ready(check, edge):
        until = time.monotonic() + 20
        while time.monotonic() < until:
            if any(p.poll() is not None for p in processes): raise ValueError("BROWSER_CHILD_EXITED")
            try:
                if check(): return
            except (OSError, ValueError, subprocess.SubprocessError): pass
            time.sleep(.05)  # bounded startup probe, no runtime polling
        raise ValueError("BROWSER_STARTUP_READBACK_TIMEOUT:" + edge)
    try:
        spawn("xvfb", [shutil.which("Xvfb"), config["display"], "-screen", "0", "1440x900x24", "-nolisten", "tcp"])
        ready(lambda: subprocess.run([shutil.which("xdpyinfo"), "-display", config["display"]],
            env=env, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, timeout=2).returncode == 0, 'X11_DISPLAY')
        spawn("vnc", [shutil.which("x11vnc"), "-display", config["display"], "-forever", "-shared", "-localhost",
            "-no6", "-rfbportv6", "0", "-rfbport", str(config["vnc_port"]), "-passwdfile", str(password)])
        spawn("novnc", [shutil.which("websockify"), "--web=" + str(package / "runtime/self-host/novnc"),
            "127.0.0.1:" + str(config["novnc_port"]), "127.0.0.1:" + str(config["vnc_port"])])
        spawn("firefox", [shutil.which("firefox-esr"), "--no-remote", "--profile", str(directory / "profile"),
            "http://127.0.0.1:" + str(config["terminal_port"]) + "/AI"])
        def browser_window():
            raw = subprocess.check_output([shutil.which("xwininfo"), "-display", config["display"], "-root", "-tree"],
                env=env, timeout=2).decode()
            title = re.search(r'<title>([^<]+)</title>', (package / 'docs/terminal/temdd/index.html').read_text())[1]
            return '"Navigator" "firefox' in raw and '"' + title in raw
        ready(browser_window, 'FIREFOX_NATIVE_DOCUMENT')
        ready(lambda: novnc_readback(package, config['novnc_port']), 'NOVNC_HTTP')
        return processes, logs
    except BaseException:
        stop_children(processes, logs)
        raise


```

```
def stop_children(processes, logs=()):
    for p in processes:
        if p.poll() is None: p.terminate()
    for p in processes:
        try: p.wait(timeout=5)
        except subprocess.TimeoutExpired: p.kill(); p.wait(timeout=2)
    for log in logs: log.close()


def start(package, pin, config_path):
    manifest = verify(package, pin)
    private_path(config_path)
    raw = config_path.read_bytes()
    config = json.loads(raw)
    fields = {"schema", "node_id", "source_head", "source_tree", "adapter", "state_dir", "host", "port", "terminal_port", "terminal_token_file", "github_webhook_secret_file", "terminal_profile", "subject_pr", "source_repository", "browser_password_file", "novnc_port", "vnc_port", "display"}
    if (set(config) - fields or config.get("schema") != "qikvrt-self-host-config/v1"
            or not re.fullmatch(r"[A-Za-z0-9_.:-]{1,100}", config.get("node_id", ""))
            or config.get("adapter") not in {"none", "github"} or config.get("host") not in {"127.0.0.1", "0.0.0.0"}
            or any(type(config.get(k)) is not int or not 1 <= config[k] <= 65535 for k in ("port", "terminal_port"))
            or config["port"] == config["terminal_port"]
            or any(config.get(k) != manifest[k] for k in ("source_head", "source_tree"))):
        raise ValueError("START_CONFIGURATION_BINDING_MISMATCH")
    profile = config.get("terminal_profile", "reference")
    if profile not in {"reference", "temdd", "firefox"}:
        raise ValueError("UNKNOWN_TERMINAL_PROFILE")
    if profile != "reference" and (type(config.get("subject_pr")) is not int or config["subject_pr"] < 1
            or config.get("source_repository") != manifest["source_repository"]
            or manifest.get("native_terminal_daemon_included") is not True):
        raise ValueError("EXACT_NATIVE_TERMINAL_SUBJECT_REQUIRED")
    if profile == "firefox" and ("browser_runtime" not in manifest
            or not re.fullmatch(r":[1-9][0-9]{0,3}", config.get("display", ""))
            or any(type(config.get(k)) is not int or not 1 <= config[k] <= 65535 for k in ("novnc_port", "vnc_port"))
            or len({config[k] for k in ("port", "terminal_port", "novnc_port", "vnc_port")}) != 4
            or not config.get("browser_password_file")):
        raise ValueError("EXACT_BROWSER_CONFIGURATION_REQUIRED")
    if config["adapter"] == "github":
        try:
            if runtime("gh")["version"].splitlines()[0].split()[:3] != ["gh", "version", "2.96.0"]:
                raise ValueError("GITHUB_ADAPTER_REQUIRES_LOCKED_GH")
        except OSError as exc:
            raise ValueError("GITHUB_ADAPTER_REQUIRES_LOCKED_GH") from exc
        if not config.get("github_webhook_secret_file"):
            raise ValueError("PRIVATE_GITHUB_WEBHOOK_SECRET_REQUIRED")
        secret = Path(config["github_webhook_secret_file"])
        private_path(secret)
        if not 32 <= len(secret.read_bytes().strip()) <= 256:
            raise ValueError("BOUNDED_PRIVATE_GITHUB_WEBHOOK_SECRET_REQUIRED")
    if config["adapter"] == "none" and config.get("github_webhook_secret_file"):
        raise ValueError("GITHUB_SECRET_WITHOUT_GITHUB_ADAPTER")
    token_file = Path(config["terminal_token_file"])
    private_path(token_file)
    token = token_file.read_text().strip()
    if len(token) < 32 or len(token) > 256 or not token.isascii() or any(c.isspace() for c in token):
        raise ValueError("BOUNDED_TERMINAL_SECRET_REQUIRED")
    volume = Path(config["state_dir"])

    private_path(volume, directory=True)
    if volume == package or package in volume.parents or volume in package.parents:
        raise ValueError("STATE_VOLUME_MUST_BE_SEPARATE_FROM_PACKAGE")
    for name in ("monitor", "temdd", "receipts"):
        (volume / name).mkdir(mode=0o700, exist_ok=True)
        private_path(volume / name, directory=True)
    lock_fd = os.open(volume / "node.lock", os.O_RDWR | os.O_CREAT | os.O_NOFOLLOW, 0o600)
    with os.fdopen(lock_fd, "w") as lock:
        private_path(volume / "node.lock")
        fcntl.flock(lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
        binding = {"schema": "qikvrt-self-host-volume/v1", "node_id": config["node_id"], "manifest_sha256": pin,
                   "config_sha256": digest(raw), "source_head": manifest["source_head"], "source_tree": manifest["source_tree"]}
        bound_path = volume / "binding.json"
        if bound_path.exists() or bound_path.is_symlink():
            private_path(bound_path)
```

```
            if json.loads(bound_path.read_bytes()) != binding:
                raise ValueError("PERSISTENT_VOLUME_BINDING_MISMATCH")
        else:
            with bound_path.open("xb") as dest:
                os.chmod(bound_path, 0o600)
                dest.write(raw_json(binding)); dest.flush(); os.fsync(dest.fileno())
            fd = os.open(volume, os.O_RDONLY)
            try: os.fsync(fd)
            finally: os.close(fd)
        for file in (volume / "monitor").iterdir():
            private_path(file)
        terminal = load_source(package, "qikvrt_effect_ack_http_terminal")
        native_runtime = ingress = None
        if profile == "reference":
            server = ThreadingHTTPServer(("127.0.0.1", config["terminal_port"]), terminal.Handler)
        else:
            native_runtime, ingress, server = native_carrier(package, pin, config, config_path, binding,
    token)
        server.terminal_auth_token = token
        server.runtime_binding = binding
        thread = threading.Thread(target=server.serve_forever, daemon=True)
        thread.start()
        ingress_thread = None
        if ingress is not None:
            ingress_thread = threading.Thread(target=ingress.serve_forever, daemon=True)
            ingress_thread.start()
        env = {"PATH": os.environ.get("PATH", ""), "LANG": "C.UTF-8", "QIKVRT_NODE_CONFIG":
    str(config_path),
               "QIKVRT_NODE_PACKAGE": str(package), "QIKVRT_NODE_MANIFEST_SHA256": pin,
               "QIKVRT_NODE_CONFIG_SHA256": digest(raw)}
        process = None
        browser_processes, browser_logs = [], []
        def stop(_signum, _frame):
            if process is not None and process.poll() is None:
                process.terminate()
        old_handlers = {s: signal.signal(s, stop) for s in (signal.SIGINT, signal.SIGTERM)}
        try:
            if profile == "firefox":
                browser_processes, browser_logs = browser_carrier(package, manifest, config, volume)
            # flock belongs to the shared open-file description. Keep it in
            # the monitor too, so an isolated launcher SIGKILL cannot admit a
            # second writer while the original monitor is still running.
            process = subprocess.Popen([shutil.which("node"), str(package /
    "docs/monitor/self-host.mjs")],
                                       cwd=package, env=env, pass_fds=(lock.fileno(),))
            if browser_processes:
                # Linux pidfds wait for actual child exits without timed polls.
                import select
                fds = [os.pidfd_open(p.pid) for p in [process, *browser_processes]]
                try:
                    select.select(fds, [], [])
                    if any(p.poll() is not None for p in browser_processes):
                        raise ValueError("BROWSER_CHILD_EXITED_AFTER_READINESS")
                    return process.wait()
                finally:
                    for fd in fds: os.close(fd)
            return process.wait()
        finally:
            if process is not None and process.poll() is None:
                process.terminate()
                try: process.wait(timeout=5)
                except subprocess.TimeoutExpired: process.kill(); process.wait()
            server.shutdown(); server.server_close(); thread.join(timeout=2)
            stop_children(browser_processes, browser_logs)
            if ingress is not None:
                native_runtime.ledger.stop()
                ingress.shutdown(); ingress.server_close(); ingress_thread.join(timeout=6)
                native_runtime.ledger.close()
            for s, handler in old_handlers.items(): signal.signal(s, handler)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("operation", choices=("pack", "verify", "run"))
    parser.add_argument("--root", type=Path, default=ROOT)
    parser.add_argument("--output", type=Path)
    parser.add_argument("--expected-head")
```

```
0493      parser.add_argument("--expected-tree")
0494      parser.add_argument("--manifest-sha256")
0495      parser.add_argument("--config", type=Path)
0496      parser.add_argument("--browser-assets-root", type=Path, help="explicit provisioned noVNC source tree;
       freeze a Firefox-capable variant")
0497      args = parser.parse_args()
0498      try:
0499          if args.operation == "pack":
0500              if not args.output or not args.expected_head or not args.expected_tree: raise
      ValueError("EXACT_EXPORT_INPUTS_REQUIRED")
0501              result = freeze(args.root, args.output, args.expected_head, args.expected_tree,
      args.browser_assets_root)
0502          elif args.operation == "verify":
0503              verify(args.root, args.manifest_sha256)
0504              result = {"state": "EXACT_PACKAGE_RUNTIME_VERIFIED", "effect_ack_done": False}
0505          else:
0506              if not args.config: raise ValueError("PRIVATE_START_CONFIGURATION_REQUIRED")
0507              return start(args.root.resolve(), args.manifest_sha256, args.config.absolute())
0508          print(json.dumps(result, sort_keys=True))
0509          return 0
0510      except (OSError, ValueError, KeyError, TypeError, subprocess.SubprocessError) as exc:
0511          print(json.dumps({"state": "HOLD", "cause": str(exc), "effect_ack_done": False}, sort_keys=True))
0512          return 2
0513
0514
0515  if __name__ == "__main__":
0516      raise SystemExit(main())
```

B4 - tools/qikvrt_temdd.py

Archivkennung F176; Ursprungspfad tools/qikvrt_temdd.py

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 740b757baca640ca4f3300b8669d0ebf09439efc; SHA-256 6d086aad67b10fe5878bad6b13beacf5f4f51e21f457d7e678816ea64bf8e667; 2042 Byte

```
#!/usr/bin/env python3
"""TEMDD v0.1 deterministic reference parser/elaborator."""
import json,re,sys
from pathlib import Path
ID=r"[A-Za-z][A-Za-z0-9_-]*"
QIKVRT_DOD={'ZERO_BUGS','ALL_PULL_REQUESTS_REGARDED','ALL_BRANCHES_REGARDED','ALL_PRODUCTIVE_BRANCHES_MERGED','FRESH_EXACT_MAIN_VALIDATION_PASS','FRESH_EFFECT_READBACK'}
def parse(text):
 def one(p,label,src=None):
  m=re.search(p,text if src is None else src,re.S)
  if not m: raise ValueError('missing '+label)
  return m
 v=one(r"^\s*temdd\s+([0-9.]+)\s*;",'version').group(1)
 if v!='0.1': raise ValueError('unsupported version')
 a=one(r"authority\s+"+ID+r"\s*=\s*\"([^\"]+)\"\s*;",'authority').group(1)
 s=one(r"subject\s+("+ID+r")\s*\{(.*?)\}",'subject'); sb=s.group(2)
 rm=one(r"repository\s*=\s*\"([^\"]+)\"\s*;",'repository',sb); one(r"binding\s*=\s*exact\s*;",'exact binding',sb)
 q=one(r"request\s+("+ID+r")\s*\{\s*target\s*=\s*("+ID+r")\s*;\s*\}",'request'); target=q.group(2)
 handlers=[]
 for m in re.finditer(r"on\s+(event|blocker)\s*\{(.*?)\}",text,re.S):
handlers.append({'event':m.group(1),'statements':[x.strip() for x in m.group(2).split(';') if x.strip()]})
 d=one(r"until\s*\{(.*?)\}",'until').group(1).strip().rstrip(';'); dod=[x.strip() for x in d.split('&&') if x.strip()]
 if not handlers or not dod: raise ValueError('handlers and DoD required')
 if len(set(dod))!=len(dod): raise ValueError('duplicate DoD predicate')
 if target=='QIKVRT_DOD' and set(dod)!=QIKVRT_DOD: raise ValueError('QIKVRT_DOD requires complete canonical predicate')
 return {'schema':'temdd_ir_v0_1','version':v,'authority':a,'subject':{'name':s.group(1),'repository':rm.group(1),'binding':'exact'},'request':{'name':q.group(1),'target':target},'handlers':handlers,'dod':dod}
def main(argv):
 if len(argv)!=2:return 64
 try: ir=parse(Path(argv[1]).read_text(encoding='utf-8'))
 except (OSError,ValueError) as e: print('BLOCK '+str(e),file=sys.stderr);return 2
 print(json.dumps(ir,sort_keys=True,separators=(',',':')));return 0
if __name__=='__main__':raise SystemExit(main(sys.argv))
```

B5 - src/temdd/v1_adapter.py

Archivkennung F177; Ursprungspfad src/temdd/v1_adapter.py

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob d9e1b874935b2f1de47c9518ca45311131987584; SHA-256 7f6248d5ac21c79875b985ba31fb03d86108b68797a75756cc02c3113b38ee0e; 1356 Byte

```
0001  #!/usr/bin/env python3
0002  """TEMDD v1 IR adapter.
0003
0004  The v0.1 language/parser remains unchanged. This adapter produces the v1
0005  canonical IR envelope as an explicit successor representation.
0006  """
0007  from __future__ import annotations
0008
0009  import json
0010  from pathlib import Path
0011  import sys
0012
0013  ROOT = Path(__file__).resolve().parents[2]
0014  if str(ROOT) not in sys.path:
0015      sys.path.insert(0, str(ROOT))
0016
0017  from tools.qikvrt_temdd import parse
0018
0019  SEMANTIC_CONTRACT = [
0020      "T13_CAUSAL_BINDING",
0021      "T14_EVIDENCE_NON_TRANSFER",
0022      "T15_EFFECT_CONSTRUCTION",
0023      "T16_CONFORMANCE_BINDING",
0024  ]
0025
0026  def adapt(text: str) -> dict:
0027      old = parse(text)
0028      return {
0029          "schema": "temdd_ir_v1",
0030          "ir_version": "1",
0031          "language_version": old["version"],
0032          "authority": old["authority"],
0033          "subject": old["subject"],
0034          "request": old["request"],
0035          "handlers": old["handlers"],
0036          "dod": old["dod"],
0037          "semantic_contract": list(SEMANTIC_CONTRACT),
0038      }
0039
0040  def main(argv) -> int:
0041      if len(argv) != 2:
0042          return 64
0043      try:
0044          ir = adapt(Path(argv[1]).read_text(encoding="utf-8"))
0045      except (OSError, ValueError) as exc:
0046          print("BLOCK " + str(exc), file=sys.stderr)
0047          return 2
0048      print(json.dumps(ir, sort_keys=True, separators=(",", ":")))
0049      return 0
0050
0051  if __name__ == "__main__":
0052      raise SystemExit(main(sys.argv))
```

B6 - spec/temdd/TEMDD_Syntax_V0_1.ebnf

Archivkennung F169; Ursprungspfad spec/temdd/TEMDD_Syntax_V0_1.ebnf

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 22ed411886a77c89e925c24d51e6c44bf5e24b84; SHA-256 dbfe62dc5e9fce8b91d06cc984ba9760fa90a67fff8961c5f86b763606148b6a; 1567 Byte

```
0001  (* TEMDD executable bootstrap grammar v0.1 *)
0002  program = "temdd", version, ";", { declaration } ;
0003  declaration = authority | subject | request | handler | dod ;
0004  authority = "authority", identifier, "=", string, ";" ;
0005  subject = "subject", identifier, "{", "repository", "=", string, ";", "binding", "=", "exact", ";", "}" ;
0006  request = "request", identifier, "{", "target", "=", identifier, ";", "}" ;
0007  handler = "on", eventkind, "{", { statement }, "}" ;
0008  eventkind = "event" | "blocker" ;
0009  statement = follow | classify | learn | execute | hold | effect ;
0010  follow = "follow", "exact", ";" ;
0011  classify = "classify", "causal", ";" ;
0012  learn = "learn", "smallest_sound_successor", ";" ;
0013  execute = "execute", "successor", ";" ;
0014  hold = "hold", identifier, "until", identifier, ";" ;
0015  effect = "effect", identifier, "{", "require", "authority", ";", "require", "validation", ";", "commit",
      ";", "readback", ";", "}" ;
0016  dod = "until", "{", predicate, { "&&", predicate }, ";", "}" ;
0017  predicate = identifier ;
0018  version = digit, { digit | "." } ;
0019  identifier = letter, { letter | digit | "_" | "-" } ;
0020  string = '"', { character }, '"' ;
0021  letter = "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P" |
      "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X" | "Y" | "Z" | "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
       | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" | "y" |
      "z" ;
0022  digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;
0023  character = letter | digit | " " | "/" | "-" | "_" | "." | ":" ;
```

B7 - next/core/src/qikvrt_exchange.c

Archivkennung F131; Ursprungspfad next/core/src/qikvrt_exchange.c

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 5249c6321f58331ed2ba18082617f160306acbd4; SHA-256 eea181a8a0d396cc81ebf8ca38f06960429539f8dfe79b611eb21a98d1e7f6f9; 5656 Byte

```
0001  /* SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0002   * Copyright 2026 Ingolf Lohmann. Implementation: OpenAI Codex. */
0003  #include "qikvrt_transputer.h"
0004  #include <string.h>
0005
0006  static int route_valid(const unsigned char *p) {
0007      return p && memcmp(p,"QXT2",4)==0 && p[4]<=7U && p[5]<=7U &&
0008          p[6]>=1U && p[6]<=3U && p[7]==0U && qikvrt_get32(p+136)<=QIKVRT_MAX_MESSAGE;
0009  }
0010  int qikvrt_route(unsigned char *route,const unsigned char *source,
0011      const unsigned char *destination,const unsigned char *subject,
0012      unsigned char sl,unsigned char dl,unsigned char codec,const unsigned char *body,unsigned int length)
      {
0013      if(!route || !source || !destination || !subject || (length && !body) ||
0014         length>QIKVRT_MAX_MESSAGE || sl>7U || dl>7U || codec<1U || codec>3U)return 0;
0015      memcpy(route,"QXT2",4);route[4]=sl;route[5]=dl;route[6]=codec;route[7]=0;
0016      memcpy(route+8,source,32);memcpy(route+40,destination,32);memcpy(route+72,subject,32);
0017      qikvrt_sha256(body,length,route+104);qikvrt_put32(route+136,(qikvrt_u32)length);memcpy(route+140,rout
      e+104,32);return 1;
0018  }
0019  int qikvrt_send(const unsigned char *route,const unsigned char *body,unsigned int length,
0020      const qikvrt_wire_frame *key,unsigned char *workspace,unsigned int capacity,
0021      qikvrt_emit_fn emit,void *context) {
0022      unsigned char *payload,digest[32];
0023      unsigned int chunks,index,offset,n,written; qikvrt_wire_frame f;
0024      if(!route_valid(route) || !key || !workspace || !emit || (length && !body) ||
0025         length>QIKVRT_MAX_MESSAGE || qikvrt_get32(route+136)!=(qikvrt_u32)length ||
0026         capacity<QIKVRT_WIRE_MAX_FRAME)return 0;
0027      qikvrt_sha256(body,length,digest);if(memcmp(digest,route+104,32)!=0)return 0;
0028      chunks=length ? (length-1U)/QIKVRT_CHUNK_BYTES+1U : 1U;
0029      f=*key;f.flags=1;f.chunk_count=(qikvrt_u32)chunks;payload=workspace+84;
0030      for(index=0;index<chunks;++index) {
0031          offset=index*QIKVRT_CHUNK_BYTES;n=length-offset;
0032          if(n>QIKVRT_CHUNK_BYTES)n=QIKVRT_CHUNK_BYTES;
0033          memcpy(payload,route,QIKVRT_ROUTE_BYTES);
0034          if(n)memcpy(payload+QIKVRT_ROUTE_BYTES,body+offset,n);
0035          f.chunk_index=(qikvrt_u32)index;f.payload_length=QIKVRT_ROUTE_BYTES+n;
0036          if(!qikvrt_wire_encode(&f,payload,workspace,capacity,&written) ||
0037             !emit(context,workspace,written))return 0;
0038      }
0039      return 1;
0040  }
0041  int qikvrt_receiver_init(qikvrt_receiver *r,unsigned char *storage,unsigned int capacity,
0042      const unsigned char *source,const unsigned char *destination,const unsigned char *subject,
0043      unsigned char sl,unsigned char dl) {
0044      if(!r || !storage || !source || !destination || !subject || sl>7U || dl>7U)return 0;
0045      memset(r,0,sizeof(*r));r->storage=storage;r->capacity=capacity;
0046
      memcpy(r->expected,source,32);memcpy(r->expected+32,destination,32);memcpy(r->expected+64,subject,32);
0047      r->source_layer=sl;r->destination_layer=dl;return 1;
0048  }
0049  static int same_key(const qikvrt_wire_frame *a,const qikvrt_wire_frame *b) {
0050      return a->session_id==b->session_id && a->nonce==b->nonce && a->source_node==b->source_node &&
0051          a->virtual_time_hi==b->virtual_time_hi && a->virtual_time_lo==b->virtual_time_lo &&
0052          a->message_id==b->message_id && a->chunk_count==b->chunk_count &&
0053          a->type==b->type && a->direction==b->direction && a->d0==b->d0 && a->d4==b->d4;
0054  }
0055  static int fail(qikvrt_receiver *r){r->failed=1;return 0;}
0056  int qikvrt_receive(qikvrt_receiver *r,const unsigned char *wire,unsigned int n) {
0057      qikvrt_wire_frame f; qikvrt_u32 bit,mask;
0058      unsigned int total,index,offset,size,chunks; unsigned char digest[32]; const unsigned char *payload;
0059      if(!r || r->failed)return 0;

0060      if(!qikvrt_wire_view(wire,n,&f,&payload) ||
0061         f.flags!=1U || f.payload_length<QIKVRT_ROUTE_BYTES ||
0062         !route_valid(payload) || memcmp(payload+8,r->expected,96)!=0 ||
0063         payload[4]!=r->source_layer || payload[5]!=r->destination_layer)return fail(r);
```

```
0064      if(f.type==1U && memcmp(payload+104,payload+140,32)!=0)return fail(r);
0065      total=(unsigned int)qikvrt_get32(payload+136);
0066      chunks=total ? (total-1U)/QIKVRT_CHUNK_BYTES+1U : 1U;
0067      if(total>r->capacity || f.chunk_count!=(qikvrt_u32)chunks)return fail(r);
0068      index=(unsigned int)f.chunk_index;offset=index*QIKVRT_CHUNK_BYTES;size=total-offset;
0069      if(size>QIKVRT_CHUNK_BYTES)size=QIKVRT_CHUNK_BYTES;
0070      if(f.payload_length!=(qikvrt_u32)(QIKVRT_ROUTE_BYTES+size))return fail(r);
0071      if(!r->started) {r->key=f;memcpy(r->route,payload,QIKVRT_ROUTE_BYTES);r->started=1;r->length=total;}
0072      else if(!same_key(&r->key,&f) || memcmp(r->route,payload,QIKVRT_ROUTE_BYTES)!=0)return fail(r);
0073      bit=(qikvrt_u32)1<<index;
0074      if(r->seen&bit) {
0075          if(size && memcmp(r->storage+offset,payload+QIKVRT_ROUTE_BYTES,size)!=0)return fail(r);
0076          return 3;
0077      }
0078      if(size)memcpy(r->storage+offset,payload+QIKVRT_ROUTE_BYTES,size);
0079      r->seen|=bit;mask=((qikvrt_u32)1<<chunks)-(qikvrt_u32)1;
0080      if(r->seen!=mask)return 1;
0081      qikvrt_sha256(r->storage,total,digest);
0082      if(memcmp(digest,r->route+104,32)!=0)return fail(r);
0083      r->complete=1;return 2;
0084  }
0085  size_t qikvrt_receiver_size(void){return sizeof(qikvrt_receiver);}
0086  size_t qikvrt_receiver_alignment(void) {
0087      struct probe {char prefix;qikvrt_receiver value;};
0088      return offsetof(struct probe,value);
0089  }
0090  unsigned int qikvrt_receiver_length(const qikvrt_receiver *r){return r && r->complete && !r->failed ?
      r->length : 0U;}
0091  unsigned char qikvrt_receiver_codec(const qikvrt_receiver *r){return r && r->complete && !r->failed ?
      r->route[6] : 0U;}
0092
0093  void qikvrt_receiver_correlation(const qikvrt_receiver *r,unsigned char out[32]) {
0094      if(out && r && r->complete && !r->failed)memcpy(out,r->route+140,32);
0095  }
```

B8 - next/core/src/qikvrt_kernel.c

Archivkennung F132; Ursprungspfad next/core/src/qikvrt_kernel.c

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 455a88356cce80184afe6d118d2e915d4584f9d6; SHA-256 6b57865464a9ff9acf0f55daa29516163396b6f156582026ad162f9ac87b46d2; 2019 Byte

```
0001  /* SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0002   * Copyright 2026 Ingolf Lohmann. Implementation: OpenAI Codex. */
0003  #include "qikvrt_transputer.h"
0004  qikvrt_u32 qikvrt_get32(const unsigned char *p) {
0005      return ((qikvrt_u32)p[0]<<24U)|((qikvrt_u32)p[1]<<16U)|
0006             ((qikvrt_u32)p[2]<<8U)|(qikvrt_u32)p[3];
0007  }
0008  void qikvrt_put32(unsigned char *p, qikvrt_u32 v) {
0009      p[0]=(unsigned char)(v>>24U); p[1]=(unsigned char)(v>>16U);
0010      p[2]=(unsigned char)(v>>8U); p[3]=(unsigned char)v;
0011  }
0012  qikvrt_u32 qikvrt_boolean_lut(qikvrt_u32 a, qikvrt_u32 b, unsigned char lut) {
0013      return ((~a&~b)&((qikvrt_u32)0-(qikvrt_u32)(lut&1U))) |
0014          ((~a&b)&((qikvrt_u32)0-(qikvrt_u32)((lut>>1U)&1U))) |
0015          ((a&~b)&((qikvrt_u32)0-(qikvrt_u32)((lut>>2U)&1U))) |
0016          ((a&b)&((qikvrt_u32)0-(qikvrt_u32)((lut>>3U)&1U)));
0017  }
0018  qikvrt_output qikvrt_evaluate(qikvrt_input i) {
0019      qikvrt_output o;
0020      o.state=QIKVRT_HOLD;
0021      if(i.binding==1U && i.authority==1U && i.distinction==1U &&
0022         i.drift==0U && i.lut<=15U && i.requested<=QIKVRT_CONTINUE) o.state=i.requested;
0023      o.value_valid=(unsigned char)(o.state==QIKVRT_CONTINUE);
0024      o.value=o.value_valid ? qikvrt_boolean_lut(i.a,i.b,i.lut) : (qikvrt_u32)0;
0025      return o;
0026  }
0027  void qikvrt_evaluate_bytes(const unsigned char *in, unsigned char *out) {
0028      qikvrt_input i; qikvrt_output o;
0029      if(!in || !out) return;
0030      i.a=qikvrt_get32(in); i.b=qikvrt_get32(in+4);
0031      i.lut=in[8]; i.requested=in[9]; i.binding=in[10];
0032      i.authority=in[11]; i.distinction=in[12]; i.drift=in[13];
0033      o=qikvrt_evaluate(i); qikvrt_put32(out,o.value); out[4]=o.state; out[5]=o.value_valid;
0034  }
0035  int qikvrt_effect_to_wire(unsigned char internal, unsigned char *wire) {
0036      static const unsigned char map[5]={0,1,4,2,3};
0037      if(!wire || internal>4U) return 0;
0038      *wire=map[internal]; return 1;
0039  }
0040  int qikvrt_effect_from_wire(unsigned char wire, unsigned char *internal) {
0041      static const unsigned char map[5]={0,1,3,4,2};
0042      if(!internal || wire>4U) return 0;
0043      *internal=map[wire]; return 1;
0044  }
```

B9 - hardware/vhdl/qikvrt_metatransistor_pkg.vhd

Archivkennung F174; Ursprungspfad hardware/vhdl/qikvrt_metatransistor_pkg.vhd

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob ef8de1403696a6343bef2c36e53e02568854681d; SHA-256 d4abe16e65600c4b4a54b6dbaab9412ad03b1df231da1e44a4e4ccfd98e5cb49; 1619 Byte

```
0001  -- SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0002  -- Copyright 2026 Ingolf Lohmann.
0003  library ieee;
0004  use ieee.std_logic_1164.all;
0005
0006  package qikvrt_metatransistor_pkg is
0007    subtype qikvrt_state_t is std_logic_vector(1 downto 0);
0008    type qikvrt_state_vector_t is array (natural range <>) of qikvrt_state_t;
0009
0010    constant QIKVRT_OBSERVE  : qikvrt_state_t := "00";
0011    constant QIKVRT_HOLD     : qikvrt_state_t := "01";
0012    constant QIKVRT_CONTINUE : qikvrt_state_t := "10";
0013    constant QIKVRT_RESERVED : qikvrt_state_t := "11";
0014
0015    function normalize_state(
0016      requested_state      : qikvrt_state_t;
0017      binding_valid        : std_logic;
0018      authority_valid      : std_logic;
0019      parent_child_differ  : std_logic;
0020      drift_detected       : std_logic
0021    ) return qikvrt_state_t;
0022  end package;
0023
0024  package body qikvrt_metatransistor_pkg is
0025    function normalize_state(
0026      requested_state      : qikvrt_state_t;
0027      binding_valid        : std_logic;
0028      authority_valid      : std_logic;
0029      parent_child_differ  : std_logic;
0030      drift_detected       : std_logic
0031    ) return qikvrt_state_t is
0032    begin
0033      -- Only a strong '0' establishes drift freedom; every other value holds.
0034      if binding_valid /= '1' or
0035         authority_valid /= '1' or
0036         parent_child_differ /= '1' or
0037         drift_detected /= '0' then
0038        return QIKVRT_HOLD;
0039      end if;
0040
0041      case requested_state is
0042        when QIKVRT_OBSERVE  => return QIKVRT_OBSERVE;
0043        when QIKVRT_HOLD     => return QIKVRT_HOLD;
0044        when QIKVRT_CONTINUE => return QIKVRT_CONTINUE;
0045        when others          => return QIKVRT_HOLD;
0046      end case;
0047    end function;
0048  end package body;
```

B10 - hardware/vhdl/qikvrt_metatransistor.vhd

Archivkennung F173; Ursprungspfad hardware/vhdl/qikvrt_metatransistor.vhd

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob ee6303ebe6a3df90303ca8a568d427ea5bd41f78; SHA-256 dc5c6f40dea9d70bf21fa320dbc5f775756d681f80cf7c4e68adf5133fd2e886; 1647 Byte

```
0001  -- SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0002  -- Copyright 2026 Ingolf Lohmann.
0003  library ieee;
0004  use ieee.std_logic_1164.all;
0005  use work.qikvrt_metatransistor_pkg.all;
0006
0007  entity qikvrt_metatransistor is
0008    port (
0009      clk_i                 : in  std_logic;
0010      reset_i               : in  std_logic;
0011      enable_i              : in  std_logic;
0012      binding_valid_i       : in  std_logic;
0013      authority_valid_i     : in  std_logic;
0014      parent_child_differ_i : in  std_logic;
0015      drift_detected_i      : in  std_logic;
0016      requested_state_i     : in  qikvrt_state_t;
0017      emitted_state_o       : out qikvrt_state_t;
0018      receipt_valid_o       : out std_logic;
0019      pass_o                : out std_logic;
0020      final_pass_o          : out std_logic;
0021      effect_ack_done_o     : out std_logic
0022    );
0023  end entity;
0024
0025  architecture rtl of qikvrt_metatransistor is
0026    signal state_q         : qikvrt_state_t := QIKVRT_OBSERVE;
0027    signal receipt_valid_q : std_logic := '0';
0028  begin
0029    process (clk_i)
0030    begin
0031      if rising_edge(clk_i) then
0032        if reset_i = '1' then
0033          state_q         <= QIKVRT_OBSERVE;
0034          receipt_valid_q <= '0';
0035        elsif enable_i = '1' then
0036          state_q <= normalize_state(
0037            requested_state_i,
0038            binding_valid_i,
0039            authority_valid_i,
0040            parent_child_differ_i,
0041            drift_detected_i
0042          );
0043          receipt_valid_q <= '1';
0044        else
0045          receipt_valid_q <= '0';
0046        end if;
0047      end if;
0048    end process;
0049
0050    emitted_state_o   <= state_q;
0051    receipt_valid_o   <= receipt_valid_q;
0052    pass_o            <= '0';
0053    final_pass_o      <= '0';
0054    effect_ack_done_o <= '0';
0055  end architecture;
```

B11 - hardware/vhdl/tb_qikvrt_metatransistor.vhd

Archivkennung F175; Ursprungspfad hardware/vhdl/tb_qikvrt_metatransistor.vhd

Commit c6a53e200c4835a4f871d8496feb81d953610831; Tree f1640191ec25495c90e997b38e77490995808b1c

Git-Blob 7d068532b9c0a59ff736d415b704117e67d48733; SHA-256 ba92c7cf916598be8303382cd37abf2dff2ad76d5aba77db480e1030406df3c9; 4987 Byte

```
0001  -- SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0002  -- Copyright 2026 Ingolf Lohmann.
0003  library ieee;
0004  use ieee.std_logic_1164.all;
0005  use std.env.all;
0006  use work.qikvrt_metatransistor_pkg.all;
0007
0008  entity tb_qikvrt_metatransistor is
0009  end entity;
0010
0011  architecture test of tb_qikvrt_metatransistor is
0012    signal clk                 : std_logic := '0';
0013    signal reset               : std_logic := '1';
0014    signal enable              : std_logic := '0';
0015    signal binding_valid       : std_logic := '1';
0016    signal authority_valid     : std_logic := '1';
0017    signal parent_child_differ : std_logic := '1';
0018    signal drift_detected      : std_logic := '0';
0019    signal requested_state     : qikvrt_state_t := QIKVRT_OBSERVE;
0020    signal emitted_state       : qikvrt_state_t;
0021    signal receipt_valid       : std_logic;
0022    signal pass_value          : std_logic;
0023    signal final_pass_value    : std_logic;
0024    signal effect_done_value   : std_logic;
0025  begin
0026    clk <= not clk after 5 ns;
0027
0028    dut : entity work.qikvrt_metatransistor
0029      port map (
0030        clk_i                => clk,
0031        reset_i              => reset,
0032        enable_i             => enable,
0033        binding_valid_i      => binding_valid,
0034        authority_valid_i    => authority_valid,
0035        parent_child_differ_i => parent_child_differ,
0036        drift_detected_i     => drift_detected,
0037        requested_state_i    => requested_state,
0038        emitted_state_o      => emitted_state,
0039        receipt_valid_o      => receipt_valid,
0040        pass_o               => pass_value,
0041        final_pass_o         => final_pass_value,
0042        effect_ack_done_o    => effect_done_value
0043      );
0044
0045    stimulus : process
0046      variable requested     : qikvrt_state_t;
0047      variable expected      : qikvrt_state_t;
0048      variable checked_cases : natural := 0;
0049    begin
0050      -- Regression for the originally reported fail-open drift input.
0051      assert normalize_state(QIKVRT_CONTINUE, '1', '1', '1', 'X') = QIKVRT_HOLD
0052        report "Unknown drift must hold rather than continue" severity failure;
0053
0054      -- Exhaust the function's finite std_logic input domain: 9**6 cases.
0055      -- The oracle permits a state only with three strong '1' guards and
0056      -- strong '0' drift. Weak, unknown, high-impedance and don't-care values
0057      -- are not evidence of a valid guard or drift freedom.
0058      for hi in std_logic loop
0059        for lo in std_logic loop
0060          requested := hi & lo;
0061          for b in std_logic loop
0062            for a in std_logic loop

0063              for p in std_logic loop
0064                for d in std_logic loop
0065                  expected := QIKVRT_HOLD;
0066                  if b = '1' and a = '1' and p = '1' and d = '0' then
```

```
0067                    case requested is
0068                      when QIKVRT_OBSERVE  => expected := QIKVRT_OBSERVE;
0069                      when QIKVRT_CONTINUE => expected := QIKVRT_CONTINUE;
0070                      when others          => expected := QIKVRT_HOLD;
0071                    end case;
0072                  end if;
0073                  assert normalize_state(requested, b, a, p, d) = expected
0074                    report "normalize_state mismatch at case " &
0075                           natural'image(checked_cases)
0076                    severity failure;
0077                  checked_cases := checked_cases + 1;
0078                end loop;
0079              end loop;
0080            end loop;
0081          end loop;
0082        end loop;
0083      end loop;
0084      assert checked_cases = 531441 severity failure;
0085      report "QIKVRT_METATRANSISTOR_NORMALIZE_CASES=531441" severity note;
0086
0087      wait for 12 ns;
0088      reset <= '0';
0089      enable <= '1';
0090      requested_state <= QIKVRT_CONTINUE;
0091      wait for 10 ns;
0092      assert emitted_state = QIKVRT_CONTINUE severity failure;
0093      assert receipt_valid = '1' severity failure;
0094
0095      drift_detected <= '1';
0096      wait for 10 ns;
0097      assert emitted_state = QIKVRT_HOLD severity failure;
0098
0099      drift_detected <= '0';
0100      requested_state <= QIKVRT_RESERVED;
0101      wait for 10 ns;
0102      assert emitted_state = QIKVRT_HOLD severity failure;
0103
0104      -- Exercise all drift values through the registered entity as well.
0105      -- Drive on the falling edge and inspect after the following rising edge.
0106      for d in std_logic loop
0107        wait until falling_edge(clk);
0108        requested_state <= QIKVRT_CONTINUE;
0109        drift_detected <= d;
0110        wait until rising_edge(clk);
0111        wait for 1 ns;
0112        expected := QIKVRT_HOLD;
0113        if d = '0' then
0114          expected := QIKVRT_CONTINUE;
0115        end if;
0116        assert emitted_state = expected
0117          report "Registered drift mismatch for " & std_logic'image(d)
0118          severity failure;
0119        assert receipt_valid = '1' severity failure;
0120        assert pass_value = '0' severity failure;
0121        assert final_pass_value = '0' severity failure;
0122        assert effect_done_value = '0' severity failure;
0123      end loop;
0124      report "QIKVRT_METATRANSISTOR_REGISTERED_DRIFT_CASES=9" severity note;

0125
0126      assert pass_value = '0' severity failure;
0127      assert final_pass_value = '0' severity failure;
0128      assert effect_done_value = '0' severity failure;
0129      stop;
0130      wait;
0131    end process;
0132  end architecture;
```

B12 - tests/test_self_host_temdd.py

Archivkennung F110; Ursprungspfad tests/test_self_host_temdd.py

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 0d444ae11b84baf790e00be062268346002b9ded; SHA-256 eaabaa8149736bf638741168e26e7c3fbb8feec48c7391edeac04556d90c160b; 10400 Byte

```
0001  # SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0002  # Copyright 2026 Ingolf Lohmann. Implementation contribution: OpenAI Codex.
0003  """Actual recovered daemon in a Git-free S1 export, never a substitute fixture."""
0004  from __future__ import annotations
0005  import hashlib
0006  import http.client
0007  import json
0008  import os
0009  from pathlib import Path
0010  import shutil
0011  import sqlite3
0012  import subprocess
0013  import sys
0014  import unittest
0015  import tempfile
0016  from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
0017  import threading
0018  import urllib.error
0019  import urllib.request
0020  from unittest.mock import patch
0021
0022  from tests.test_self_host import StandaloneTests, host, ROOT
0023
0024
0025  class SourceRecoveryTests(unittest.TestCase):
0026      def test_novnc_startup_reads_actual_loopback_bytes_under_a_dead_inherited_proxy(self):
0027          raw = b'<!doctype html><title>actual local noVNC asset fixture</title>'
0028          class Handler(BaseHTTPRequestHandler):
0029              def log_message(self, *args): pass
0030              def do_GET(self):
0031                  self.send_response(200); self.send_header('Content-Length', str(len(raw)))
0032                  self.end_headers(); self.wfile.write(raw)
0033          with tempfile.TemporaryDirectory() as tmp:
0034              package = Path(tmp); asset = package/'runtime/self-host/novnc/vnc.html'
0035              asset.parent.mkdir(parents=True); asset.write_bytes(raw)
0036              server = ThreadingHTTPServer(('127.0.0.1',0), Handler)
0037              thread = threading.Thread(target=server.serve_forever); thread.start()
0038              try:
0039                  bad = 'http://127.0.0.1:1'
0040                  with patch.dict(os.environ,
      {'HTTP_PROXY':bad,'http_proxy':bad,'NO_PROXY':'','no_proxy':''}):
0041                      with self.assertRaises(urllib.error.URLError):
0042
      urllib.request.build_opener().open('http://127.0.0.1:'+str(server.server_port)+'/vnc.html', timeout=1)
0043                      self.assertTrue(host.novnc_readback(package, server.server_port))
0044                      asset.write_bytes(b'unadmitted other asset')
0045                      self.assertFalse(host.novnc_readback(package, server.server_port))
0046              finally: server.shutdown(); server.server_close(); thread.join()
0047
0048      def test_every_recovered_source_keeps_its_original_blob_identity(self):
0049          r = json.loads((ROOT / 'evidence/self_host/SOURCE_RECOVERY_20261005.json').read_bytes())
0050          self.assertEqual(r['result'], 'EXACT_HISTORICAL_ORIGINAL_BLOBS_RECOVERED')
0051          self.assertFalse(r['boundaries']['effect_ack_done'])
0052          for f in r['files']:
0053              raw = (ROOT / f['restored_path']).read_bytes()
0054              self.assertEqual(len(raw), f['bytes'], f['restored_path'])
0055              self.assertEqual(hashlib.sha256(raw).hexdigest(), f['sha256'])
0056              self.assertEqual(hashlib.sha1(('blob '+str(len(raw))+'\0').encode()+raw).hexdigest(),
      f['git_blob_sha1'])
0057
0058      def test_native_kernel_is_in_the_export_and_no_provider_is_required(self):
0059          definition = json.loads((ROOT / host.DEFINITION).read_bytes())

0060          self.assertTrue(definition['native_terminal_daemon_included'])
0061          self.assertIn('src/qikvrt_temdd_event_ledger.py', definition['files'])
0062          self.assertEqual(definition['providers_required'], [])
0063
```

```
0064
0065  class NativeStandaloneTests(unittest.TestCase):
0066      # Reuse the existing export/process helpers. Existing reference cases are
0067      # still executed unchanged; no duplicated test daemon implements this path.
0068      setUpClass = classmethod(StandaloneTests.setUpClass.__func__)
0069      tearDownClass = classmethod(StandaloneTests.tearDownClass.__func__)
0070      save_config = StandaloneTests.save_config
0071      command = StandaloneTests.command
0072      start = StandaloneTests.start
0073      stop = StandaloneTests.stop
0074      get = StandaloneTests.get
0075      denied = StandaloneTests.denied
0076
0077      def setUp(self):
0078          StandaloneTests.setUp(self)
0079          self.config.update(terminal_profile='temdd', subject_pr=457,
      source_repository='ingolf-lohmann/qik-vrt')
0080          self.save_config()
0081
0082      def cli(self, operation, prepared=None, path=None):
0083          body = self.work / 'input.json'
0084          body.write_text(json.dumps({'schema':'qikvrt_terminal_input_v1','text':'private-test-input: Größe
       Ω'}))
0085          args = [sys.executable,'-B',str(self.export/'src/qikvrt_effect_ack_http_terminal.py'),operation,
0086
      '--root',str(self.export),'--state-dir',str(self.volume),'--repository','ingolf-lohmann/qik-vrt',
0087              '--pr','457','--expected-head',self.head,'--expected-tree',self.tree,'--manifest-sha256',self
      .pin,'--input',str(body)]
0088          if prepared:
0089              file = self.work / 'prepared.json'; file.write_bytes(host.raw_json(prepared));
      file.chmod(0o600)
0090              args += ['--prepared',str(file),'--prepare-hash',prepared['prepare_hash']]
0091          env = dict(os.environ)
0092          if path is not None: env['PATH'] = str(path)
0093          result = subprocess.run(args, capture_output=True, text=True, env=env, timeout=10)
0094          return result.returncode, json.loads(result.stdout)
0095
0096      def sql(self, query):
0097          path = self.volume / 'temdd/events.sqlite3'
0098          with sqlite3.connect(path.as_uri()+'?mode=ro', uri=True) as db:
0099              return db.execute(query).fetchall()
0100
0101      def commit_input(self):
0102          code, prepared = self.cli('durable-prepare')
0103          self.assertEqual(code, 0, prepared)
0104          self.assertEqual(self.sql('SELECT COUNT(*) FROM events'), [(0,)])
0105          code, receipt = self.cli('durable-commit', prepared)
0106          self.assertEqual(code, 0, receipt)
0107          self.assertTrue(receipt['durable_persisted'])
0108          self.assertFalse(receipt['EFFECT_ACK_DONE'])
0109          return prepared, receipt['durable_readback']
0110
0111      def test_actual_daemon_owner_commit_and_fresh_sqlite_sse_readback(self):
0112          runtime = self.start()
0113          self.assertTrue(runtime['native_terminal_daemon_available'])
0114          prepared, event = self.commit_input()
0115          self.assertEqual(self.sql('SELECT COUNT(*) FROM events'), [(1,)])
0116          self.assertEqual(event['subject']['head'], self.head)

0117          self.assertEqual(event['subject']['tree'], self.tree)
0118          self.assertFalse(event['dod'])
0119          c = http.client.HTTPConnection('127.0.0.1', self.config['terminal_port'], timeout=5)
0120          try:
0121              c.request('GET', '/api/temdd/events')
0122              response = c.getresponse(); self.assertEqual(response.status, 200)
0123              frames = []
0124              for _ in range(2):
0125                  lines = []
0126                  while True:
0127                      line = response.readline()
0128                      self.assertTrue(line)
0129                      if line == b'\n': break
0130                      lines.append(line)
0131                  frames.append(b''.join(lines))
0132              data = next(line[6:] for line in frames[1].splitlines() if line.startswith(b'data: '))
0133              self.assertEqual(json.loads(data), event)
0134          finally:
0135              if 'response' in locals(): response.close()
```

```
0136             c.close()
0137         code, duplicate = self.cli('durable-commit', prepared)
0138         self.assertEqual(code, 2)
0139         self.assertIn('ALREADY_COMMITTED', duplicate['reason'])
0140         self.assertEqual(self.sql('SELECT COUNT(*) FROM events'), [(1,)])
0141
0142     def test_native_prepare_commit_and_readback_need_neither_git_nor_gh(self):
0143         binpath = self.work/'bin'; binpath.mkdir()
0144         commands = ('node','python3','python3.12')
0145         if 'browser_runtime' in self.manifest: commands += host.BROWSER_COMMANDS
0146         for name in commands:
0147             found = shutil.which(name)
0148             if found: (binpath/name).symlink_to(found)
0149         self.assertIsNone(shutil.which('git',path=str(binpath)))
0150         self.assertIsNone(shutil.which('gh',path=str(binpath)))
0151         self.start(path=binpath)
0152         code, prepared = self.cli('durable-prepare', path=binpath); self.assertEqual(code, 0, prepared)
0153         code, event = self.cli('durable-commit', prepared, path=binpath); self.assertEqual(code, 0,
      event)
0154         code, fresh = self.cli('durable-readback', prepared, path=binpath); self.assertEqual(code, 0,
      fresh)
0155         self.assertEqual(event['durable_readback'], fresh['durable_readback'])
0156
0157     def test_sigkill_restart_keeps_epoch_native_id_and_original_event_bytes(self):
0158         self.start(); prepared, event = self.commit_input()
0159         before = self.sql('SELECT binding,source,native_id,native_digest,body,body_digest FROM events')
0160         epoch = self.get('/api/terminal')[1]['ledger_id']
0161         self.stop(abrupt=True); self.start()
0162         self.assertEqual(self.get('/api/terminal')[1]['ledger_id'], epoch)
0163         self.assertEqual(self.sql('SELECT binding,source,native_id,native_digest,body,body_digest FROM
      events'), before)
0164         code, fresh = self.cli('durable-readback', prepared); self.assertEqual(code, 0, fresh)
0165         self.assertEqual(fresh['durable_readback'], event)
0166
0167     def test_public_monitor_has_no_private_event_body_or_durable_http_writer(self):
0168         self.start(); self.commit_input()
0169         code, body = self.get('/api/terminal'); self.assertEqual(code, 200)
0170         self.assertEqual(body['state'], 'NATIVE_TEMDD_READY')
0171         self.assertFalse(body['public_event_bodies'])
0172         self.assertNotIn('private-test-input', json.dumps(body))
0173         self.assertNotEqual(self.get('/api/temdd/events')[0], 200)
0174         for path in ('/terminal/input','/api/temdd/append'):
0175             self.assertEqual(self.get(path,method='POST',url='http://127.0.0.1:'+str(self.config['termina

      l_port']))[0], 405)
0176         self.assertEqual(self.sql('SELECT COUNT(*) FROM events'), [(1,)])
0177
0178     def test_export_drift_holds_original_daemon_before_another_append(self):
0179         self.start()
0180         path = self.export/'src/qikvrt_temdd_event_ledger.py'; original = path.read_bytes()
0181         try:
0182             path.write_bytes(original+b'\n# unadmitted drift\n')
0183             code, body = self.get('/api/temdd/subject',
      url='http://127.0.0.1:'+str(self.config['terminal_port']))
0184             self.assertNotEqual(code, 200); self.assertFalse(body['dod'])
0185             self.assertEqual(self.sql('SELECT COUNT(*) FROM events'), [(0,)])
0186         finally: path.write_bytes(original)
0187
0188     def test_native_subject_is_explicit_and_wrong_bindings_never_start(self):
0189         self.config['subject_pr'] = 0; self.save_config()
0190         self.assertIn('NATIVE_TERMINAL_SUBJECT', self.denied())
0191         self.assertFalse((self.volume/'temdd/events.sqlite3').exists())
0192
0193
0194 if __name__ == '__main__': unittest.main()
```

B13 - tests/test_self_host_firefox.py

Archivkennung F111; Ursprungspfad tests/test_self_host_firefox.py

Commit 3bf52747bc94de847d390ffacabecae183aee588; Tree 96c7bb691675dcd15bb5045871201adb400074f0

Git-Blob 286a15030c9a7bd8c1c04b9493264c0a9edcb5ab; SHA-256 f4365f8c37b776be987e0f6cf08badec013c39e18751d42af24d659769a8fc29; 10825 Byte

```
0001  # SPDX-License-Identifier: PolyForm-Noncommercial-1.0.0
0002  # Copyright 2026 Ingolf Lohmann. Implementation contribution: OpenAI Codex.
0003  """Actual immutable candidate export + Firefox/X11/VNC/WebSocket runtime.
0004
0005  No provider network, no borrowed production profile, no deployed/public claim.
0006  Run only in the declared Firefox dependency carrier. Missing tools are errors.
0007  """
0008  from __future__ import annotations
0009  import base64
0010  from datetime import datetime, timezone
0011  import json
0012  import os
0013  from pathlib import Path
0014  import shutil
0015  import socket
0016  import subprocess
0017  import sys
0018  import tempfile
0019  import unittest
0020
0021  from tests.test_self_host_temdd import NativeStandaloneTests
0022  from tests.test_self_host import host, ROOT
0023
0024
0025  class FirefoxCarrierTests(NativeStandaloneTests):
0026      # The inherited native controls also execute against this browser profile.
0027      START_TIMEOUT = 60
0028      @classmethod
0029      def setUpClass(cls):
0030          cls.scratch = tempfile.TemporaryDirectory(prefix='qikvrt-firefox-exact-')
0031          cls.base = Path(cls.scratch.name)
0032          cls.source = ROOT
0033          cls.head = host.git(ROOT,'rev-parse','HEAD').decode()
0034          cls.tree = host.git(ROOT,'rev-parse','HEAD^{tree}').decode()
0035          cls.export = cls.base/'export'
0036          cls.receipt = host.freeze(ROOT,cls.export,cls.head,cls.tree,Path('/usr/share/novnc'))
0037          cls.pin = cls.receipt['manifest_sha256']
0038          cls.manifest = json.loads((cls.export/'MANIFEST.json').read_bytes())
0039
0040      def setUp(self):
0041          super().setUp()
0042          def port():
0043              with socket.socket() as s:
0044                  s.bind(('127.0.0.1',0)); return s.getsockname()[1]
0045          password = self.work/'vnc.password'; password.write_text('Test9Vnc'); password.chmod(0o600)
0046          self.config.update(terminal_profile='firefox',display=':'+str(100 + os.getpid()%5000),
0047              novnc_port=port(),vnc_port=port(),browser_password_file=str(password))
0048          self.save_config()
0049
0050      def start(self,path=None):
0051          # Build-time Git is available, but no Git/gh/provider is in runtime PATH.
0052          binpath=self.work/'runtime-bin';binpath.mkdir(exist_ok=True)
0053          for name in ('node','python3','python3.12',*host.BROWSER_COMMANDS):
0054              found=shutil.which(name)
0055              self.assertIsNotNone(found, name)
0056              if not (binpath/name).exists(): (binpath/name).symlink_to(found)
0057          self.assertIsNone(shutil.which('git',path=str(binpath)))
0058          self.assertIsNone(shutil.which('gh',path=str(binpath)))
0059          from tests.test_self_host import StandaloneTests
0060          try:
0061              return StandaloneTests.start(self,path=binpath)
0062          except AssertionError as exc:

0063              # These are isolated synthetic fixtures, never a production profile.
0064              # Retain bounded diagnostics before the private fixture is deleted.
0065              evidence = os.environ.get('QIKVRT_BROWSER_TEST_EVIDENCE')
0066              if evidence:
```

```
                p=Path(evidence);p.mkdir(parents=True,exist_ok=True)

logs={f.name:f.read_bytes()[-65536:].decode('utf-8','replace').replace('Test9Vnc','<synthetic-password>')
                    for f in (self.volume/'browser/logs').glob('*.log')}
                (p/('FAILURE_'+self._testMethodName+'.json')).write_bytes(host.raw_json({
                    'source_head':self.head,'source_tree':self.tree,'manifest_sha256':self.pin,
                    'runtime_failure':str(exc),'fixture_component_logs':logs,'effect_ack_done':False}))
            raise

    def websocket_rfb_auth(self):
        with socket.create_connection(('127.0.0.1',self.config['novnc_port']),timeout=5) as s:
            key=base64.b64encode(b'QIKVRT-test-key16').decode()
            s.sendall(('GET /websockify HTTP/1.1\r\nHost: 127.0.0.1:'+str(self.config['novnc_port'])+
                '\r\nUpgrade: websocket\r\nConnection: Upgrade\r\nSec-WebSocket-Key: '+key+
                '\r\nSec-WebSocket-Version: 13\r\nSec-WebSocket-Protocol: binary\r\n\r\n').encode())
            f=s.makefile('rb')
            header=[]
            while True:
                line=f.readline();self.assertTrue(line)
                if line==b'\r\n':break
                header.append(line)
            self.assertIn(b'101',header[0])
            def frame():
                first=f.read(2);self.assertEqual(len(first),2)
                length=first[1]&127
                if length==126:length=int.from_bytes(f.read(2),'big')
                if length==127:length=int.from_bytes(f.read(8),'big')
                self.assertLess(length,65536)
                data=f.read(length);self.assertEqual(len(data),length)
                return data
            banner=frame();self.assertEqual(banner,b'RFB 003.008\n')
            mask=b'QIKV';payload=b'RFB 003.008\n'
            s.sendall(bytes([0x82,0x80|len(payload)])+mask+bytes(v^mask[i%4] for i,v in
enumerate(payload)))
            schemes=frame();self.assertGreater(len(schemes),1)
            count=schemes[0];self.assertEqual(len(schemes),count+1)
            self.assertIn(2,schemes[1:]);self.assertNotIn(1,schemes[1:])
            f.close()
            return
{'websocket_status':101,'rfb_version':banner.decode().strip(),'vnc_authentication_required':True}

    def kernel_listeners(self):
        listeners=[]
        for name in ('tcp','tcp6'):
            for line in Path('/proc/net/'+name).read_text().splitlines()[1:]:
                row=line.split()
                if row[3] != '0A': continue
                address,port=row[1].split(':')
                self.assertEqual(name, 'tcp', 'The IPv4-only carrier must have no extra IPv6 listener')
                self.assertEqual(address, '0100007F', 'All carrier listeners must be loopback')
                listeners.append({'address':'127.0.0.1','port':int(port,16)})
        self.assertEqual({e['port'] for e in listeners}, {self.config[k] for k in
('port','terminal_port','novnc_port','vnc_port')})
        return sorted(listeners,key=lambda e:e['port'])

    def test_actual_firefox_window_novnc_websocket_and_candidate_binding(self):
        interfaces = sorted(name for _, name in socket.if_nameindex())
        self.assertEqual(interfaces, ['lo'], 'This acceptance requires the declared network-none

carrier')
        init_name=Path('/proc/1/comm').read_text().strip()
        self.assertIn(init_name, ('docker-init','tini'), 'The declared container must have a real child
reaper')
        init_binding={'name':init_name,'executable_sha256':host.digest(Path('/proc/1/exe').read_bytes())}
        runtime=self.start()
        self.assertTrue(runtime['browser_startup_verified'])
        self.assertTrue(runtime['native_terminal_daemon_available'])
        self.assertEqual(runtime['source_head'], self.head)
        self.assertFalse((self.export/'.git').exists())
        readback=self.websocket_rfb_auth()
        listeners=self.kernel_listeners()
        prepared,event=self.commit_input()
        terminal=self.get('/api/terminal')[1]
        self.assertEqual(terminal['subject'],event['subject'])
        evidence=os.environ.get('QIKVRT_BROWSER_TEST_EVIDENCE')
        if evidence:
            p=Path(evidence);p.mkdir(parents=True,exist_ok=True)
            receipt={'schema':'qikvrt-self-host-firefox-runtime-readback/v1',
```

```
0138
    'observed_at':datetime.now(timezone.utc).isoformat(),'source_head':self.head,'source_tree':self.tree,
0139                 'manifest_sha256':self.pin,'config_sha256':host.digest(self.config_path.read_bytes()),
0140                 'native_subject':terminal['subject'],'ledger_id':terminal['ledger_id'],
0141
    'native_durable_event':event,'firefox_version':self.manifest['browser_runtime']['firefox_version'],
0142
    'browser_executables':self.manifest['browser_runtime']['executables'],'transport':readback,
0143
    'firefox_navigator_window_observed':True,'git_in_runtime_path':False,'gh_in_runtime_path':False,
0144                 'native_terminal_document_title_observed':True,
0145                 'outbound_network':'DOCKER_NETWORK_NONE','observed_network_interfaces':interfaces,
0146                 'observed_listeners':listeners,'container_init':init_binding,
0147                 'scope':'ACTUAL_EXACT_CANDIDATE_RUNTIME_IN_CI_CONTAINER',
0148                 'independent_host_identity_verified':False,'public_https_readback_verified':False,
0149                 'deployment_performed':False,'effect_ack_done':False}
0150             (p/'FIREFOX_NATIVE_READBACK.json').write_bytes(host.raw_json(receipt))
0151             (p/'MANIFEST.json').write_bytes((self.export/'MANIFEST.json').read_bytes())
0152             (p/'DISTRIBUTION_PACKAGES.tsv').write_bytes(Path('/usr/local/share/qikvrt-distribution-packag
    es.tsv').read_bytes())
0153
0154     def test_crash_restart_preserves_firefox_profile_and_acknowledged_native_bytes(self):
0155         self.start(); prepared,event=self.commit_input()
0156         marker=self.volume/'browser/profile/qikvrt-test-restart-marker'
0157         marker.write_bytes(b'private fixture marker; no credentials')
0158         before=self.sql('SELECT binding,source,native_id,native_digest,body,body_digest FROM events')
0159         old_xvfb=[]
0160         for p in Path('/proc').glob('[0-9]*'):
0161             try:
0162                 if p.joinpath('comm').read_text().strip()=='Xvfb' and
    os.getpgid(int(p.name))==self.process.pid:
0163                     old_xvfb.append(int(p.name))
0164             except (OSError,ValueError): pass
0165         self.assertEqual(len(old_xvfb),1)
0166         self.stop(abrupt=True);self.start()
0167         self.assertTrue(all(not Path('/proc/'+str(pid)).exists() for pid in old_xvfb))
0168         listeners_after_restart=self.kernel_listeners()
0169         self.assertEqual(marker.read_bytes(),b'private fixture marker; no credentials')
0170         self.assertEqual(self.sql('SELECT binding,source,native_id,native_digest,body,body_digest FROM
    events'),before)
0171         code,readback=self.cli('durable-readback',prepared)
0172         self.assertEqual(code,0,readback);self.assertEqual(readback['durable_readback'],event)
0173         self.websocket_rfb_auth()

0174         evidence=os.environ.get('QIKVRT_BROWSER_TEST_EVIDENCE')
0175         if evidence:
0176             (Path(evidence)/'FIREFOX_NATIVE_RESTART_READBACK.json').write_bytes(host.raw_json({
0177
    'schema':'qikvrt-self-host-firefox-restart-readback/v1','source_head':self.head,'source_tree':self.tree,
0178                 'manifest_sha256':self.pin,'native_durable_event':event,'profile_marker_preserved':True,
0179                 'native_original_rows_preserved':True,'crashed_xvfb_pids':old_xvfb,
0180                 'crashed_xvfb_reaped':True,'observed_listeners_after_restart':listeners_after_restart,
0181                 'public_https_readback_verified':False,'effect_ack_done':False}))
0182
0183     # The inherited adversarial native controls apply to this actual browser
0184     # variant too. They exercise the same kernel, with actual GUI children.
0185
0186
0187 if __name__=='__main__':unittest.main()
```